Create service account for MongoDB Prod with Read only access	https://ctlabs.verizon.net/vztracking/browse/OSLTE-11314
Get access to all OSND project AWS accounts from SOE platform	OSLTE-11302

Q) Are A record an Alias record one and same in route53?

No, an A record and an Alias record in Amazon Route 53 are not the same thing, though an Alias record can **act as** an A record (or AAAA record) under the hood.

Key Differences

- **Standard A Record**: Maps a hostname directly to a static **IPv4 address** (e.g., `192.0.2.1`). You must manually update the IP address if the underlying resource changes.
- **Alias Record**: A proprietary extension by **Amazon Web Services (AWS)** that behaves like a smart pointer. It points your domain to an **AWS resource** (such as an **Application Load Balancer**, **CloudFront** distribution, or **S3** bucket) rather than a raw IP address.

Route53 Mapping for https://oslte-admin-preprod.pact.verizon.com/

Shell

```
# cmd
dig oslte-admin-preprod.pact.verizon.com CNAME +short

# o/p --> displays cname record mapping
oslte-admin.osndpreprod.pact.verizon.com.
```

Shell

```
# cmd
dig oslte-admin.osndpreprod.pact.verizon.com +short

# o/p --> private ip addr of nginx load balancers
63.106.192.186
63.106.192.156
63.106.192.203
```

TypeScript

```
1. User enters URL in Browser
   https://oslte-admin-preprod.pact.verizon.com
       |
       ▼
2. Corporate DNS / VPC Resolver
   Maps CNAME ——> oslte-admin.osndpreprod.pact.verizon.com which is in aws hosted
       zone "osndpreprod.pact.verizon.com"
       |
       ▼
3. Route 53 Latency & Health Check Engine
   Hosted Zone: osndpreprod.pact.verizon.com
   oslte-admin.osndpreprod.pact.verizon.com
       |
       ▼
   va-oslte-admin.osndpreprod.pact.verizon.com (US-East) (va --> N. Virginia)
                       OR
   oslte-admin.osndpreprod.pact.verizon.com
       |
       ▼
   or-oslte-admin.osndpreprod.pact.verizon.com (US-West) (or --> Oregon)
```

```
   Evaluates health of US-East (1a75f9de...) vs US-West (ec7613f2...)
   Assume It Selects ——> va-oslte-admin.osndpreprod.pact.verizon.com (US-East)
       |
       ▼
4. Route 53 Regional Alias A Record
   Resolves va-oslte-admin.osndpreprod.pact.verizon.com
   Alias to DNS of Load Balancer with prefix "dualstack."——>
dualstack.internal-k8s-nginx-nginxing-973df86a0a-1331994892.us-east-1.elb.amazonaw
s.com

DNS Name of Load Balancer:
internal-k8s-nginx-nginxing-973df86a0a-1331994892.us-east-1.elb.amazonaws.com
       |
       ▼
5. Final IP Resolution
   Returns Private IPs directly to client:
   • 63.106.192.186
   • 63.106.192.156
   • 63.106.192.203
       |
       ▼
6. Request Lands on NGINX Load Balancer (us-east-1)
   Routes packet through EKS Ingress ——> spog-admin-app Pod (Port 8080)
```

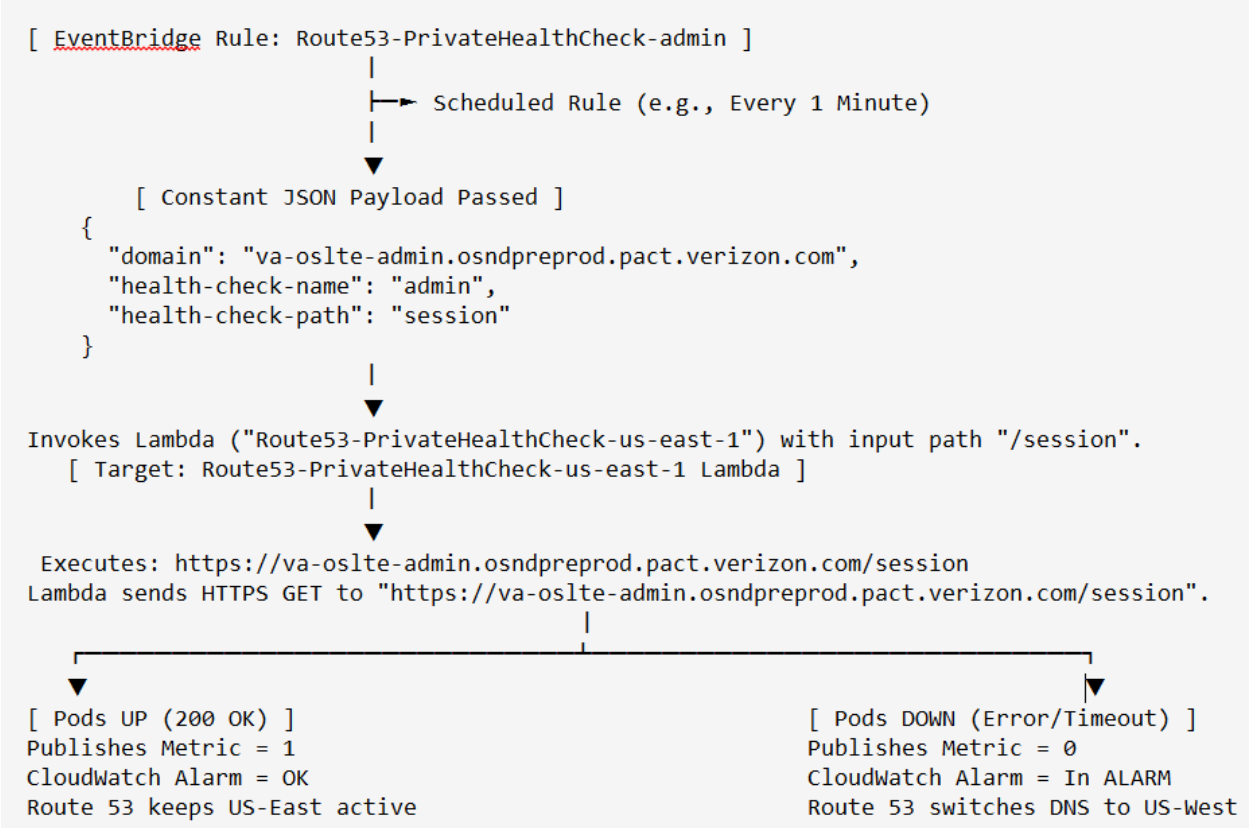
Q) Explain to me how a health check works, what it is doing to identify if east is healthy or not ?

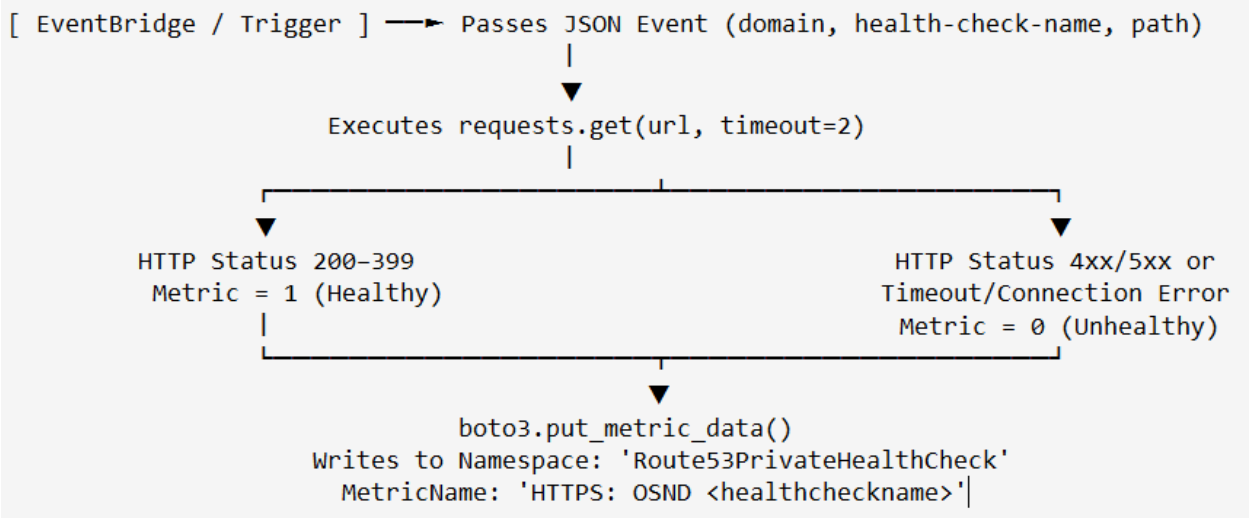
Here is the exact step-by-step mechanism of how AWS evaluates us-east-1 health:

Step 1: Endpoint Evaluation (The Target)

EventBridge Triggers the Health Checks

EventBridge acts as the cron scheduler that executes the health check Lambda automatically on a recurring timer:





Behind the scenes, an internal monitor makes HTTPS requests to your us-east-1 health endpoint (e.g.,https://va-oslte-admin.osndpreprod.pact.verizon.com/)

Step 1: Dynamic Inputs from the Trigger: The function extracts parameters passed in the event payload:

- `domain`: e.g., `va-oslte-admin.osndpreprod.pact.verizon.com`
- `health-check-name`: e.g., `admin`
- `health-check-path`: e.g., `session`

The HTTPS Request & Strict 2-Second Timeout:

```python
r = requests.get(url, timeout=2, verify=False)
```

- It makes an HTTPS request to your regional endpoint inside the VPC.
- `verify=False`: Ignores internal SSL/TLS certificate validation errors.
- `timeout=2`: If the load balancer or application doesn't respond within **2 seconds**, it aborts and triggers a timeout exception.

Evaluating Health (1 vs 0):

- **Healthy (`metric = 1`):** Returned if the status code is between `200` and `399`.
- **Unhealthy (`metric = 0`):** Returned if the application throws an HTTP error (e.g., `500 Internal Server Error`, `502 Bad Gateway`, `404 Not Found`).

Exception Catching (`metric = 0`): If the pods are down or unreachable, the `requests` library raises a `ConnectionError` (e.g., connection refused, DNS resolution failed, or socket timed out). The `except` block catches this error and explicitly sets `metric = 0`.

Publishing to CloudWatch: Whether the result is `1` or `0`, the function calls `cloudwatch.put_metric_data()` to publish the metric to the `Route53PrivateHealthCheck` namespace.

Step 2: Metric Generation (Route53PrivateHealthCheck)

Every minute, AWS collects the pass/fail results and generates the metric **HTTPS: OSND admin** under the Route53PrivateHealthCheck namespace:

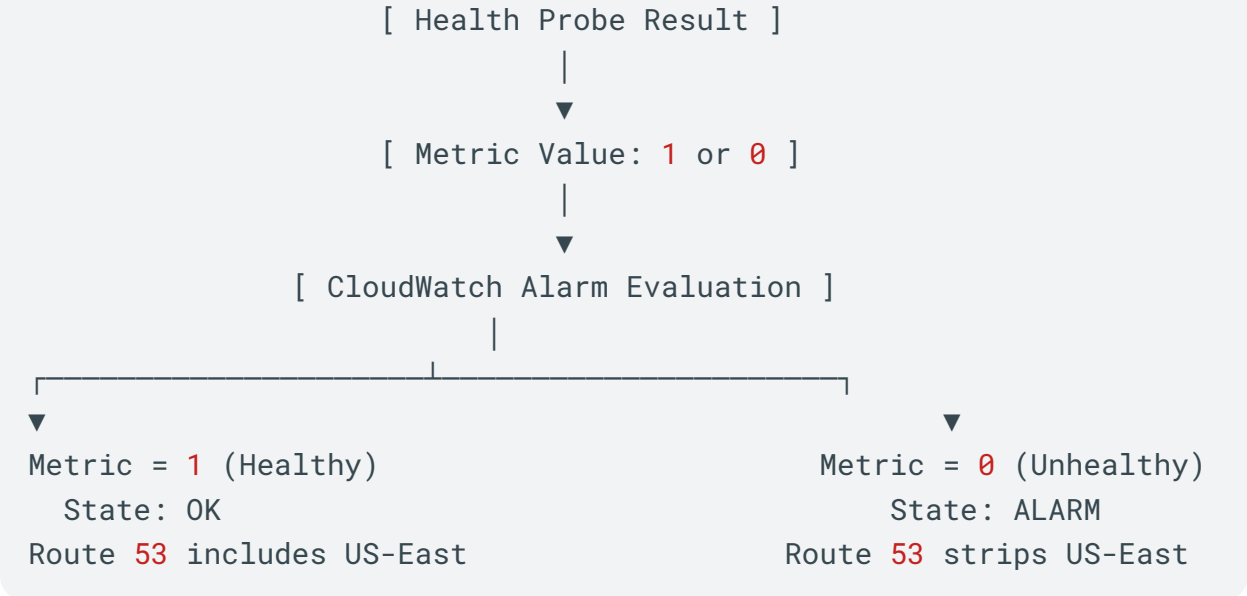
- **Metric = 1**: Endpoint responded successfully (Healthy).
- **Metric = 0**: Endpoint failed to respond or returned an error (Unhealthy).

Step 3: CloudWatch Alarm Evaluation

The CloudWatch Alarm evaluates this metric using the rule shown in your screenshot:

$$Threshold: HTTPS: OSNDadmin < 1 for 1 datapoint within 1 minute$$

Shell



1. **Normal Operations (State: OK):**
   The metric stays at 1. CloudWatch keeps the alarm in the OK state, telling Route 53 that us-east-1 is operational. Route 53 continues serving the us-east-1 load balancer IPs (63.106.192.x).

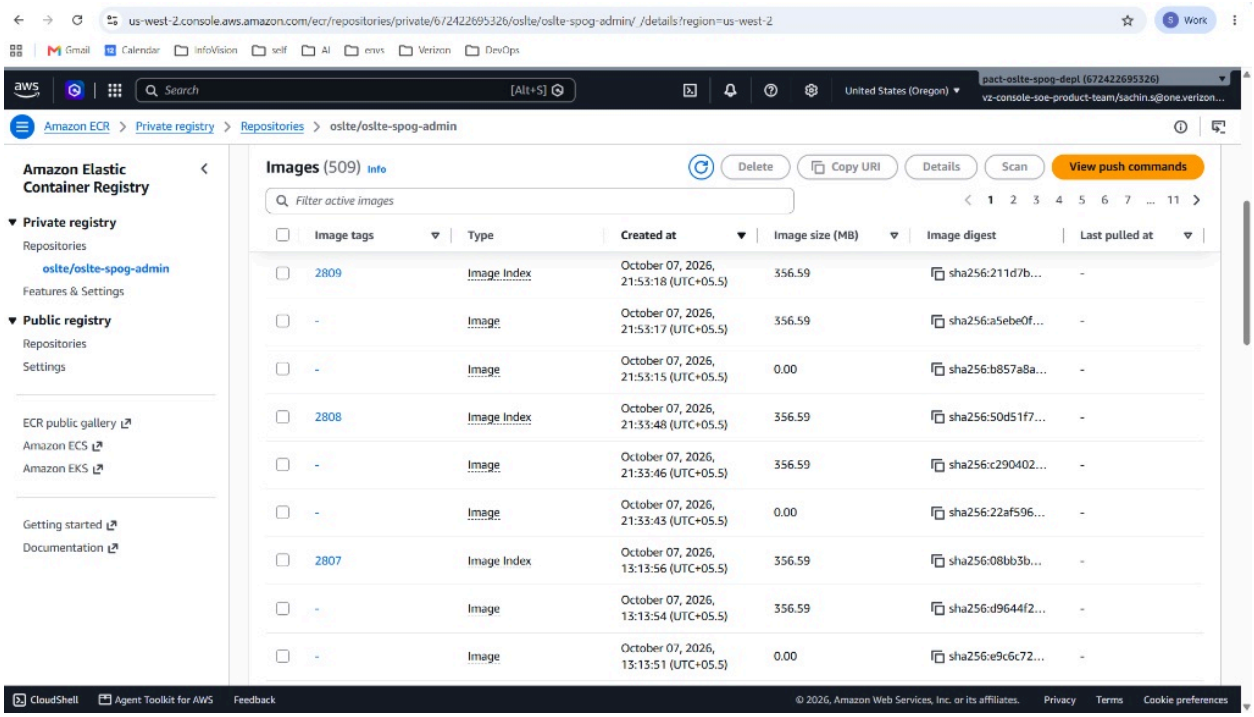
2. **Outage Detected (State: ALARM):** When your application pods go down, the health check returns an error or times out. The metric drops to 0. After **1 minute** (1 failed datapoint), CloudWatch trips the alarm into ALARM state.

Step 4: Route 53 DNS Failover Execution

Route 53 monitors this CloudWatch Alarm state in real time:

1. As soon as the alarm trips to ALARM, Route 53 marks Health Check ID 1a75f9de-cd84-4fb9-b934-3666b07b4f06 as **Unhealthy**.
2. Route 53 immediately removes va-oslte-admin.osndpreprod.pact.verizon.com (us-east-1) from its active Latency Routing Pool.
3. All subsequent DNS queries for oslte-admin.osndpreprod.pact.verizon.com are automatically directed to or-oslte-admin.osndpreprod.pact.verizon.com (us-west-2).

Q) when an image is pushed to ECR, why are there 3 of them?



In your Jenkinsfile, the docker build is triggered via `docker build` using the Docker BuildKit engine configured on your `cicd-nodes` build agent:

```
Shell
docker build -t ${OSND_DOCKER_REGISTRY}/oslte/oslte-spog-admin:${BUILD_NUMBER}
--build-arg OPEN_JDK_IMAGE=${OSND_DOCKER_REGISTRY}/oslte/openjdk11:latest .
```

When Docker BuildKit (or `docker buildx`) builds and pushes an image to Amazon ECR, it automatically generates a **multi-manifest bundle** rather than a single standalone image artifact.

Here is why 3 entries appear in ECR for every push:

1. The Image Index (OCI Manifest Index) — Tagged (2809)

- **Type:** Image Index
- **Size:** ~356 MB
- **Role:** This is the top-level parent manifest that carries your tag (${BUILD_NUMBER}). It acts as a routing pointer: when Kubernetes, ECS, or docker run pulls oslte-spog-admin:2809, Docker reads this index first to determine which underlying platform architecture to fetch.

2. The Container Image Artifact — Untagged (-)

- **Type:** Image
- **Size:** ~356 MB
- **Role:** This is the actual container layer filesystem containing your compiled Java/Gradle binaries and base OpenJDK 11 image. It is attached as a child manifest under the Image Index for the linux/amd64 architecture.

3. The Build Attestation / Provenance Manifest — Untagged (-)

- **Type:** Image
- **Size:** 0.00 MB
- **Role:** Modern Docker BuildKit automatically creates and pushes **Build Attestations** (SLSA provenance and SBOM metadata) alongside your build. ECR stores this metadata manifest as an untagged 0.00 MB child artifact under the main Image Index.

How to Prevent the Extra Untagged Entries (If Desired)

If you do not need build attestations/provenance metadata in ECR and want a single direct image push, you can disable attestations in your Jenkins pipeline.

Update the docker build step in your Jenkinsfile to set BUILDX_NO_DEFAULT_ATTESTATIONS=1:

```
JSON
stage('Dockerizing oslte-spog-admin') {
    steps {
        script {
            def utils = new com.vzc.devops.Utils()
            def awsCreds = utils.getAwsCredsForExport('osl-int',
env.AWS_DEFAULT_REGION)
            sh """
                set -e
                ${awsCreds} aws ecr get-login-password --region
${AWS_DEFAULT_REGION} | docker login --username AWS --password-stdin
${OSND_DOCKER_REGISTRY}
                docker pull ${OSND_DOCKER_REGISTRY}/oslte/openjdk11:latest

                # Disable default buildx provenance attestations
                export BUILDX_NO_DEFAULT_ATTESTATIONS=1

                docker build -t
${OSND_DOCKER_REGISTRY}/oslte/oslte-spog-admin:${BUILD_NUMBER} --build-arg
OPEN_JDK_IMAGE=${OSND_DOCKER_REGISTRY}/oslte/openjdk11:latest .
```

```
                ${awsCreds} docker push
${OSND_DOCKER_REGISTRY}/oslte/oslte-spog-admin:${BUILD_NUMBER}
            """
        }
    }
}
```

ECR Lifecycle Policy

```JSON
{
  "rules": [
    {
      "rulePriority": 1,
      "description": "Keep last 20 tagged parent images",
      "selection": {
        "tagStatus": "tagged",
        "tagPatternList": [
          "*"
        ],
        "countType": "imageCountMoreThan",
        "countNumber": 20
      },
      "action": {
        "type": "expire"
      }
    },
    {
      "rulePriority": 2,
      "description": "Expire untagged images when count exceeds 1",
      "selection": {
        "tagStatus": "untagged",
        "countType": "imageCountMoreThan",
        "countNumber": 1
      },
      "action": {
        "type": "expire"
      }
    }
  ]
}
```

Q) How does this guarantee your child images stay intact?

1. **Tagged Images:** Retains the 20 newest tagged parent images/indexes.
2. **Untagged Images:** Retains at most 1 untagged image. All other orphaned untagged child manifests will be marked for deletion.
3. **Active Child Protection:** Any child layers that belong to your latest 20 tagged parent images will **not** be deleted, as AWS ECR automatically protects actively referenced manifests.

Steps to add ECR Policy using AWS Management Console

Navigate to Amazon ECR:

- Open the AWS Management Console.
- Search for and select **Elastic Container Registry** (ECR).

Select your Repository:

- In the left sidebar, select **Private registry** > **Repositories**.
- Click on the name of your repository (e.g., `oslte/mcp-code-samples`).

Open the Lifecycle Policy tab:

- Select the **Lifecycle policy** tab near the top of the repository page.
- Click the **Create rule** or **Edit** button (or **Edit policy JSON** under the **Actions** dropdown).

Paste your Policy JSON:

- Choose the **JSON** editor view if prompted.
- Clear any existing content and paste your lifecycle policy JSON:

Save the Policy:

- Click **Save** / **Save changes**. You should see a green success banner (`Successfully updated lifecycle policy.`).

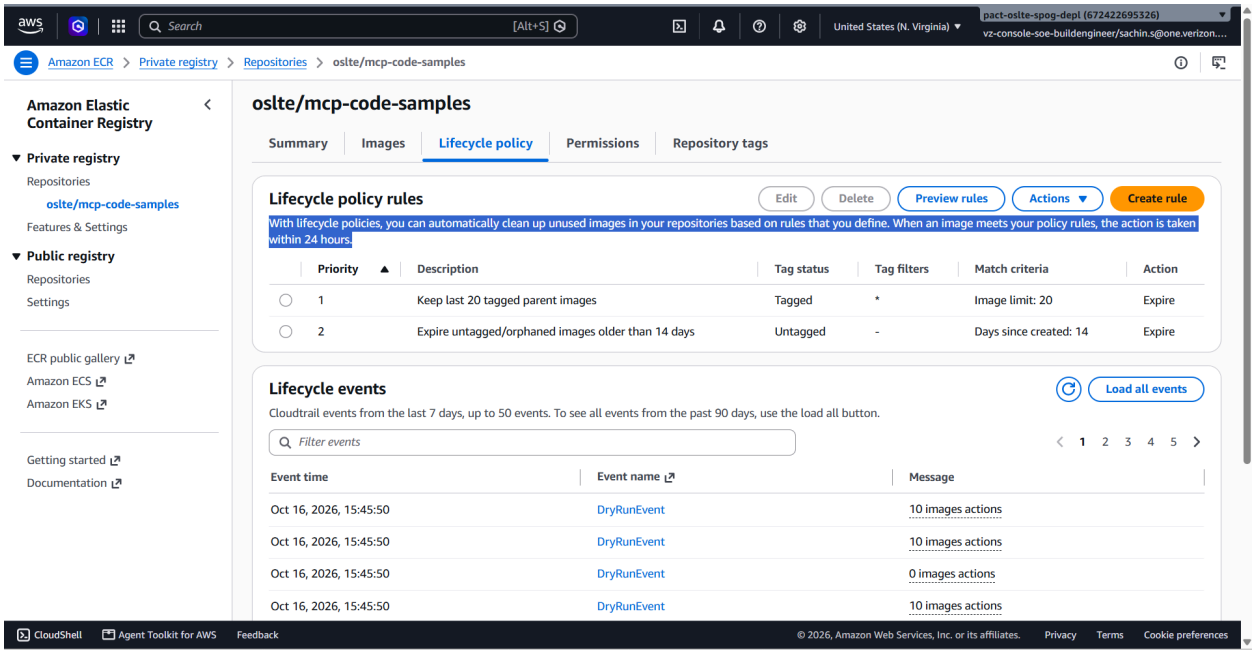
Preview the Policy Execution (Dry Run):

- On the **Lifecycle policy** tab, click **Preview rules**.
- Click on **Add** to add new preview rules or click on **Copy policy from** dropdown to copy added existing lifecycle policy
- Click on **Start preview**
- ECR will calculate which images match your criteria and show you a preview list of images that will be deleted.
- If the result or o/p looks good, click on **Apply preview as policy** button. It will add the Dry Run policy as actual ECR policy

Note:

Overwriting: Applying the preview directly as the policy will save those rules as the active production lifecycle policy for the repository.

Execution Timeline: Once applied, the actual deletion of matching images won't happen instantly; ECR will schedule and execute the cleanup within its normal 24-hour window.



There are two main reasons why your Amazon ECR repository still shows all old images despite setting up this lifecycle policy:

1. Asynchronous Execution Delay

AWS ECR Lifecycle Policies do **not** run immediately after you save them.

- ECR evaluates and applies lifecycle policy rules asynchronously, typically taking **up to 24 hours** to process the repository and delete matching images.
- As shown in your screenshot, the banner states: *"When an image meets your policy rules, the action is taken within 24 hours."*

2. Multi-Architecture Images (Image Index & Child Manifests)

Looking at your repository list:

- Your images are pushed as multi-architecture OCI Image Indexes (type **Image Index**) that reference child platform manifests (type **Image**).
- Rule 1 applies to **tagged parent images** (the Image Index entries tagged 420, 419, etc.).
- **Crucial Behavior:** An untagged child image manifest referenced by a tagged Image Index is **not** considered orphaned or untagged by ECR lifecycle policies. As long as the parent Image Index exists, ECR treats its referenced child manifests as tagged and protected. They will only be cleaned up *after* Rule 1 expires the parent tagged index.

Recommended Next Steps

1. **Verify policy predictions with a dry run:** Click the **Preview rules** button on the *Lifecycle policy* tab. This generates a dry run preview showing exactly which images will be deleted during the next evaluation cycle.
2. **Wait for ECR's execution window:** Allow up to 24 hours for ECR to execute the cleanup job automatically.
3. **Verify parent/child cleanup:**
   Once Rule 1 deletes older Image Index entries, their child manifests will become true untagged/orphaned images, allowing Rule 2 (or a subsequent evaluation cycle) to clean them up completely.

Q) There are 100+ repos. How can I apply same lifecycle policy for all?

Applying a policy manually across 100+ repositories in the AWS Console is extremely tedious. Here are the three best ways to automate applying the lifecycle policy across all your ECR repositories:

Option 1: AWS CLI & Bash Script (Quickest One-Time Solution)

Run a loop in your terminal using the AWS CLI to list all repositories and apply the policy automatically.

Step 1: Save the JSON Policy

Save your lifecycle policy as `lifecycle-policy.json`:

JSON

```
None
{
  "rules": [
    {
      "rulePriority": 1,
      "description": "Keep last 20 tagged parent images",
      "selection": {
        "tagStatus": "tagged",
        "tagPatternList": ["*"],
        "countType": "imageCountMoreThan",
        "countNumber": 20
      },
      "action": {
        "type": "expire"
      }
    },
    {
      "rulePriority": 2,
      "description": "Expire untagged/orphaned images older than 14
days",
      "selection": {
        "tagStatus": "untagged",
```

```
        "countType": "sinceImagePushed",
        "countUnit": "days",
        "countNumber": 14
      },
      "action": {
        "type": "expire"
      }
    }
  ]
}
```

Step 2: Run the Shell Script

Run this single-line loop in **Bash**, **Zsh**, or **AWS CloudShell**:

Bash

```
None
for repo in $(aws ecr describe-repositories --query
'repositories[*].repositoryName' --output text); do
  echo "Applying policy to $repo..."
  aws ecr put-lifecycle-policy \
    --repository-name "$repo" \
    --lifecycle-policy-text file://lifecycle-policy.json
done
```

Option 2: Python (Boto3 Script)

If you prefer Python or want to run this across multiple regions or accounts:

Python

```
None
import boto3
import json

policy_text = json.dumps({
    "rules": [
```

```
        {
            "rulePriority": 1,
            "description": "Keep last 20 tagged parent images",
            "selection": {
                "tagStatus": "tagged",
                "tagPatternList": ["*"],
                "countType": "imageCountMoreThan",
                "countNumber": 20
            },
            "action": {"type": "expire"}
        },
        {
            "rulePriority": 2,
            "description": "Expire untagged/orphaned images older
than 14 days",
            "selection": {
                "tagStatus": "untagged",
                "countType": "sinceImagePushed",
                "countUnit": "days",
                "countNumber": 14
            },
            "action": {"type": "expire"}
        }
    ]
})

ecr = boto3.client('ecr')

paginator = ecr.get_paginator('describe_repositories')
for page in paginator.paginate():
    for repo in page['repositories']:
        repo_name = repo['repositoryName']
        print(f"Applying policy to {repo_name}...")
        ecr.put_lifecycle_policy(
            repositoryName=repo_name,
            lifecyclePolicyText=policy_text
```

```
        )

print("Done!")
```

Option 3: Auto-apply for Future Repositories (ECR Repository Creation Templates)

To ensure **newly created repositories** automatically get this lifecycle policy applied without running manual scripts, configure an **ECR Repository Creation Template**.

1. Go to **ECR** > **Private registry** > **Settings** > **Repository creation templates**.
2. Click **Create template**.
3. Define a repository prefix filter (e.g., `*` for all new repos or `app/*` for specific ones).
4. Attach your lifecycle policy JSON directly inside the template settings.

This will automatically attach the lifecycle policy whenever a developer or CI/CD pipeline creates a new repository in ECR.

```shell
kubectl exec -it spog-portal-app-deployment-9cb746c8-42jqs -n applications -- ls -l /etc/secrets/
```

This command lists all the files mounted in the /etc/secrets/ directory of the spog-portal-app container.

```shell
kubectl exec -it spog-portal-app-deployment-9cb746c8-42jqs -n applications -- keytool -list -v -keystore /etc/secrets/oslte.spog.atlas.truststore -storepass 'Atlas123#'
```

This command retrieves and prints the full, detailed (verbose) metadata of every certificate currently stored inside the oslte.spog.atlas.truststore file mounted inside your running application pod.

```shell
openssl s_client -connect api-trusted.securitysvcs.verizon.com:443 -servername api-trusted.securitysvcs.verizon.com </dev/null 2>/dev/null | openssl x509 -noout -dates -subject -issuer
```

This command fetches the live SSL/TLS certificate from the remote server and extracts its key details into human-readable text.

To view the exact SSL certificate and certificate authority (CA) chain that `api-trusted.securitysvcs.verizon.com` is presenting, run the following `openssl` command from your terminal, workstation, or from within a test pod inside your Kubernetes cluster:

Bash

```none
openssl s_client -showcerts -connect api-trusted.securitysvcs.verizon.com:443 </dev/null
```

Key Things to Inspect in the Output

1. **Certificate Chain (`Certificate chain` section):** Look at the `s:` (Subject) and `i:` (Issuer) lines. This shows who issued the certificate:
    - **`s:` (Subject):** The endpoint name (`CN = api-trusted.securitysvcs.verizon.com`).
    - **`i:` (Issuer):** The intermediate/root CA that signed it (e.g., `CN = Verizon Internal Root CA`). If this issuer is not in Java's default `cacerts`, it causes the `PKIX path building failed` error.
2. **Verification Status (Bottom line):**
    - If you see `Verify return code: 21 (unable to verify the first certificate)` or `code: 19 (self signed certificate in chain)`, it confirms that your client environment does not trust the signing CA.

Download & Save the Certificate to a File

To extract and save the certificate bundle directly so you can view it or import it later:

Bash

```
None
openssl s_client -showcerts -connect api-trusted.securitysvcs.verizon.com:443 </dev/null 2>/dev/null | sed -ne '/-BEGIN CERTIFICATE-/,/-END CERTIFICATE-/p' > verizon-cert.crt
```

View certificate details in readable text:

Bash

```
None
openssl x509 -in verizon-cert.crt -text -noout
```

Check that `Issuer` matches your corporate CA name and that the `Validity` dates are active.

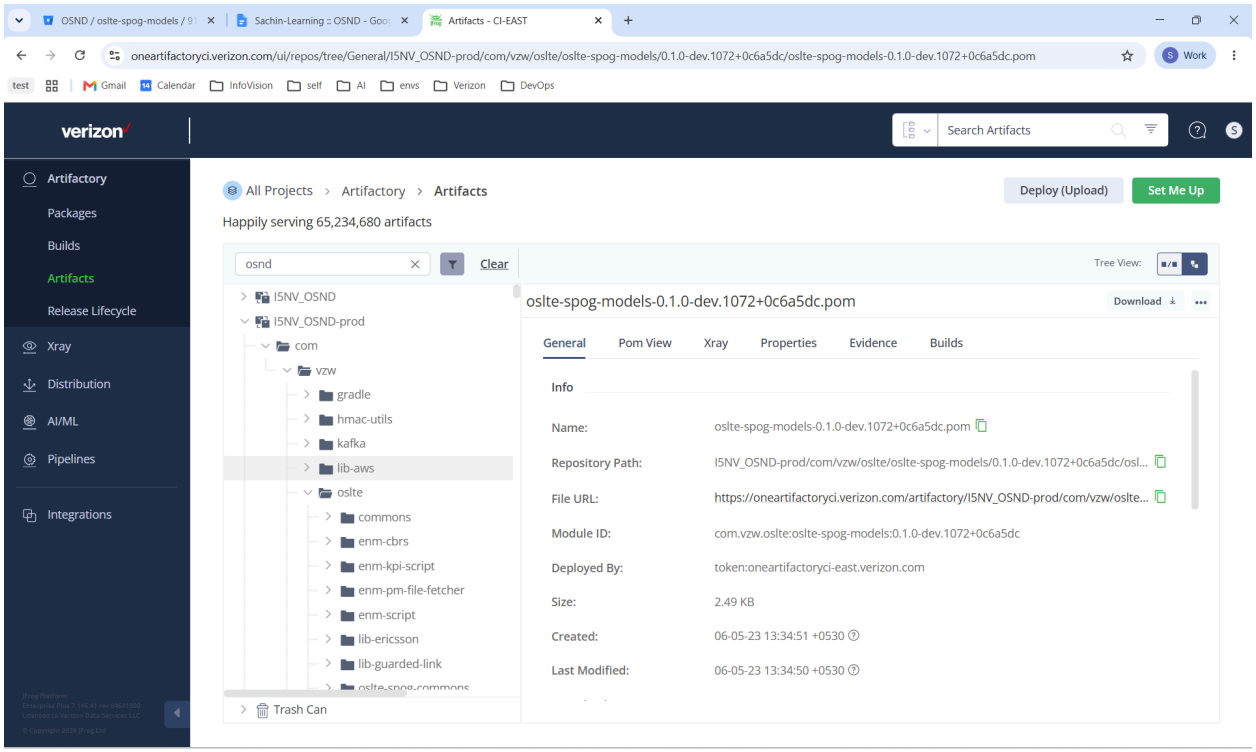
https://confluence.verizon.com/spaces/TPDSOE/pages/416157471/Digicert+SSL+Certificate+Reissuance+Renewal+Automation+Runbook

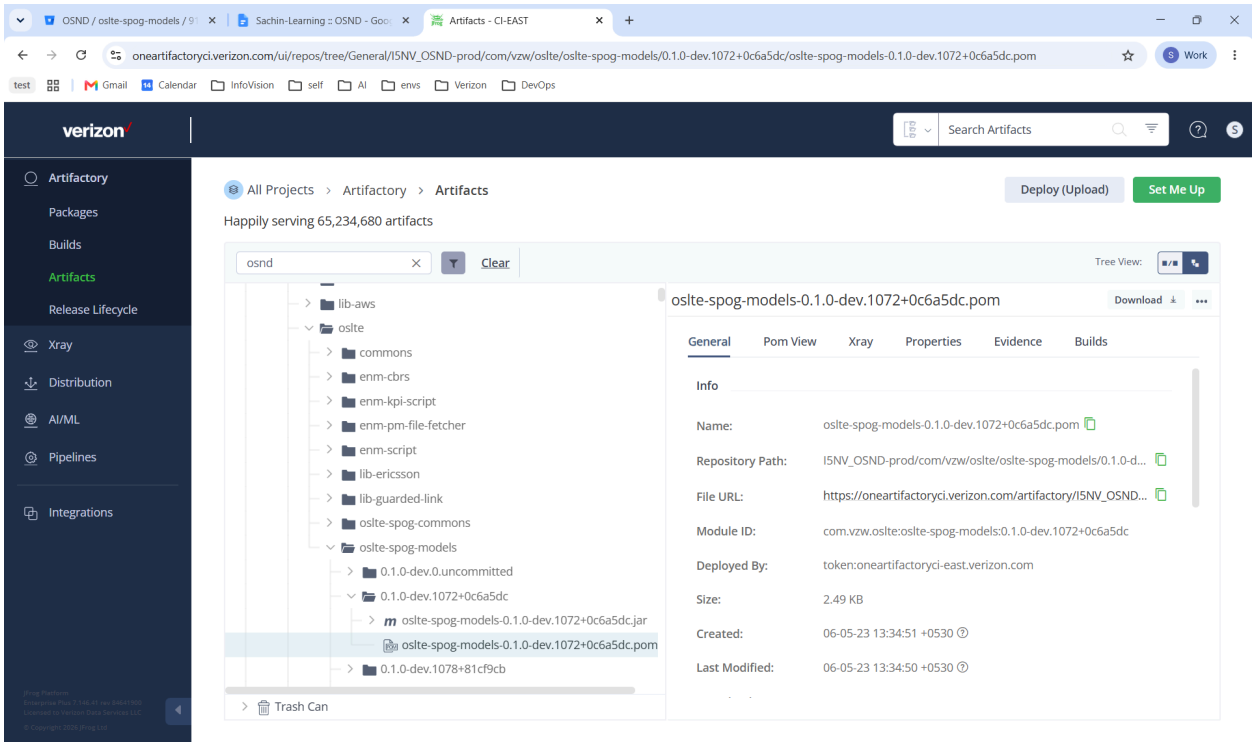
Q) How gradle versions are updated in the release branch?
JFrog Artifactory:
https://oneartifactoryci.verizon.com/ui/repos/tree/General/I5NV_OSND

https://ctlabs.verizon.net/stash/projects/POSND/repos/oslte-spog-models/browse/Jenkinsfile?at=release%2FMMP3-PI-41-Sprint-24-25

The Jenkins pipeline of release branches for oslte-spog-models will keep publishing new models when new commits are identified. But it won't be auto-updated in the release branch code base of oslte-spog-admin, oslte-spog-portal, etc. Devs will have to manually update it in the build.gradle file.

ex:
https://ctlabs.verizon.net/stash/projects/POSND/repos/oslte-spog-admin/browse/backend/build.gradle?at=refs%2Fheads%2Frelease%2FMMP3-PI-41-Sprint-24-25
oslte-spog-admin → backend → build.gradle → search for “models” and update the model version




OSND / oslte-spog-models / 9
Sachin-Learning :: OSND - Goo
Artifacts - CI-EAST
oneartifactoryci.verizon.com/ui/repos/tree/General/I5NV_OSND-prod/com/vzw/oslte/oslte-spog-models/0.1.0-dev.1072+0c6a5dc/oslte-spog-models-0.1.0-dev.1072+0c6a5dc.pom
Work
test
Gmail
Calendar
InfoVision
self
AI
envs
Verizon
DevOps
verizon
Search Artifacts
Artifactory
Packages
Builds
Artifacts
Release Lifecycle
Xray
Distribution
AI/ML
Pipelines
Integrations
All Projects
Artifactory
Artifacts
Deploy (Upload)
Set Me Up
Happily serving 65,234,680 artifacts
osnd
Clear
Tree View:
lib-aws
oslte
commons
enm-cbrs
enm-kpi-script
enm-pm-file-fetcher
enm-script
lib-ericsson
lib-guarded-link
oslte-spog-commons
oslte-spog-models
0.1.0-dev.0.uncommitted
0.1.0-dev.1072+0c6a5dc
oslte-spog-models-0.1.0-dev.1072+0c6a5dc.jar
oslte-spog-models-0.1.0-dev.1072+0c6a5dc.pom
0.1.0-dev.1078+81cf9cb
Trash Can
oslte-spog-models-0.1.0-dev.1072+0c6a5dc.pom
Download
General
Pom View
Xray
Properties
Evidence
Builds
Info
Name:
oslte-spog-models-0.1.0-dev.1072+0c6a5dc.pom
Repository Path:
I5NV_OSND-prod/com/vzw/oslte/oslte-spog-models/0.1.0-d...
File URL:
https://oneartifactoryci.verizon.com/artifactory/I5NV_OSND...
Module ID:
com.vzw.oslte:oslte-spog-models:0.1.0-dev.1072+0c6a5dc
Deployed By:
token:oneartifactoryci-east.verizon.com
Size:
2.49 KB
Created:
06-05-23 13:34:51 +0530
Last Modified:
06-05-23 13:34:50 +0530

To pinpoint whether this is another DNS resolution failure or a network connectivity timeout, run `curl` with verbose logging (`-v`) through the proxy:

Bash

```
Shell
curl -I -v -x http://63.106.224.104:3128
https://cxp-b6vv-beta2.ebiz.verizon.com/b6vv/nssit3/deviceProductMa
nagement/search
```

Look at the **`X-Squid-Error`** header in the HTTP 503 response:

1. **`X-Squid-Error: ERR_DNS_FAIL`**:
    - **Cause:** The Squid pod at `63.106.224.104` cannot resolve `cxp-b6vv-beta2.ebiz.verizon.com`.
    - **Fix:** Add `dns_nameservers 63.106.208.2` to `squid.conf` in your `proxy-config` ConfigMap or add a `hostAliases` entry in your Deployment spec.
2. **`X-Squid-Error: ERR_CONNECT_FAIL 110`**:
    - **Cause:** DNS succeeded, but the proxy pod timed out attempting to establish a TCP connection to `159.67.70.221:443`.
    - **Fix:** Check your AWS Route Tables and Security Groups attached to the Fargate pod subnet (`proxy-subnet`) to ensure outbound traffic to `159.67.0.0/16` or `159.67.70.221` on port `443` is allowed and routed through your Transit Gateway / VPN.

Q) How to identify the direction of traffic (inbound or outbound) for any fqdn?

To determine the direction, evaluate who is reaching out to whom:

1. Outbound (Egress) — *From your EC2 instance's perspective*

When you execute curl https://cxp-b6vv-nssit1.ebiz.verizon.com/ from your EC2 terminal (63.106.193.56), your machine is initiating the connection outbound to the ALB (10.74.123.193:443).

- Source: 63.106.193.56 (Your EC2)
- Destination: 10.74.123.193 (Verizon Internal ALB)
- Traffic Direction: Outbound (Egress) from EC2.

2. Inbound (Ingress) — *From the Kubernetes/ALB's perspective*

When the ALB receives that connection from your EC2 instance on port 443, it sees it coming in from 63.106.193.56.

- Source: 63.106.193.56
- Destination: 10.74.123.193
- Traffic Direction: Inbound (Ingress) to the Load Balancer/K8s Service.

Summary Checklist to Determine Direction

Command Tool	Can it show inbound vs outbound?	How to use it
nslookup	❌ No	Only translates Domain ➔ IP.
ss -tunp / netstat	✅ Yes	Shows active socket direction based on high client port vs. target port.
tcpdump	✅ Yes	Shows SYN packet flow (Source IP > Dest IP).

VPC Flow Logs	✅ Yes	Shows srcaddr and dstaddr along with action (ACCEPT/REJECT).

Steps to Test the direction of traffic

Ex: FQDN → cxp-b6vv-nssit1.ebiz.verizon.com

check resolution IP for fqdn using nslookup

```
[ec2-user@ip-63-106-193-56 bin]$ nslookup cxp-b6vv-nssit1.ebiz.verizon.com
Server:         63.106.192.2
Address:        63.106.192.2#53


Non-authoritative answer:
cxp-b6vv-nssit1.ebiz.verizon.com        canonical name =
internal-k8s-b6vvsit1-b6vvsit1-78f397e1c3-916244335.us-east-1.elb.amazonaws.com.

Name:
internal-k8s-b6vvsit1-b6vvsit1-78f397e1c3-916244335.us-east-1.elb.amazonaws.com
Address: 10.74.123.193

Name:
internal-k8s-b6vvsit1-b6vvsit1-78f397e1c3-916244335.us-east-1.elb.amazonaws.com
Address: 10.74.124.235

Name:
internal-k8s-b6vvsit1-b6vvsit1-78f397e1c3-916244335.us-east-1.elb.amazonaws.com
Address: 10.74.178.79
```

To test the connection direction and state to `cxp-b6vv-nssit1.ebiz.verizon.com` (`10.74.x.x`) using `netstat`, you first need to initiate traffic to it (for example, by running `curl` or `nc` in one terminal session) and then inspect the active network sockets in another.

Step-by-Step Diagnostic Method

1.Start the Outbound Request: Terminal 1.
Initiate a long-running connection attempt or request to one of the resolved Load Balancer IPs:

```shell
curl -v https://10.74.123.193:443
```

Verification: The command will either attempt to connect or hang/timeout, keeping the socket open long enough to capture.

2.Inspect Sockets with netstat: Terminal 2.
While the command in Terminal 1 is running, filter netstat output for the target IP (10.74.123.193):

```
Shell
netstat -anp | grep 10.74.123.193
```

Verification: You will see an active socket entry printed showing your local IP and high port connecting to the destination IP on port 443.

3.Evaluate the Direction & State: Analysis.
Analyze the output line structure:

```
None
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      1 63.106.193.56:54321     10.74.123.193:443       SYN_SENT    12345/curl
```

- **Outbound Confirmation:** Your machine (63.106.193.56) uses a random high client port (54321) to hit the standard service port (443) on the remote server (10.74.123.193).
- **Connection Status:**
  - **SYN_SENT**: Outbound TCP connection request was sent, but no response came back (indicates a firewall, Security Group drop, or missing VPC route).
  - **ESTABLISHED**: Outbound TCP handshake succeeded.

how to test direction of traffic using Socket Statistics “ss”?

The ss (socket statistics) utility determines traffic direction by analyzing **Local Address** versus **Foreign Address** along with the **TCP port numbers**.

When analyzing traffic, the system initiating the connection is assigned a **high ephemeral port** (typically 32768 to 60999 or 1024 to 65535), while the receiving server listens on a **known service port** (80, 443, 3128, 22, etc.).

Understanding ss Field Mapping

Run ss -tunp to list active TCP/UDP sockets with process names and port numbers:

- -t: Show TCP sockets
- -u: Show UDP sockets
- -n: Show numerical addresses and ports (do not resolve hostnames/service names)
- -p: Show process using the socket

Plaintext

```
None
Netid  State       Recv-Q  Send-Q    Local Address:Port      Peer
Address:Port     Process
tcp    ESTABLISHED 0       0         63.106.193.56:54321
10.74.123.193:443     users:(("curl",pid=1234,fd=3))
tcp    ESTABLISHED 0       0         10.100.124.99:3128
63.106.222.62:51902   users:(("squid",pid=5678,fd=12))
```

- **Outbound Traffic:** Your IP is in Local Address with a **high random port**, connecting to a remote service port in Peer Address.
- **Inbound Traffic:** Your IP is in Local Address with your **application's listening port**, receiving a connection from a client's high random port in Peer Address.

Step-by-Step Examples

1.Test and Inspect Outbound Traffic: Method 1.
Initiate an outbound HTTP request from your machine to an external IP (10.74.123.193:443) in one terminal:

Bash

```
None
curl -v https://10.74.123.193:443
```

In a second terminal, filter ss for destination port 443:

Bash

```
None
ss -tunp 'dport = :443'
```

Example Output:

Plaintext

```
None
Netid  State      Recv-Q  Send-Q    Local Address:Port      Peer
Address:Port     Process
tcp    SYN_SENT   0       1         63.106.193.56:58210
10.74.123.193:443     users:(("curl",pid=9120,fd=3))
```

Verification: Check Local Address:Port. The high port 58210 on your IP (63.106.193.56) proves your machine initiated an **outbound request**. State SYN_SENT confirms traffic is leaving your server (if stuck in SYN_SENT, an outbound firewall/SG is dropping SYN-ACK packets).

2.Test and Inspect Inbound Traffic: Method 2.
Start a service listening on a specific local port (such as port 8080) in one terminal:

Bash

```
None
nc -l -p 8080
```

Simulate an inbound connection from another host (or local terminal) targeting 8080, then filter ss for local port 8080:

Bash

```
None
ss -tunp 'sport = :8080'
```

Example Output:

Plaintext

```
None
Netid  State        Recv-Q  Send-Q    Local Address:Port      Peer
Address:Port     Process
tcp    ESTABLISHED  0       0         10.0.1.50:8080
63.106.222.62:51902   users:(("nc",pid=4102,fd=4))
```

Verification: Check Local Address:Port. Your machine is bound to service port 8080 while the external machine (63.106.222.62) connects via high port 51902. This confirms the connection is **inbound**.

3.Inspect Listening Sockets (Inbound Readiness): Method 3.
To see all services on your host currently open and waiting to receive **inbound** connections:

Bash

```
None
ss -tulpn
```

Example Output:

Plaintext

```
None
Netid  State    Recv-Q  Send-Q    Local Address:Port    Peer
Address:Port    Process
```

```
tcp    LISTEN   0       128       0.0.0.0:3128          0.0.0.0:*
users:(("squid",pid=1102,fd=6))
```

Verification: State LISTEN on Local Address:Port 0.0.0.0:3128 confirms the Squid proxy process is bound and ready to accept inbound requests on port 3128.

Quick Cheat Sheet for ss Directional Filtering

Command	Goal	Description
ss -tunp 'dport = :443'	Track **Outbound** HTTPS	Filters for external destination port 443
ss -tunp 'sport = :3128'	Track **Inbound** to Proxy	Filters for connections hitting local service port 3128
ss -tunp 'dst 10.74.123.193'	Track Target Host Traffic	Filters all active sockets targeting IP 10.74.123.193
ss -tulpn	List Inbound Listeners	Shows all local processes in LISTEN state

Option 1: Output the file to your home directory Redirect the logs to `~` (your home directory) where `ec2-user` has full write permissions:

```
Shell
kubectl logs ossmediator-57556994cf-7fmw4 -c kafka-plugin -n
oss-mediator > ~/pod_logs.txt
```

Option 2: Change to a writable directory first Navigate to your home directory or `/tmp` before running the command:

```
Shell
cd ~
kubectl logs ossmediator-57556994cf-7fmw4 -c kafka-plugin -n
oss-mediator > pod_logs.txt
```

Option 3: Use `sudo tee` (if you must write to `/bin`) If you specifically need the log file saved inside the current system binary directory:

```
Shell
kubectl logs ossmediator-57556994cf-7fmw4 -c kafka-plugin -n
oss-mediator | sudo tee /bin/pod_logs.txt > /dev/null
```

Upload to S3 (Cleanest method for large files)

If you have an S3 bucket in your AWS account, upload it from the browser terminal and download it via the S3 web console:

Bash

```
None
aws s3 cp /tmp/pod_logs.txt s3://YOUR-BUCKET-NAME/
```

1. Open a new Chrome tab and navigate to **S3** in the AWS Console.
2. Go into `YOUR-BUCKET-NAME`.
3. Select `pod_logs.txt` and click **Download**.

Cmds to deliver the resulting `.tar.gz` files from QA and the corresponding Prod region. Each archive contains:

- `inventory_heap.hprof` (Loadable into Eclipse MAT, IntelliJ, or VisualVM)
- `jvm_threads.tdump` (Thread execution traces)
- `jvm_flags.txt` (Heap sizes, GC algorithm like G1GC/ZGC)
- `sanitized_environment.txt` (Configs and env vars, secrets redacted)
- `pod_spec_runtime.json` & `pod_describe.txt` (Resource limits, node placement, restart status)
- `application_tail.log` (Last 3000 lines of app output)

```
POD="spog-inventory-app-deployment-6b96c98c5d-5q9d9"
CONTAINER="spog-inventory-app"

# 1. Create a local dump folder
mkdir -p inventory_dump && cd inventory_dump

# 2. Kubernetes manifests & logs
k get pod $POD -o json > pod_spec.json
k describe pod $POD > pod_describe.txt
k logs $POD -c $CONTAINER --tail=3000 > application.log

# 3. Environment & process list (with passwords/secrets masked)
k exec $POD -c $CONTAINER -- ps aux > process_list.txt
k exec $POD -c $CONTAINER -- env | sed -E
's/(PASSWORD|SECRET|KEY|TOKEN|AUTH|CREDENTIAL|PWD)=.*/\1=[REDACTED]/gI' >
sanitized_env.txt

# 4. JVM thread dump & flags
k exec $POD -c $CONTAINER -- jcmd 1 Thread.print > jvm_threads.tdump
k exec $POD -c $CONTAINER -- jcmd 1 VM.flags > jvm_flags.txt

# 5. JVM heap dump (generated, downloaded, then deleted from pod /tmp)
k exec $POD -c $CONTAINER -- jcmd 1 GC.heap_dump /tmp/heap.hprof
kubectl cp applications/$POD:/tmp/heap.hprof ./inventory_heap.hprof -c $CONTAINER
k exec $POD -c $CONTAINER -- rm -f /tmp/heap.hprof

# 6. Package it up for the dev
cd ..
tar -czvf inventory_dump_$(date +%F_%H%M).tar.gz inventory_dump/
```

MongoDB Collection Dump

commands to dump the `inventory` collection using either binary BSON (compressed) or human-readable JSON.

Option 1: Compressed BSON Archive (`mongodump` - Recommended)

This creates a compressed dump (`inventory_qa.gz`) that developers can restore directly into a local MongoDB instance using `mongorestore`.

Bash

```
None
mongodump
--uri="mongodb+srv://spogrw:W4FyYza0to0aBApG@spog-qa-pl-1.njey0.mongodb.net/spog" \
  --collection="inventory" \
  --gzip \
  --archive="inventory_collection_qa.gz"
```

Option 2: Human-Readable JSON (`mongoexport`)

If the developer wants to open the file in VS Code, inspect records, or run a `git diff` against Prod:

Bash

```
None
mongoexport
--uri="mongodb+srv://spogrw:W4FyYza0to0aBApG@spog-qa-pl-1.njey0.mongodb.net/spog" \
  --collection="inventory" \
  --out="inventory_collection_qa.json" \
  --jsonArray
```

If `mongodump` / `mongoexport` is not installed on your EC2 box

If you get `command not found: mongodump`, install the official MongoDB database tools:

On Amazon Linux 2 / RHEL / CentOS:

Bash

```
None
sudo yum install -y
https://fastdl.mongodb.org/tools/db/mongodb-database-tools-amazon2-
x86_64-100.9.4.rpm
```

On Amazon Linux 2023:

Bash

```
None
sudo dnf install -y
https://fastdl.mongodb.org/tools/db/mongodb-database-tools-amazon20
23-x86_64-100.9.4.rpm
```

On Ubuntu / Debian:

Bash

```
None
sudo apt-get install -y mongo-tools
```

If your EC2 cannot resolve MongoDB Atlas (`mongodb+srv`) or is IP-blocked

MongoDB Atlas clusters often use Network Access IP Whitelisting. If the connection hangs or returns a timeout:

1. Ensure your EC2 NAT Gateway IP or Elastic IP is added to the Atlas **Network Access / IP Access List**.
2. Alternatively, run the export command directly inside your EKS cluster where pod IPs are already whitelisted:

Bash

```
None
kubectl run mongo-dumper -n applications --rm -i --tty \
  --image=mongo:6.0 --restart=Never -- \
  mongodump
--uri="mongodb+srv://spogrw:W4FyYza0to0aBApG@spog-qa-pl-1.njey0.mongodb.ne
t/spog" \
  --collection="inventory" \
  --gzip \
  --archive=/tmp/inventory_collection_qa.gz
```

Is the below logic correct for n/w connectivity tests?

If you update the check to treat all standard HTTP response codes (100–499, plus application-level 500s) as a pass and only fail on 000 (timeout/unreachable) or proxy failures (502/503/504), both HTTP 400 and HTTP 307 will correctly report as PASS

Yes, this logic is **fundamentally correct** for layer 4/layer 7 **network reachability tests**, with one subtle proxy edge case you should be aware of regarding 500 vs 502/503/504.

Why this logic is sound for Network Connectivity

In a network connectivity / firewall validation test, your goal is to answer:

"Can traffic traverse the client → proxy → target server → complete TLS negotiation → receive a response?"

Any standard HTTP status code generated by the target server proves the complete network circuit succeeded:

- **1xx (Informational), 2xx (Success), 3xx (Redirects):**
  - E.g., 301, 302, 307: Proves DNS, routing, proxy CONNECT, TLS handshake, and remote HTTP server reached.
- **4xx (Client Errors):**
  - E.g., 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 405 Method Not Allowed: Proves full bidirectional connectivity. The request was dropped/rejected by the **target application**, not the network.
- **000:**
  - Network/transport failure: curl could not establish a connection, DNS failed, or connection timed out before any handshake could occur. This is always a network **FAIL**.

The One Catch: Who generated the 5xx status code?

When testing over an HTTP forward proxy (Squid) via HTTPS (CONNECT tunnel):

1. **For HTTPS (https://... via CONNECT tunnel):**
   - If Squid cannot establish the TCP tunnel to the remote server, Squid aborts the tunnel negotiation with its own HTTP response:
     - **503 Service Unavailable** (Squid timeout / ERR_CONNECT_FAIL 110 / ERR_DNS_FAIL)
     - **502 Bad Gateway** (Squid connection refused / ERR_CONNECT_FAIL 111)
     - **504 Gateway Timeout**
   - If the tunnel **succeeds**, Squid's role drops to passing encrypted bytes between client and server. The client then performs TLS handshake directly with the target host.

    - If the target host's backend has a bug, the **target server** will return 500 Internal Server Error (over the encrypted tunnel). This still means the **network connection was successful**.
2. **For plain HTTP (http://... via forward proxy):**
    - If an endpoint uses unencrypted http:// (like your xLPT 4G on port 8181: http://xlpt.eng.vzwcorp.com:8181/), Squid acts as a standard forward proxy rather than a blind TCP tunnel. If the destination is down, Squid will return 503 or 502.

Best-Practice Rule Set for Network Probing

HTTP Code	Source	Network Reachability Verdict
000	curl (Timeout / Network Down / Proxy Unreachable)	**FAIL**
502, **503**, **504**	Proxy failed to connect to destination host	**FAIL**
100 – 499	Remote Destination Server reached and responded	**PASS**
500, **501**, **505**	Remote Destination Server reached (application error)	**PASS**

vi test_endpoints.sh

```shell
#!/usr/bin/env bash

# ================================================================================
# Configuration
# ================================================================================
NAMESPACE="proxy"
```

```
PROXY_PORT="3128"
TIMEOUT=5

echo
"================================================================================="
echo " Fetching Dynamic Proxy Pod IP from K8s (Namespace: ${NAMESPACE})"
echo
"================================================================================="

# Retrieve live Pod IP for a Running pod in the proxy namespace
PROXY_POD_IP=$(kubectl get pods -n "${NAMESPACE}" \
  --field-selector=status.phase=Running \
  -o jsonpath='{.items[0].status.podIP}' 2>/dev/null)

if [[ -z "$PROXY_POD_IP" ]]; then
  echo "Error: Could not retrieve a running Proxy Pod IP from namespace
'${NAMESPACE}'."
  echo "Current pods in '${NAMESPACE}':"
  kubectl get pods -n "${NAMESPACE}"
  exit 1
fi

PROXY_URL="http://${PROXY_POD_IP}:${PROXY_PORT}"
echo "Proxy IP Identified : ${PROXY_POD_IP}"
echo "Proxy Endpoint URL  : ${PROXY_URL}"
echo
"================================================================================="
echo ""

# Formatted Table Header
printf "%-32s | %-40s | %-16s | %-8s | %-28s\n" \
  "SERVICE / TARGET" "HOST / FQDN" "DNS (NSLOOKUP)" "STATUS" "DETAILS"
echo
"---------------------------------------------------------------------------------
-----------------------------------------------"

# ================================================================================
# Helper Function: Execute DNS + Connectivity Check
# Arguments:
#   $1: Service Name
#   $2: Host / FQDN
#   $3: Port
#   $4: Test Mode ("curl-proxy", "nc-proxy", "nc-direct")
#   $5: Full URL (optional, for curl endpoints)
# ================================================================================
```

```
run_test() {
  local name="$1"
  local host="$2"
  local port="$3"
  local mode="$4"
  local url="${5:-https://${host}:${port}/}"

  # 1. DNS Resolution Check (nslookup)
  local dns_display="FAIL"
  local resolved_ip
  resolved_ip=$(nslookup "$host" 2>/dev/null | awk '/^Address: / { print $2 }' |
tail -n1)

  if [[ -n "$resolved_ip" ]]; then
    dns_display="OK ($resolved_ip)"
  fi

  # 2. Connectivity Check
  local conn_status="FAIL"
  local details=""

  case "$mode" in
    "curl-proxy")
      # Probe via HTTP CONNECT tunnel through Squid proxy
      local resp
      resp=$(curl -s -k -o /dev/null -w "%{http_code}" \
        --connect-timeout "$TIMEOUT" \
        --max-time "$((TIMEOUT + 2))" \
        --proxy "$PROXY_URL" \
        "$url" 2>/dev/null || echo "000")

      # Network reachability evaluation:
      # 000 = Client failed to reach proxy or connection timed out
      # 502/503/504 = Squid proxy failed to reach destination (ERR_CONNECT_FAIL,
etc.)
      # Any other code (1xx, 2xx, 3xx, 4xx, 500) = Destination host responded
(network OK)
      if [[ "$resp" == "000" ]]; then
        conn_status="FAIL"
        details="Client Timeout / Unreachable"
      elif [[ "$resp" =~ ^(502|503|504)$ ]]; then
        conn_status="FAIL"
        details="Proxy $resp (Host Unreachable)"
      else
        conn_status="PASS"
```

```
        details="HTTP $resp (Reached Host)"
      fi
      ;;

    "nc-proxy")
      # Tunnel TCP port through Squid proxy
      if nc -zv -w "$TIMEOUT" "$host" "$port" --proxy
"${PROXY_POD_IP}:${PROXY_PORT}" &>/dev/null; then
        conn_status="PASS"
        details="TCP $port OPEN (via proxy)"
      else
        conn_status="FAIL"
        details="Proxy TCP tunnel refused/timeout"
      fi
      ;;

    "nc-direct")
      # Direct TCP port check (no proxy)
      if nc -zv -w "$TIMEOUT" "$host" "$port" &>/dev/null; then
        conn_status="PASS"
        details="Direct TCP $port OPEN"
      else
        conn_status="FAIL"
        details="Direct TCP $port FAILED"
      fi
      ;;
  esac

  # Color formatting
  local color_dns="\033[0;31m"
  [[ "$dns_display" =~ ^OK ]] && color_dns="\033[0;32m"

  local color_conn="\033[0;31m"
  [[ "$conn_status" == "PASS" ]] && color_conn="\033[0;32m"

  printf "%-32s | %-40s | ${color_dns}%-16s\033[0m | ${color_conn}%-8s\033[0m |
%-28s\n" \
    "${name:0:32}" "${host:0:40}" "${dns_display:0:16}" "$conn_status" "$details"
}

# ================================================================================
# Endpoint Test Suite (All 24 targets from the migration sheet)
# ================================================================================

# 1. GuardedLink (443)
```

```
run_test "GuardedLink (443)" "glinkws-prod-tomcat.vzwcorp.com" 443 "curl-proxy"
"https://glinkws-prod-tomcat.vzwcorp.com/GLWeb/Onlte/GL_ONSITELTE_APISoapHttpPort"

# 2. IbwaveUnityServer (443)
run_test "IbwaveUnityServer (443)" "buildservices.verizon.com" 443 "curl-proxy"
"https://buildservices.verizon.com/Mobile/API/Login"

# 3. s360 ticketing (443)
run_test "s360 ticketing (443)" "oa.verizon.com" 443 "curl-proxy"
"https://oa.verizon.com/jgwv-s360-incident-mgmt-services/vbsm/getCaseDetails/WB240
619795932"

# 4. flow 360 (443)
run_test "flow 360 (443)" "oa.verizon.com" 443 "curl-proxy"
"https://oa.verizon.com/hvpv-swap-commonservice"

# 5. Spidernet (443)
run_test "Spidernet (443)" "CTTPMIBGVE2-SC-EMS-VIP.msc.vzwnet.com" 443
"curl-proxy" "https://CTTPMIBGVE2-SC-EMS-VIP.msc.vzwnet.com"

# 6. vEDA blrck (8181 & 8383)
run_test "vEDA blrck (8181)" "blrck6030071-o-eda-01.vnoe.vzwnet.com" 8181
"nc-proxy"
run_test "vEDA blrck (8383)" "blrck6030071-o-eda-01.vnoe.vzwnet.com" 8383
"nc-proxy"

# 7. vEDA vavit (8181 & 8383)
run_test "vEDA vavit (8181)" "vavit6247086-o-eda-01.vnoe.vzwnet.com" 8181
"nc-proxy"
run_test "vEDA vavit (8383)" "vavit6247086-o-eda-01.vnoe.vzwnet.com" 8383
"nc-proxy"

# 8. CUDB blrck & holtl (636 - Direct LDAP/SSL)
run_test "CUDB blrck (636)" "blrck6030071-o-cudb-01-fe.vnoe.vzwnet.com" 636
"nc-direct"
run_test "CUDB holtl (636)" "holtl6717827-o-cudb-01-fe.vnoe.vzwnet.com" 636
"nc-direct"

# 9. Thingspace(pMEC) (443)
run_test "Thingspace (443)" "apimec.m2m.cdsapps.com" 443 "curl-proxy"
"https://apimec.m2m.cdsapps.com/api/v1/health/check"

# 10. xLPT 4G & 5G
run_test "xLPT 4G (8181)" "xlpt.eng.vzwcorp.com" 8181 "curl-proxy"
"http://xlpt.eng.vzwcorp.com:8181/"
```

```
run_test "xLPT 5G (9091)" "5gpt.vh.vzwnet.com" 9091 "curl-proxy"
"http://5gpt.vh.vzwnet.com:9091/"

# 11. ECPD (443)
run_test "ECPD (443)" "ecpdms-rd.verizon.com" 443 "curl-proxy"
"https://ecpdms-rd.verizon.com/ewiapi/ecpd/osnd/osndValidateEcpdId"

# 12. SAIL point server (22 via proxy)
run_test "SAIL Point (22)" "cisinventory.verizon.com" 22 "nc-proxy"

# 13. NDAC manager (443)
run_test "NDAC manager (443)" "console.west.us.dac.nokia.com" 443 "curl-proxy"
"https://console.west.us.dac.nokia.com/"

# 14. NMP (443)
run_test "NMP (443)" "us.private5g.ericsson.net" 443 "curl-proxy"
"https://us.private5g.ericsson.net/"

# 15. FOTA SERVER (22 - Direct SSH)
run_test "FOTA SERVER (22)" "svzwxfr.st.saas.nokia.com" 22 "nc-direct"

# 16. GSMA DB (443)
run_test "GSMA DB (443)" "imeidb.gsma.com" 443 "curl-proxy"
"https://imeidb.gsma.com"

# 17. Nokia SFTP server (22 - Direct SFTP)
run_test "Nokia SFTP (22)" "sftp.ext.net.nokia.com" 22 "nc-direct"

# 18. GIS (443)
run_test "GIS (443)" "fs-wa.verizon.com" 443 "curl-proxy"
"https://fs-wa.verizon.com/fimp3gis"

# 19. S360- VBSM ticketing (443)
run_test "S360 VBSM (443)" "oa.verizon.com" 443 "curl-proxy"
"https://oa.verizon.com/jgwv-s360-incident-mgmt-services/vbsm/getCaseDetails/WB240
619795932"

# 20. SMTP - NP (25 - Direct SMTP)
run_test "SMTP NP (25)" "vzsmtp.verizon.com" 25 "nc-direct"

# 21. MyBiz (PROD & SDC)
run_test "MyBiz PROD (443)" "sso.verizonenterprise.com" 443 "curl-proxy"
"https://sso.verizonenterprise.com/account/business/login/unifiedlogin"
run_test "MyBiz SDC (443)" "mb.sdc.vzwcorp.com" 443 "nc-proxy"
```

```
# 22. CTS (443)
run_test "CTS (443)" "data.cts.errigal.com" 443 "curl-proxy"
"https://data.cts.errigal.com"

echo
"------------------------------------------------------------------------------
------------------------------------------------"
echo "All tests finished."
```

Shell

```
chmod +x test_endpoints.sh

./test_endpoints.sh
```

SMTP

```shell
curl -v telnet://tpaapd1kva109.verizon.com:25
telnet tpaapd1kva109.verizon.com 25
curl -v smtp://tpaapd1kva109.verizon.com:25
nc -zv -w 5 tpaapd1kva109.verizon.com 25
```

https://docs.google.com/document/d/1gtd3yHHnvOwscq3eQ9OkoWtnyn1t1VBLXBYPhquRles/edit?tab=t.0

Help Doc: https://atyourservice.verizon.com/ays?id=ays_kb_article&number=KB0026766

Raise Request:
https://atyourservice.verizon.com/ays?id=ays_sc_cat_item&sys_id=e177e287db7ae300df86f55eae96193a

Ticket opened by me:
https://atyourservice.verizon.com/ays?id=ticket&table=sc_req_item&sys_id=183d1454978c17940b3b722f2153afba&view=sp

AYS request raised by me with consolidated apptier subnet range:

Opened by S, Sachin

Contact S, Sachin

Requested For S, Sachin

Preferred Contact Method Corporate Email

Contact Method sachin.s@verizon.com

Requester's Manager Dabbuguntla, Harikrishna Yadav

Business Justification

OSLTE SPOG Application - Microservice applications running on AWS EKS (App Tier) require direct outbound TCP port 25 connectivity to internal Verizon SMTP relays (tpaapd1kva109.verizon.com) to transmit automated system notifications, alert emails, and transactional report emails. Access is required for Fargate and EC2 application worker nodes across Dev, Test, and Staging VPC environments. These services are deployed on AWS EKS in Verizon's SOE paltform.

Requestor Business Unit EDN

IP address

63.106.208.0/24, 63.106.222.0/24, 63.106.209.0/24, 63.106.220.0/24, 63.106.222.0/24, 63.106.193.0/24, 63.106.194.192/26, 63.106.196.192/26, 63.106.225.0/24

Hostname

oslte-portal-dev.pact.verizon.com, oslte-portal-test.pact.verizon.com, oslte-portal-preprod.pact.verizon.com

Application / VAST ID On Site LTE Single Pane of Glass

Envelope Senders Address (@verizon.com, @verizonwireless.com)

donotreply@verizon.com , donotreply@verizonwireless.com

Sending Batches? yes

Expected Messaging Volume? less than 3,600 emails per hour

Email Usage Rate Daily

System Point of Contact Dabbuguntla, Harikrishna Yadav

Email harikrishna.yadav.dabbuguntla@in.verizon.com

Number 7676014114

Additional Details

Dev Env: oslte-portal-dev.pact.verizon.com
63.106.208.0/24
63.106.222.0/24

Test Env: oslte-portal-test.pact.verizon.com
63.106.209.0/24
63.106.220.0/24
63.106.222.0/24

Staging env: oslte-portal-preprod.pact.verizon.com
[us-east-1]:
63.106.193.0/24
63.106.194.192/26
63.106.196.192/26

[us-west-2]:
63.106.225.0/24

Subnet Details:

in dev env, we have apptier subnets like:
63.106.222.64/26
63.106.222.0/26
63.106.208.160/27
63.106.208.128/27

in test env,
63.106.209.128/27
63.106.209.160/27
63.106.220.0/25
63.106.220.128/25
63.106.222.128/26
63.106.222.192/26

in staging env:
us-east-1
63.106.193.64/26
63.106.194.192/26
63.106.193.192/26
63.106.196.192/26
63.106.193.128/26
63.106.193.0/26

us-west-2
63.106.225.0/26
63.106.225.128/26
63.106.225.64/26

Consolidated Subnet Range:
DEV ENVIRONMENT (VPC 63.106.208.0/20):
- 63.106.208.128/25 (Covers 208.128/27 & 208.160/27)[cite: 1]
- 63.106.222.0/25 (Covers 222.0/26 & 222.64/26)[cite: 1]
Alternatively: 63.106.208.0/24 and 63.106.222.0/24[cite: 1]

TEST ENVIRONMENT (VPC 63.106.208.0/20):
- 63.106.209.128/25 (Covers 209.128/27 & 209.160/27)[cite: 1]
- 63.106.220.0/24 (Covers 220.0/25 & 220.128/25)[cite: 1, 2]
- 63.106.222.128/25 (Covers 222.128/26 & 222.192/26)[cite: 1, 2]
Alternatively: 63.106.209.0/24, 63.106.220.0/24, and 63.106.222.0/24[cite: 1, 2]

STAGING ENVIRONMENT - US-EAST-1 (VPC 63.106.192.0/20):
- 63.106.193.0/24 (Covers 193.0/26, 193.64/26, 193.128/26, & 193.192/26)[cite: 1]
- 63.106.194.192/26[cite: 1]
- 63.106.196.192/26[cite: 1]

STAGING ENVIRONMENT - US-WEST-2 (VPC 63.106.224.0/20):
- 63.106.225.0/24 (Covers 225.0/26, 225.64/26, & 225.128/26)[cite: 1]

Previous request which was created by my senior before me:

Preferred Contact Method Corporate Email

Contact Method vamshi.krishna2@verizon.com

Requester's Manager Thambirajah, Sundarraj (Raj)

Business Justification OSLTE SPOG application services (kpi-processor, ndac-kpi-processor, ibc-kpi-processor, portal) require SMTP relay access to send automated email notifications and alerts to operations teams. These services are deployed on AWS EKS in Verizon's PACT environment.

Requestor Business Unit EDN

IP address 63.106.222.0/24 63.106.220.0/24 63.106.209.0/24 63.106.212.128/25

Hostname ip-63-106-220-118.ec2.internal , ip-63-106-193-61.ec2.internal

Application / VAST ID On Site LTE Single Pane of Glass UAT

Envelope Senders Address (@verizon.com, @verizonwireless.com)
donotreply@verizon.com , donotreply@verizonwireless.com

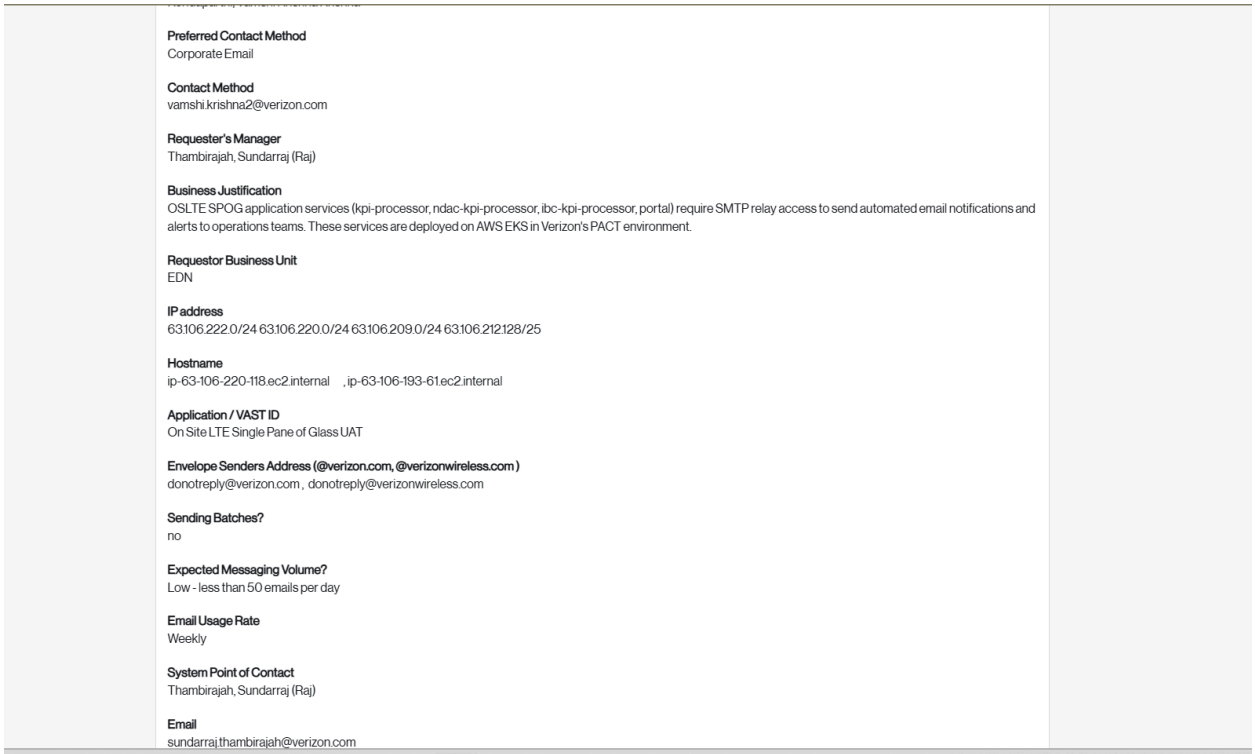
Sending Batches? no

Expected Messaging Volume? Low - less than 50 emails per day

Email Usage Rate Weekly

System Point of Contact Thambirajah, Sundarraj (Raj)

Email sundarraj.thambirajah@verizon.com



Preferred Contact Method
Corporate Email

Contact Method
vamshi.krishna2@verizon.com

Requester's Manager
Thambirajah, Sundarraj (Raj)

Business Justification
OSLTE SPOG application services (kpi-processor, ndac-kpi-processor, ibc-kpi-processor, portal) require SMTP relay access to send automated email notifications and alerts to operations teams. These services are deployed on AWS EKS in Verizon's PACT environment.

Requestor Business Unit
EDN

IP address
63.106.222.0/24 63.106.220.0/24 63.106.209.0/24 63.106.212.128/25

Hostname
ip-63-106-220-118.ec2.internal , ip-63-106-193-61.ec2.internal

Application / VAST ID
On Site LTE Single Pane of Glass UAT

Envelope Senders Address (@verizon.com, @verizonwireless.com)
donotreply@verizon.com , donotreply@verizonwireless.com

Sending Batches?
no

Expected Messaging Volume?
Low - less than 50 emails per day

Email Usage Rate
Weekly

System Point of Contact
Thambirajah, Sundarraj (Raj)

Email
sundarraj.thambirajah@verizon.com

Reed, Michael about a month agoabout a month ago Additional comments

TASK0018404019: Approved
TASK0018405600: Requested IP has been added to the VzApproved list for the application SMTP relay platform. Please refer to the below AYS article for proper configuration and testing:

******** Please Read*******

Enterprise SMTP Relay Configuration Settings:
https://atyourservice.verizon.com/ays?id=ays_kb_article&number=KB0026766

Setting Envelope FROM and tracking Non Delivery Reports (NDRs)
If you are not tracking NDRs and accepting bounce emails within your application:

Envelope FROM and Reply TO MUST be set to:
do-not-reply@verizon.com or
donotreply@verizon.com or
donotreply@verizonwireless.com

If you are tracking NDRs:
Ensure the internal domain (Example: *.verizon.com) being specified in the envelope FROM is resolvable and your system is accepting port 25 inbound for the domain being used.
NOTE:

DO NOT set the Envelope FROM: field to a distribution list or invalid email address. Doing so will cause a messaging loop between our on premise and cloud systems and will cause us to block your source system from sending email.
DO NOT use an email FROM: or TO: an invalid address. Examples of invalid addresses:
root@
oracle@
apache@
mysql@

@hostname.verizon.com addressing unless authorized in advance

If your application is generating messages like the above examples in the Envelope FROM: or Recipient To: list - it will be dropped or blocked.

Q. How much email will Gmail accept?
A. If you are planning to set up a host to send alerts to internal recipients on Verizon Gmail, be sure that you do not breach Gmail delivery thresholds. These are global settings in Google, we do not have the ability to change these settings in our Google Workspace tenant. If the following thresholds are breached, Google blocks delivery to the recipient's mailbox. If the source continues to generate alerts, the queues will build on the mail relays. In that situation, we will be forced to block the source IP from sending and you'll need to open an AYS ticket explaining what measures you have taken to prevent future flooding of Gmail with alert emails. Keep in mind the limits are for ALL email going to a mailbox and not individual hosts, total alerts being sent should be well below the thresholds listed since other messages going to users can push the cumulative number over causing a throttle.

Gmail Message Limits:

Per minute: 60
Per hour: 3,600
Per day: 86,400
Size limit: You can receive emails of up to 50 MB.

Q. How long does the Google suspension last on a mail object that has been throttled?

A. If an account reaches a receiving limit, the restriction on getting new mail typically lasts for about 24 hours. After 24 hours, receiving limits are automatically reset and the account can get mail. When an account is reset, messages sent to the account during the restricted period can't be recovered.

Shell

```
<<K9s-Shell>> Pod: applications/spog-kpi-processor-app-deployment-c997544c8-ppfl6
| Container: spog-kpi-processor-app

/app $ nc -zv -w 5 tpaapd1kva109.verizon.com 25
tpaapd1kva109.verizon.com (63.27.16.22:25) open

/app $ curl -v telnet://tpaapd1kva109.verizon.com:25
* Uses proxy env variable no_proxy ==
'10.100.0.1,localhost,127.0.0.1,63.106.208.0/20,169.254.169.254,.internal,s3.amazonaws.com,.s3.us-east-1.amazonaws.com,api.ecr.us-east-1.amazonaws.com,dkr.ecr.us-east-1.amazonaws.com,ec2.us-east-1.amazonaws.com,.us-east-1.eks.amazonaws.com,sts.us-east-1.amazonaws.com,elasticloadbalancing.us-east-1.amazonaws.com'
* Host tpaapd1kva109.verizon.com:25 was resolved.
* IPv6: (none)
* IPv4: 63.27.16.22
*   Trying 63.27.16.22:25...
```

```
* Connected to tpaapd1kva109.verizon.com (63.27.16.22) port 25
554-tpaapd1kva109.verizon.com
554 Your access to this mail system has been rejected because the IP address
(63.106.222.225) is not authorized for relaying. If you believe that this failure
is in error, please see fill out the SMTP Gateway Access form at:
https://atyourservice.verizon.com/ays?id=ays_sc_cat_item&sys_id=e177e287db7ae300df
86f55eae96193a
```

The connection test succeeded at the TCP level (the port is open), but the Verizon SMTP server rejected your container's source IP address: **`63.106.222.225`**.

The server returned an explicit SMTP rejection code (`554 Your access to this mail system has been rejected because the IP address... is not authorized for relaying`).

Root Cause & What's Happening

1. **`nc -zv`**: Only performs a raw TCP SYN/ACK handshake. It reported `open` because the port accepted the TCP connection.
2. **`curl telnet://`**: Opens the TCP connection and reads the initial SMTP response banner. The remote server immediately checked your egress IP (`63.106.222.225`) against its internal relay allowlist, rejected it, and sent code `554`.
3. **`^C` hang**: `telnet://` mode holds the connection open waiting for interactive input, which is why it sat there until you pressed `Ctrl+C`.

Response for my ticket:

> TASK0018688947: DRS Not Found: This is an internal address which is allowed to access the SMTP Gateway
>
> Closed by: Guy Devin Reinert on Fri, 16 Oct 2026 at 18:54:40 GMT
> TASK0018693299: IP's have been added to the Non Prod/ DEV/UAT appliance for test relaying.
>
> ***Please Read***
>
> Please refer to the below AYS article for proper testing and configuration for the sender host application. Take note of the "How should I set Envelope FROM", "How much email will Gmail accept?" and "What if I am DEV/UAT Testing?" sections. Any

mail flooding of the VZ GMail environment or invalid envelope sender addressing will be blocked at the source IP level of the application/senderhost.

SMTP Relay Information
https://atyourservice.verizon.com/ays?id=ays_kb_article&number=KB0026766

How should I set Envelope FROM and can I track NDRs (Non Delivery Reports)?
If you are not tracking NDRs and accepting bounce emails within your application the Envelope FROM: and Reply TO: should be set to do-not-reply@verizon.com, donotreply@verizon.com, or donotreply@verizonwireless.com. If you are tracking NDRs ensure the internal domain (Example: *.verizon.com) being specified in the envelope FROM is resolvable and your system is accepting port 25 inbound for the domain being used.
NOTE: We do not support any email going to or coming from invalid addresses, Examples would include: root@, oracle@, apache@, mysql@, @hostname.verizon.com addressing unless authorized in advance, if your application is generating messages with those in the Envelope FROM: or Recipient To: list it will dropped or blocked.
Applications should NEVER set the Envelope FROM: field to a distribution list or invalid email address. If recipient throttling in Gmail occurs setting the envelope to a DL or invalid address will cause a messaging loop between our on premise and cloud systems, in turn we will block your source system from sending email.

What if I am DEV/UAT Testing?
63.27.16.22 tpaapd1kva109.verizon.com is the relay specifically for DEV/UAT testing, we have blackhole addressing available when doing volume testing so please contact our group for details before proceeding. If we find any systems sending DEV/UAT traffic through the production relays the source IP will be blocked.

How much email will Gmail accept?
If you are planning to set up a host to send alerts to internal recipients on Verizon Gmail, be sure that you do not breach Gmail delivery thresholds. These are global settings in Google, we do not have the ability to change these settings in our Google Workspace tenant. If the following thresholds are breached, Google blocks delivery to the recipient's mailbox (suspension). If the source continues to generate alerts, the queues will build on the mail relays. In that situation, we will be forced to block the source IP from sending and you'll need to open an AYS ticket explaining what measures you have taken to prevent future flooding of Gmail with alert emails.

Gmail Message Limits:
Per minute 60
Per hour 3,600
Per day 86,400
Size limit: You can receive emails of up to 50 MB.

How long does Google suspension last?

If an account reaches a receiving limit, the restriction on getting new mail typically lasts for about 24 hours. After 24 hours, receiving limits are automatically reset and the account can get mail. When an account is reset, messages sent to the account during the restricted period can't be recovered.

Tests to cross check once IP blocks are allowed:

Retest Connection & Relay from Pod

From inside your container (`spog-kpi-processor-app`):

1. Quick Banner & EHLO Check

```shell
nc -zv -w 5 tpaapd1kva109.verizon.com 25
```

Run an interactive or scripted SMTP handshake to confirm the `554` rejection is gone:

```shell
cat <<EOF | nc -w 10 tpaapd1kva109.verizon.com 25
EHLO oslte-portal-dev.pact.verizon.com
QUIT
EOF
```

Expected response: A `220` greeting banner followed by `250-...` feature announcements and `221 Bye` (no `554 Access Rejected` upon initial connect).

2. Send an End-to-End Test Email

```shell
cat <<EOF | nc -w 15 tpaapd1kva109.verizon.com 25
EHLO oslte-portal-dev.pact.verizon.com
MAIL FROM:<donotreply@verizon.com>
RCPT TO:<sachin.s@verizon.com>
DATA
From: donotreply@verizon.com
To: sachin.s@verizon.com
Subject: Test Relay from EKS Dev Pod
```

```
This is a test notification from spog-kpi-processor-app.
.
QUIT
EOF
```

Expected response: `250 2.0.0 OK: queued as ...`

```
[ec2-user@ip-63-106-220-43 bin]$ nc -zv -w 5 tpaapd1kva109.verizon.com 25
Ncat: Version 7.93 ( https://nmap.org/ncat )
Ncat: Connected to 63.27.16.22:25.
Ncat: 0 bytes sent, 0 bytes received in 0.29 seconds.
[ec2-user@ip-63-106-220-43 bin]$ cat <<EOF | nc -w 10 tpaapd1kva109.verizon.com 25
EHLO oslte-portal-dev.pact.verizon.com
QUIT
EOF
220 tpaapd1kva109.verizon.com ESMTP
250-tpaapd1kva109.verizon.com
250-8BITMIME
250-SIZE 27262976
250 STARTTLS
221 tpaapd1kva109.verizon.com
[ec2-user@ip-63-106-220-43 bin]$ cat <<EOF | nc -w 15 tpaapd1kva109.verizon.com 25
EHLO oslte-portal-dev.pact.verizon.com
MAIL FROM:<donotreply@verizon.com>
RCPT TO:<sachin.s@verizon.com>
DATA
From: donotreply@verizon.com
To: sachin.s@verizon.com
Subject: Test Relay from EKS Dev Pod

This is a test notification from spog-kpi-processor-app.
.
QUIT
EOF
220 tpaapd1kva109.verizon.com ESMTP
250-tpaapd1kva109.verizon.com
250-8BITMIME
250-SIZE 27262976
250 STARTTLS
250 sender <donotreply@verizon.com> ok
250 recipient <sachin.s@verizon.com> ok
354 go ahead
250 ok:  Message 263894707 accepted
221 tpaapd1kva109.verizon.com
```

Solutions

Option 1: Keep Direct App Tier Egress (Easiest) → same as prod setup

If company policy allows email to exit directly from the App Tier:

1.Identify App Tier CIDRs:

Gather the App Tier subnets where your Fargate/EC2 worker nodes reside in production (e.g., 63.106.220.0/25, 63.106.220.128/25).

2.Submit Verizon Gateway Access Form:

Submit the Verizon SMTP Gateway Access Form requesting access for those App Tier CIDR ranges.

Verification: Once approved, telnet vzsmtp.verizon.com 25 from an App Tier pod/node will return the 220 SMTP banner instead of the 554 rejection.

Option-2: Route SMTP Through Proxy Tier (Architecturally Preferred)

If all outbound traffic must exit through static Proxy Tier IPs (63.106.209.0/28):

1.Update Prod Squid ConfigMap:

Add port 25 to Safe_ports and SSL_ports in proxy-config and restart the proxy deployment:

```
Shell
kubectl rollout restart deployment/proxy -n proxy
```

2.Configure JavaMail Proxy Settings: (ConfigMap)

Add the Spring Mail proxy properties to oslte-kpi-processor-spring-config and oslte-admin-spring-config:

Properties

```
Shell
# Spring Mail Proxy Configuration
spring.mail.host=vzsmtp.verizon.com
spring.mail.port=25
spring.mail.properties.mail.smtp.proxy.host=proxy-service.proxy
spring.mail.properties.mail.smtp.proxy.port=3128
```

3.Whitelist Proxy Tier CIDRs:

Submit the Verizon SMTP Gateway Access Form requesting access for the static **Proxy Tier CIDRs** (63.106.209.0/28 and 63.106.209.16/28).

Verification: Test from your application pod using curl -v -x http://proxy-service.proxy:3128 telnet://vzsmtp.verizon.com:25 and verify Squid returns HTTP/1.1 200 Connection established.

Test connection from within Pod:

```shell
curl -v -x http://proxy-service.proxy:3128
telnet://tpaapd1kva109.verizon.com:25
```

```shell
/app $ curl -v -x http://proxy-service.proxy:3128
telnet://tpaapd1kva109.verizon.com:25
* Uses proxy env variable no_proxy ==
'10.100.0.1,localhost,127.0.0.1,63.106.208.0/20,169.254.169.254,.in
ternal,s3.amazonaws.com,.s3.us-east-1.amazonaws.com,api.ecr.us-east
-1.amazonaws.com,dkr.ecr.us-east-1.amazonaws.com,ec2.us-east-1.amaz
onaws.com,.us-east-1.eks.amazonaws.com,sts.us-east-1.amazonaws.com,
elasticloadbalancing.us-east-1.amazonaws.com,spog-device-app-servic
e'
* Host proxy-service.proxy:3128 was resolved.
* IPv6: (none)
* IPv4: 10.100.118.202
*   Trying 10.100.118.202:3128...
* CONNECT tunnel: HTTP/1.1 negotiated
* allocate connect buffer
* Establish HTTP proxy tunnel to tpaapd1kva109.verizon.com:25
> CONNECT tpaapd1kva109.verizon.com:25 HTTP/1.1
> Host: tpaapd1kva109.verizon.com:25
> User-Agent: curl/8.12.1
> Proxy-Connection: Keep-Alive
>
< HTTP/1.1 403 Forbidden
< Server: squid/6.8
< Mime-Version: 1.0
< Date: Thu, 15 Oct 2026 10:51:10 GMT
< Content-Type: text/html;charset=utf-8
< Content-Length: 1741
< X-Squid-Error: ERR_ACCESS_DENIED 0
< Vary: Accept-Language
< Content-Language: en
```

```
< Cache-Status: proxy-d59974fc-kwfdg
< Via: 1.1 proxy-d59974fc-kwfdg (squid/6.8)
< Connection: keep-alive
<
* CONNECT tunnel failed, response 403
* closing connection #0
curl: (56) CONNECT tunnel failed, response 403
```

403 Forbidden meant Squid blocked the request internally because of port restrictions.

Squid Proxy Port Restriction → Missing: Port 25 was omitted from Safe_ports and SSL_ports in squid.conf.

Solution to be implemented from my end:

Port 25 was added in two places under the squid.conf section of the proxy-config ConfigMap data:

1. Added to Safe_ports

```
None
    # Standard Safe & SSL Ports
    acl Safe_ports port 25          # SMTP mail relay
```

2. Added to SSL_ports

```
None
acl SSL_ports port 25           # SMTP tunnel via CONNECT
```

3. Add the Spring Mail proxy properties to app ConfigMaps Properties

```
Shell
# Spring Mail Proxy Configuration
spring.mail.host=vzsmtp.verizon.com
spring.mail.port=25
spring.mail.properties.mail.smtp.proxy.host=proxy-service.proxy
```

```
spring.mail.properties.mail.smtp.proxy.port=3128
```

4.Whitelist Proxy Tier CIDRs:

Submit the Verizon SMTP Gateway Access Form requesting access for the static **Proxy Tier CIDRs** (63.106.209.0/28 and 63.106.209.16/28).

Restart the Proxy Deployment to load the updated ConfigMap:

```
Shell
kubectl rollout restart deployment/proxy -n proxy
```

5. Verify the CONNECT tunnel from inside your application pod:

```
Shell
curl -v -x http://proxy-service.proxy:3128
telnet://tpaapd1kva109.verizon.com:25
```

```
Shell
<<K9s-Shell>> Pod: applications/spog-kpi-processor-app-deployment-c997544c8-ppfl6
| Container: spog-kpi-processor-app
/app $ curl -v -x http://proxy-service.proxy:3128
telnet://tpaapd1kva109.verizon.com:25
* Uses proxy env variable no_proxy ==
'10.100.0.1,localhost,127.0.0.1,63.106.208.0/20,169.254.169.254,.internal,s3.amazo
naws.com,.s3.us-east-1.amazonaws.com,api.ecr.us-east-1.amazonaws.com,dkr.ecr.us-ea
st-1.amazonaws.com,ec2.us-east-1.amazonaws.com,.us-east-1.eks.amazonaws.com,sts.us
-east-1.amazonaws.com,elasticloadbalancing.us-east-1.amazonaws.com'
* Host proxy-service.proxy:3128 was resolved.
* IPv6: (none)
* IPv4: 10.100.118.202
*   Trying 10.100.118.202:3128...
* CONNECT tunnel: HTTP/1.1 negotiated
* allocate connect buffer
* Establish HTTP proxy tunnel to tpaapd1kva109.verizon.com:25
> CONNECT tpaapd1kva109.verizon.com:25 HTTP/1.1
> Host: tpaapd1kva109.verizon.com:25
> User-Agent: curl/8.12.1
> Proxy-Connection: Keep-Alive
>
```

```
< HTTP/1.1 503 Service Unavailable
< Server: squid/6.8
< Mime-Version: 1.0
< Date: Thu, 15 Oct 2026 11:00:32 GMT
< Content-Type: text/html;charset=utf-8
< Content-Length: 1816
< X-Squid-Error: ERR_CONNECT_FAIL 110
< Vary: Accept-Language
< Content-Language: en
<
* CONNECT tunnel failed, response 503
* closing connection #0
curl: (56) CONNECT tunnel failed, response 503
/app $
```

- **503 Service Unavailable** means **Squid successfully accepted your CONNECT request for port 25**, but when Squid tried to connect out to tpaapd1kva109.verizon.com:25 on the backend, the connection timed out (ERR_CONNECT_FAIL 110).
- **Status:** Fixed. Your 403 Forbidden error cleared, meaning Squid now permits the HTTP CONNECT tunnel for port 25.

Backend Network Reachability (Current Blocker)

- **Missing:** Outbound port 25 egress from the **Proxy Tier subnets** (`63.106.209.0/28`) to the Verizon mail relay (`63.27.16.22`).
- **Impact:** Causes the **`503 Service Unavailable`** **(`ERR_CONNECT_FAIL 110`)** connection timeout.
- **Action Needed:** Allow outbound TCP port 25 in the Proxy Tier's AWS Security Group, and submit the Proxy Tier CIDRs on the Verizon SMTP Gateway allowlist form.

On-Site Network Dashboard (OSND)

DevOps Onboarding — flux-config-appcode

Welcome. This repository is a **Flux-watched, raw-manifest GitOps repo** for the "OSLTE / SPOG" (Verizon PACT/OSND) platform running on EKS Fargate. There are no Kustomize overlays, Helm charts, or Flux `GitRepository`/`Kustomization`/`HelmRelease` CRs checked in — Flux (or an equivalent controller) simply applies whatever YAML lives under each environment folder. Promotion between environments is done by **shell scripts that `sed`-patch image tags and git-push**, not by a Flux image-automation controller.

0. Repo layout (read this first)

```
flux-config-appcode/              <- git root
├── dev-config/                   <- Dev environment manifests
├── test-config/                  <- Test environment manifests  (you are here)
├── preprod-config/
│   ├── us-east-1/                <- Preprod, region us-east-1
│   └── us-west-2/                <- Preprod, region us-west-2
├── prod-config/
│   ├── us-east-1/                <- Prod, region us-east-1
│   └── us-west-2/                <- Prod, region us-west-2
├── promote-to-test.sh              <- dev-config      -> test-config
├── promote-to-preprod.sh (= 1.sh) <- test-config      -> preprod-config/{east,west}
├── promote-to-prod.sh    (= 2.sh) <- preprod-config   -> prod-config/{east,west}
└── scripts/                      <- ad-hoc API Gateway import/export notes
```

Each env-config folder has the **same internal shape**:

```
<env>-config/oslte/<service>/     <- ~20 microservices, one folder each
   config.yaml | configmap.yaml   Spring Boot / app ConfigMap
   deployment.yaml                Deployment (+ fluentd sidecar)
   hpa.yaml                       HorizontalPodAutoscaler (not on every service)
   secret.yaml                    Opaque Secret (creds, checked in base64)
   service.yaml | service-nlb.yaml  ClusterIP or internal NLB Service
   serviceAccount.yaml            ServiceAccount + IRSA role + RoleBindings
   fluentd.yml / fluentd.yaml     Fluentd config ConfigMap
<env>-config/oslte/common/        ClusterRole + newrelic Secret shared by all services
<env>-config/oslte-1/             a *second*, parallel copy of the same services — this is the "QA2" environment (only present in test-config)
<env>-config/nginx/               single reverse-proxy Deployment + ALB Ingress
```

<env>-config/squid-proxy/ egress HTTP proxy (Squid)
<env>-config/confluent-kafka-crd/ Confluent-for-Kubernetes operator CRs (Kafka)
<env>-config/namespaces/ **only one** Namespace object is defined here
```

`test-config` (this working directory) additionally contains **`oslte-1`**, a second full copy of the service set used for the "QA2" environment (see `deploy_qa2.sh`). Everything below describes `test-config`; `dev-config`/`preprod-config`/`prod-config` mirror the same shape (preprod/prod are further split per AWS region).

---

## 1. Architecture & Resources

**~20 microservices**, each following an identical pattern, deployed twice in `test-config` (`oslte/` = Test, `oslte-1/` = QA2):

| Service | Notes |
|---|---|
| admin, portal, api-first, scheduler, processor | Exposed via `service-nlb.yaml` (internal NLB, used as an API Gateway VPC-Link target) |
| device, cloud-connect, inventory, inventory-post, kpi-processor, ndac-kpi-processor, ibc-kpi-processor, osnd-api, s360-ticket, scim, vmb-processor | Plain `service.yaml` (ClusterIP) |
| wsevents | WebSocket gateway, exposed via its own internal NLB (`type: LoadBalancer`, port 443→8080) |
| oss-mediator | Multi-container pod: collector + Elasticsearch plugin + embedded Grafana + fluentd, backed by 4 EFS volumes |
| spog-aiops, spog-mongo-mcp, spog-opensearch-mcp, spog-osnd-mcp | Newer, lighter Python-style services — the "mcp" ones are **Model Context Protocol servers** exposing MongoDB / OpenSearch / OSND data to an AI/agent tool layer |
| pm-file-manager | Only present in `oslte-1` (QA2), not in `oslte` (Test) |

Resource kinds in use: `Deployment`, `Service` (ClusterIP / NodePort / internal `LoadBalancer`), `Ingress` (ALB), `HorizontalPodAutoscaler`, `ConfigMap`, `Secret` (all `Opaque`), `ServiceAccount`, `ClusterRole` / `ClusterRoleBinding` / `RoleBinding`, `PersistentVolume` / `PersistentVolumeClaim`, `StorageClass`, and Confluent-operator **CRDs** (`KRaftController`, `Kafka`, `Connect`, `ControlCenter`). No `StatefulSet`, `DaemonSet`, `Job`/`CronJob`, `NetworkPolicy`, or `PodDisruptionBudget` exist anywhere in the repo.

Almost every application Deployment runs **two containers**: the app itself and a `fluentd` log-shipping sidecar that tails `/app/log` and ships to a shared AWS OpenSearch domain. `oss-mediator` additionally runs an Elasticsearch-plugin sidecar and an embedded Grafana container in the same pod.
```

---

2. Traffic & Networking

Ingress is intentionally minimal and pushes almost all routing logic into an in-cluster **nginx reverse proxy**, not the Ingress object itself:

```
Internet/VPN
  │  https://oslte-<service>-test.pact.verizon.com  (ACM certs, internal ALB)
  ▼
ALB Ingress (nginx namespace, alb.ingress.kubernetes.io/*)
  │  single defaultBackend → nginx-service:8080 (NodePort, no host rules on the Ingress object itself)
  ▼
nginx Deployment (replicas: 1)  — routing table lives in a ConfigMap (reverse-proxy.conf)
  │  matches on Host/server_name, then proxy_pass by hostname:
  ├─► most REST services: https://<api-id>.execute-api.us-east-1.amazonaws.com/test/...
  │     (AWS API Gateway REST APIs — IDs are hardcoded per hostname in the ConfigMap;
  │      API Gateway VPC-Links front the *-service-nlb Services below)
  ├─► WebSocket (/ws*): https://oslte-wsapi-test.pact.verizon.com → wsevents' own internal NLB
  ├─► OpenSearch Dashboards: proxy_pass straight to the AWS OpenSearch domain endpoint
  ├─► oss-mediator Grafana: http://oss-mediator-grafana-service.oss-mediator.svc.cluster.local:3001
  └─► Confluent Control Center: http://controlcenter-0-internal.confluent.svc.cluster.local:9021
```

Two distinct "front door" patterns coexist:
1. **API-Gateway-fronted**: `admin`, `portal`, `api-first`, `scheduler`, `processor` are exposed only through `service-nlb.yaml` (`aws-load-balancer-type: nlb-ip`, internal). AWS API Gateway (execute-api, public regional endpoint) is VPC-Linked to that NLB. nginx proxies to the *public* execute-api URL, so a request can leave the VPC boundary and come back in via the VPC Link rather than going NLB→pod directly.
2. **Direct internal NLB**: `wsevents` gets its own internal NLB and DNS name (`oslte-wsapi-test.pact.verizon.com`) and is called directly — used for the WebSocket path since API Gateway REST doesn't suit it well here.

Service discovery: plain Kubernetes DNS (`<service>.<namespace>.svc.cluster.local`) for in-cluster calls (Grafana, Kafka Control Center, Kafka bootstrap `kafka.confluent:9092`). A `ClusterRole`/`ClusterRoleBinding` named `service-discovery-client` (get/watch/list on services, namespaces, pods, configmaps, endpoints) is bound cluster-wide to nearly every service's

ServiceAccount — this looks like a leftover from a Eureka/Consul-style discovery pattern rather than something actively used with native k8s DNS.

The nginx ConfigMap hardcodes a custom DNS resolver (`resolver 10.100.0.10 63.106.208.2 valid=60s;`) with no documented fallback.

---

3. Configuration & State

- **App config**: Spring Boot properties are embedded as literal blocks inside `config.yaml`/`configmap.yaml` ConfigMaps (`application.properties`, `SPRING_PROFILE`), mounted as files or read via `configMapKeyRef`.
- **Secrets**: every `secret.yaml` is a plain `Opaque` Secret with **base64-encoded (not encrypted) values committed directly to git** — MongoDB URIs with embedded credentials, OAuth2 client secrets, keystore passwords, and (in `vmb-processor/cert-secret.yaml`) full TLS **private keys and certificates**. There is no SealedSecrets/External-Secrets/SOPS integration anywhere in the repo — anyone with repo read access has every credential in every environment.
- **Cross-cutting secrets**: `oslte/common/newrelic-secret.yaml` defines the *same* New Relic keys three times, once per namespace (`applications`, `applications-1`, `osnd`), instead of one Secret being referenced across namespaces (Secrets are namespace-scoped, so this is the "correct" workaround, but it means one key rotation = editing N copies by hand).
- **Persistent storage**: only `oss-mediator` uses storage — four EFS-backed `PersistentVolume`/`PersistentVolumeClaim` pairs (`es`, `grafana`, `ndac`, `secret`/checkpoints), each **statically provisioned** (explicit `volumeHandle`, no EFS dynamic provisioning) and — despite EFS natively supporting `ReadWriteMany` — mounted `ReadWriteOnce`, capping the deployment to a single consuming pod. Confluent Kafka uses `aws-ebs`-backed dynamic PVCs (`confluent-sc` StorageClass, `Retain` reclaim policy) driven by the Confluent operator, sized 50Gi (dev/test KRaftController) up to 500Gi (Kafka brokers).
- **IAM**: every ServiceAccount carries an `eks.amazonaws.com/role-arn` annotation (IRSA) — clean per-service AWS permission boundaries, at least in principle.
- **Runtime env vars**: proxy settings (`http_proxy`/`https_proxy`/`no_proxy`), New Relic APM config, and AWS region are injected via literal `env:` blocks repeated in almost every Deployment (copy-pasted per service rather than templated).

---

4. GitOps / Deployment Strategy

There is **no Flux `Kustomization`/`HelmRelease`/`ImageUpdateAutomation` object in this repo** — Flux (per the repo name) is presumed to be configured elsewhere in-cluster to simply `kubectl apply -R` whatever it finds under each environment path, polling git. All the "GitOps" intelligence lives in shell scripts at the repo root, not in Flux CRs:

- `promote-to-test.sh` (dev → test): reads the image tag from every `dev-config/oslte/<svc>/deployment.yaml`, and if it differs from `test-config`'s tag, `sed`-rewrites the test deployment's `image:` line and prints (does not run) the `git add/commit/push`.
- `promote-to-preprod.sh` / `1.sh` (test → preprod, both `us-east-1` and `us-west-2`): same pattern, one-way diff-and-patch.
- `promote-to-prod.sh` / `2.sh` (preprod → prod, both regions): same pattern. Note `2.sh` promotes to **both** prod regions from **`preprod_east`** only — `preprod-west` versions are never read, so west-region prod always inherits east's tag.
- Inside `test-config` specifically: `auto-deploy.sh` conditionally deploys a single service's image based on a branch-name gate in `auto-deploy.conf` (`auto_deploy=true/false`); `deploy.sh` bulk-deploys the latest ECR image per branch across all Test services; `deploy_qa2.sh` does the same for the QA2 copy (`oslte-1`).
- All of these scripts write directly to `deployment.yaml` `image:` lines with `sed`, then `git commit`/`git push` straight to `master` — there is no PR/review gate, no CI validation of the YAML, and no rollback tooling beyond re-running an older tag by hand.
- **Environments**: Dev → Test (`oslte`) → QA2 (`oslte-1`, test-config only) → Preprod (east+west) → Prod (east+west). Preprod/Prod are active/active across two AWS regions; Dev/Test/QA2 exist only in `us-east-1`.

---

5. Key Operational Gotchas

1. **Namespace provisioning is not GitOps-managed.** Only one `Namespace` object exists in the entire repo (`namespaces/applications.yaml`), yet manifests target at least `applications`, `applications-1`, `nginx`, `proxy`, `oss-mediator`, `confluent`, `confluent-1`, and `osnd`. Those namespaces must be created out-of-band (manually or by another tool) or Flux will fail to apply anything into them.
2. **Secrets are committed in the clear (base64 only).** DB URIs, OAuth secrets, keystore passwords, and private TLS keys live in git history forever, in every environment folder. Treat this repo as already-compromised for those credentials and prioritize migrating to SOPS/External Secrets/Sealed Secrets.
3. **Single points of failure in the network path.** Both `nginx` (the reverse proxy that fronts *every* hostname-based route) and `squid-proxy` (the *only* egress path — see §6) run `replicas: 1`, `type: NodePort`, no PodDisruptionBudget, no anti-affinity. Losing that one pod takes down ingress or all outbound/proxy-dependent traffic for the whole environment.
4. **QA2 (`oslte-1`) has almost no autoscaling.** Only `ibc-kpi-processor`, `ndac-kpi-processor`, and `s360-ticket` have an `hpa.yaml` in `oslte-1`; the other ~13 services have no `replicas:` override either, so they run a bare single pod with no HPA — Test (`oslte`) has HPAs (1–10 replicas, CPU 60% / memory 80%) for nearly all services by comparison.
5. **Routing config is hardcoded and manual.** The nginx ConfigMap hardcodes API Gateway `execute-api` IDs per hostname. If an API Gateway is redeployed/recreated, every reference to

its ID in `nginx/config.yaml` must be found and hand-edited, then the nginx pod restarted — there's no dynamic discovery.
6. **Traffic hairpins through public AWS endpoints.** For API-Gateway-fronted services, in-cluster nginx calls the *public* `execute-api.us-east-1.amazonaws.com` hostname rather than a private VPC endpoint, adding an unnecessary internet-facing hop (mitigated only by the fact this is an internal ALB and presumably routed via VPC endpoints/PrivateLink, which isn't visible from these manifests).
7. **No CI gate on deploys.** The promotion/deploy scripts `sed`-patch YAML and push straight to `master`; there's no schema validation, no PR review, no automated test before an image reaches Test/Preprod/Prod.
8. **`prod-west` silently follows `preprod-east`, never `preprod-west`.** In `promote-to-prod.sh`, both `promote_to_prod_east` and `promote_to_prod_west` iterate over `preprod_east_micro_services` — a version deployed only to preprod-west will never be promoted to either prod region by this script.
9. **Mixed image-tagging discipline.** Most images are pinned to explicit tags (`fluentd:v1.17-1`, `fluentd:v1.18.0`), but several fluentd sidecars (`wsevents`, `cloud-connect`/`scheduler`/`osnd-api`/etc. in `oslte-1`) use `fluentd:latest` — non-reproducible and can drift between pod restarts.
10. **EFS volumes are pinned to `ReadWriteOnce`** despite EFS natively supporting `ReadWriteMany`, which blocks any future horizontal scaling of `oss-mediator` without a manifest change.
11. **Broad, repeated RBAC.** The `service-discovery-client` ClusterRole is bound (via ClusterRoleBinding) to nearly every service's ServiceAccount, granting cluster-wide read on services/pods/configmaps/endpoints/namespaces — broader than any service appears to need, and duplicated per-service instead of centrally managed.
12. **Liveness/readiness probes are inconsistent.** Some services (e.g. `device`, `wsevents`) define both `readinessProbe` and `livenessProbe`; others (e.g. `spog-mongo-mcp`) have probes fully commented out — a hung MCP pod would never be restarted or removed from service.

---

6. Proxies and Traffic Flow Through Them

There are **two independent proxy layers**, and recent history shows their configuration has been actively churned (see commits `047609d6c` "replaced pact proxy" and `650da429a` "reverting proxy changes back to it's original setting"):

a) Egress HTTP/HTTPS proxy — in-cluster Squid (`squid-proxy/`)
- Deployed as `proxy` Deployment in the `proxy` namespace, image `elestio/squid:v6.8.0`, single replica, exposed as `proxy-service` (NodePort 3128).
- ACL policy (`squid.conf`) allows CONNECT only to a specific SSL port allow-list (443, 8443, 8383, 8181, 9091, 3128, 8081, 8282, 18002, 27016) and only from `spog-net` (`63.106.0.0/16`); everything else is denied.

- Most application containers set `http_proxy`/`https_proxy=http://proxy-service.proxy:3128/` with a `no_proxy` list carving out AWS/internal endpoints (ECR, S3, EKS, STS, ELB, `.internal`) so those calls bypass the proxy.
- Spring services additionally configure the **same proxy at the app layer** via `httpconnpool.proxyHost=proxy-service.proxy` / `proxyPort=3128` (see `oslte/device/config.yaml`) for their internal HTTP client pool, plus a separate New Relic-specific proxy setting (`management.metrics.export.newrelic.proxy-host/port`).
- **Inconsistency to watch**: `oss-mediator/deployment.yaml` still points at the *old* external proxy `http://proxy-nonprod.pact.verizon.com:8080/` instead of the in-cluster `proxy-service.proxy:3128` that `device`, `wsevents`, `spog-mongo-mcp`, etc. were migrated to. This is exactly the kind of drift the recent proxy-related commits were fixing/reverting — worth confirming which value is actually correct before touching either.

b) Ingress-side reverse proxy — nginx (`nginx/`)
- Not an egress proxy — it's the reverse proxy described in §2 that terminates internal-ALB traffic and `proxy_pass`es per-hostname to API Gateway, the WebSocket NLB, OpenSearch, Grafana, or Kafka Control Center. It does **not** route through the Squid proxy; it talks to its `$backend`s directly on port 80 from inside the `nginx` namespace.

Overall traffic-through-proxy picture
```
Outbound app calls (device, wsevents, mongo-mcp, etc.)
  app_container --(http_proxy=proxy-service.proxy:3128)--> Squid (proxy ns) --> Internet/allowed SSL ports
  app_container --(no_proxy match: ECR/S3/EKS/STS/ELB/.internal)--> direct, bypassing Squid

oss-mediator (drifted config)
  ossmediator/es-plugin containers --(http_proxy=proxy-nonprod.pact.verizon.com:8080)--> corporate proxy, NOT the in-cluster Squid

Inbound app traffic
  Client --> ALB (internal) --> nginx pod --> proxy_pass --> API Gateway / NLB / OpenSearch / Grafana / Kafka CC
  (this path never touches Squid; Squid is egress-only)
```

Suggested first tasks for a new engineer
- Confirm with the team whether `oss-mediator`'s `proxy-nonprod.pact.verizon.com:8080` should be migrated to `proxy-service.proxy:3128` like the other services, given the recent revert commit.
- Ask how/where the non-`applications` namespaces (`nginx`, `proxy`, `oss-mediator`, `confluent`, `osnd`, `applications-1`, `confluent-1`) actually get created, since it isn't in this repo.

- Get read access to the New Relic and MongoDB Atlas dashboards referenced by the hardcoded secrets so you're not tempted to rotate credentials by editing YAML alone (you'd need to update every namespace copy).
- Ask whether a Flux `Kustomization`/image-automation setup is planned, since promotions currently depend on engineers running shell scripts and pushing to `master` by hand.

System Architecture & Environments

The OSND platform is deployed across four primary environments hosted on AWS:

Environment	Architecture	AWS Regions	Traffic Distribution
Development (Dev)	Single-Region	us-east-1	N/A
Testing (QA)	Single-Region	us-east-1	N/A
Staging / Pre-Prod	Multi-Region	us-east-1, us-west-2	Active-Active
Production (Prod)	Multi-Region	us-east-1, us-west-2	Active-Active

Architecture Note: In multi-region deployments (Pre-Prod & Prod), both regions run independently in an **Active-Active** configuration.

Access Request & Onboarding Procedures

Requesting Jenkins Access

To trigger deployment jobs and view build pipelines, submit an ADOM access request:

1. Navigate to the **Verizon Webtools Request Portal**.
2. Under **Domain**, select **VDSI**.
3. In the search field, paste:

```
None
SSO_28054_JGBV_SOE_Jenkins-Prod_osl_buildengineer
```

4. Click **Request access to this resource**.
5. Ensure your username is selected under the self-grant option.
6. Provide the business reason: OSND project onboarding.
7. Check **"I certify that this request is for legitimate Verizon business"**.
8. Click **Submit Access Request** and notify your manager for approval.

3. Operations & Tooling Guide

3.1 Kubernetes Management with K9s

k9s is the primary terminal-based UI for managing OSND Kubernetes clusters.

3.2 MongoDB Access Procedure

MongoDB is hosted externally as a SaaS instance outside our main AWS account. To connect from your local environment via a bastion host:

3.3 Infrastructure as Code (Terraform)

3.1 Kubernetes Management with K9s

k9s is the primary terminal-based UI for managing OSND Kubernetes clusters.

Navigation Commands

Enter : to open command mode in K9s:

Command	Action
:pod or :po	Switch to Pods view
:ns	Switch Namespaces
:ctx	Switch Kubernetes Contexts
:node	Switch to Worker Nodes view
Ctrl + a	Show resources across all namespaces
/	Filter resources in current view by name/label

Pod Management Shortcuts

Shortcut	Action	Description
l	**Logs**	Stream pod logs (0 for all containers, 1-9 for specific container)
s	**Shell**	kubectl exec into selected pod container
d	**Describe**	Run kubectl describe pod
e	**Edit**	Open pod's live YAML in default text editor
v	**View**	View read-only YAML definition
Ctrl + d	**Delete**	Delete / terminate selected pod (standard grace period)
Ctrl + k	**Kill**	Force-kill selected pod (grace-period=0)
r	**Port-Forward**	Set up port forwarding (:pf views active forwards)
Esc	**Back**	Exit current view / return to previous screen

3.2 MongoDB Access Procedure

MongoDB is hosted externally as a SaaS instance outside our main AWS account. To connect from your local environment via a bastion host:

1. **Retrieve Connection String:**
   Locate secret.yaml in the flux-config-appcode repository under the target environment folder:

```
None
flux-config-appcode -> <env>-config -> oslte -> <microservice-name> -> secret.yaml
```

2. **Decode URI:**
   Extract the value for spring.data.mongodb.uri and decode it:

```
None
echo "<base64_encoded_uri>" | base64 -d
```

3. **Establish Connection:**
   Connect to an AWS instance within the targeted environment's AWS account and launch mongosh:

```
None
mongosh "<decoded_connection_string>"
```

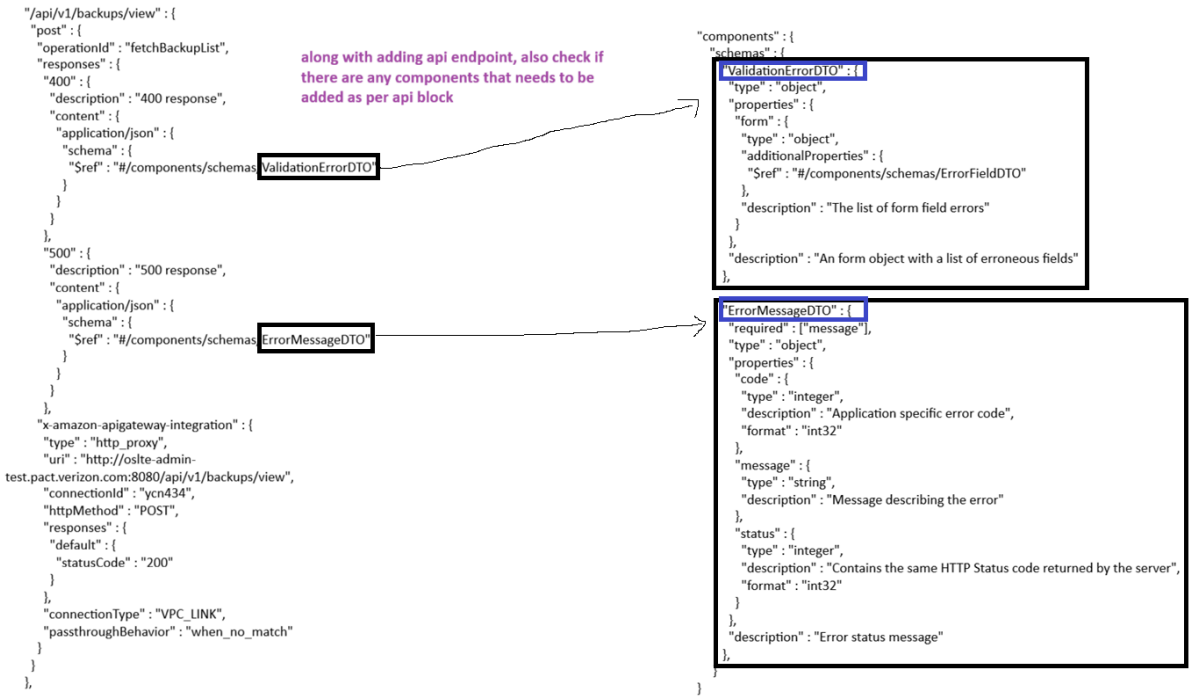
Now you can start executing DB scripts.

3.3 Infrastructure as Code (Terraform)

- **Repository:** [osnd-aws-infra]

Adding API Gateways

When dev/QA teams provide a new gateway ID, base URL, or gateway name, update the corresponding API block modules in osnd-aws-infra. Check for dependent components required by the spec.



Newrelic Helm

To configure and deploy the newrelic-bundle Helm chart on a **new EC2 instance**, **different environment** (e.g., prod, staging, dev), and a **different EKS/Kubernetes cluster**, follow this complete setup guide.

Step 1: Set Up Prerequisites on the New EC2 Instance

Ensure your new EC2 instance has CLI access configured to reach the target cluster.

Bash

```
None
# 1. Verify kubectl is pointing to the correct new cluster context
kubectl config current-context
kubectl get nodes

# 2. Add and sync the official New Relic Helm repository
helm repo add newrelic https://helm-charts.newrelic.com
helm repo update
```

Step 2: Create an Environment-Specific values.yaml File

Create a tailored values-<environment>.yaml file (e.g., values-prod.yaml) on your EC2 instance. Make sure to update the **cluster name**, **environment label**, and **license key**:

YAML

```
None
# values-prod.yaml
global:
  # Set your new cluster name
  cluster: "pact-oslte-eks-prod-spog-cluster"

  # Insert your New Relic ingest license key
  licenseKey: "YOUR_NEW_RELIC_LICENSE_KEY_HERE"

  # Set environment label (e.g., prod, staging, dev)
  labels:
    osnd.env: prod

  # Mandatory for AWS EKS Fargate environments
  fargate: true
```

```
# Enable the infrastructure sidecar operator and metadata injection
newrelic-infra-operator:
  enabled: true

nri-metadata-injection:
  enabled: true
  ignoreNamespaces:
    - kube-system
    - kube-public
    - kube-node-lease

# Enable Kubernetes event monitoring & state metrics
infrastructure:
  enabled: true

ksm:
  enabled: true

kubeEvents:
  enabled: true

# Disable unused modules to keep the deployment lightweight
newrelic-logging:
  enabled: false

newrelic-prometheus-agent:
  enabled: false

pixie-chart:
  enabled: false
```

Step 3: Deploy the Helm Release to the New Cluster

Run helm upgrade --install using your dedicated values file:

Bash

```
None
helm upgrade --install newrelic-bundle newrelic/nri-bundle \
  --namespace newrelic \
  --create-namespace \
  -f values-prod.yaml
```

Step 4: Verify the Installation

Check that the New Relic operator and admission webhook pods are running in the newrelic namespace:

1.Check Pod Status: Deployment Check.
Ensure all operator pods in the newrelic namespace reach the Running state:

Bash

```
None
kubectl get pods -n newrelic
```

2.Verify Mutating Webhook: Webhook Check.
Confirm that the Mutating Webhook Configuration was generated and registered successfully:

Bash

```
None
kubectl get mutatingwebhookconfigurations -l
app.kubernetes.io/name=newrelic-infra-operator
```

3.Verify Sidecar Auto-Injection: Test Workload Injection.
Deploy or restart a workload in an application namespace and check if the infrastructure-agent sidecar gets attached:

Bash

```
None
kubectl rollout restart deployment <your-app-deployment> -n
<your-app-namespace>
kubectl get pod <your-app-pod-name> -n <your-app-namespace> -o
jsonpath='{.spec.containers[*].name}'
```

Expected Output: your-app-container infrastructure-agent

Managing New Relic Sidecar Injection in AWS EKS Fargate: Architecture, Configuration, and Lifecycle Control

This technical reference guide covers the architecture, configuration inspection, and operational procedures for controlling New Relic sidecar auto-injection within an AWS EKS Fargate cluster using the newrelic-bundle Helm release.

1. Core Summary & Objectives

On AWS EKS Fargate, traditional Kubernetes DaemonSets cannot run because worker nodes are managed as isolated, ephemeral execution environments. To achieve observability, New Relic uses a **Mutating Admission Webhook** (newrelic-infra-operator / nri-metadata-injection) to dynamically inject an infrastructure-agent sidecar container into target application Pods at creation time.

Objectives

- Inspect Helm configuration values (COMPUTED VALUES) for deployed components.
- Understand the internal workflow of Kubernetes Mutating Admission Webhooks during sidecar injection.
- Identify node runtime constraints (EKS Fargate unprivileged requirements).
- Safely disable sidecar auto-injection cluster-wide—both temporarily (without cluster degradation) and via Helm configuration updates—and restore functionality when needed.

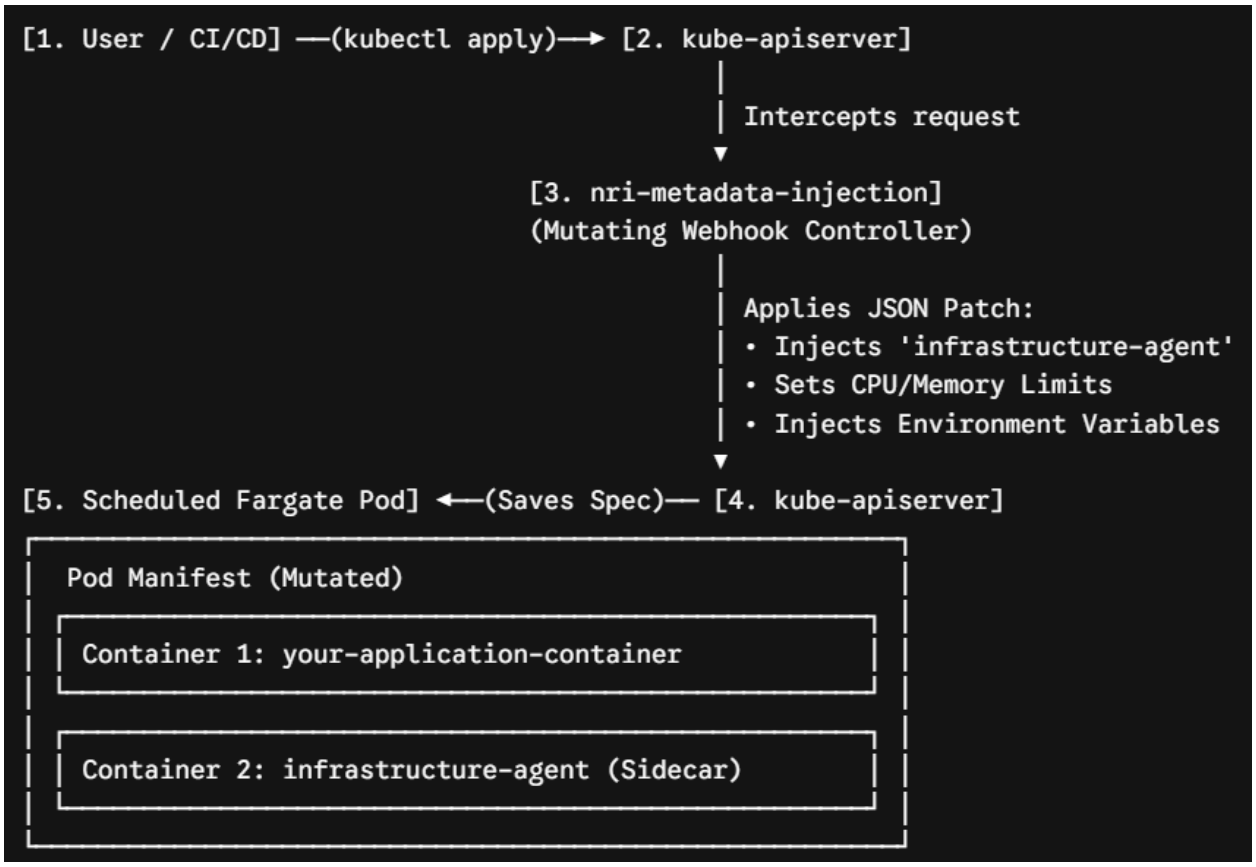
2. Key Concepts & Definitions

Technical Term	Definition & Role in Environment
newrelic-bundle	Umbrella Helm chart combining New Relic observability components (Infrastructure, Metrics, Events, Operator).
Mutating Admission Webhook	An API server plugin that intercepts Pod creation requests before persisting specs to etcd, allowing JSON patches to modify Pod definitions.

newrelic-infra-operator	Kubernetes Operator managing webhook certificates and injecting sidecar containers into matching Pods.
infrastructure-agent	The injected sidecar container (newrelic/infrastructure-k8s) running co-located inside the Pod to collect metrics via localhost.
failurePolicy	A webhook configuration attribute determining whether Kubernetes allows (Ignore) or blocks (Fail) Pod creation if the webhook endpoint is unreachable.
EKS Fargate Profile	AWS-managed compute profile where each Pod runs in its own isolated VM, requiring sidecars instead of node-level DaemonSets.

3. Architecture & Injection Mechanics

Mutating Webhook Sequence Flow



Injected Sidecar Configuration (Extracted from Values)

- **Image:** newrelic/infrastructure-k8s:2.13.15-unprivileged
- **Resource Limits:** CPU 200m, Memory 100M
- **Resource Requests:** CPU 100m, Memory 50M
- **Ignored Namespaces:** kube-system, kube-public, kube-node-lease
- **Default Environment Overrides:** DISABLE_KUBE_STATE_METRICS="true"

4. Operational Guidelines & Workflows

Phase A: Inspecting Current Configuration

To view the full merged configuration (defaults + custom overrides):

Bash

```
None
helm get values newrelic-bundle -n newrelic --all
```

To check the current active failurePolicy of all webhooks in the cluster:

Bash

```
None
kubectl get mutatingwebhookconfigurations \
  -o jsonpath='{range
.items[*].webhooks[*]}{.name}{"\tFailurePolicy:
"}{.failurePolicy}{"\n"}{end}'
```

Phase B: Temporarily Disabling Injection (Fast Operational Path)

To immediately halt sidecar injection without running a Helm upgrade (ideal when external repo access is restricted):

Bash

```
None
# Step 1: Scale down the operator deployment to 0 replicas
kubectl scale deployment -n newrelic \
  -l app.kubernetes.io/name=newrelic-infra-operator \
  --replicas=0

# Step 2: Delete the mutating webhook registration to eliminate API
server timeouts
kubectl delete mutatingwebhookconfiguration \
  -l app.kubernetes.io/name=newrelic-infra-operator

# Step 3: Restart workload deployments to recreate Pods without
sidecars
kubectl rollout restart deployment <your-app-deployment> -n
<your-app-namespace>
```

Phase C: Re-enabling Injection

When sidecar telemetry is needed again:

Bash

```
None
# Step 1: Scale the operator back up (the operator automatically
recreates missing webhooks)
kubectl scale deployment -n newrelic \
  -l app.kubernetes.io/name=newrelic-infra-operator \
  --replicas=1
```

```
# Step 2: Restart workload deployments to reinject sidecars
kubectl rollout restart deployment <your-app-deployment> -n
<your-app-namespace>
```

Phase D: Disabling via Helm Upgrade (Configuration-as-Code Path)

When repo access or offline chart archives are available:

Bash

```
None
# Step 1: Register and sync the repo
helm repo add newrelic https://helm-charts.newrelic.com
helm repo update

# Step 2: Disable operator and injection values
helm upgrade newrelic-bundle newrelic/nri-bundle \
  --namespace newrelic \
  --reuse-values \
  --set newrelic-infra-operator.enabled=false \
  --set nri-metadata-injection.enabled=false \
  --wait=false
```

5. Clarified Doubts & Troubleshooting

Q1: helm upgrade returned Error: failed to download "newrelic/nri-bundle"

- **Cause:** The chart alias newrelic/nri-bundle requires the newrelic repository to be added locally via helm repo add.
- **Fix:** Use --repo https://helm-charts.newrelic.com along with the chart identifier nri-bundle directly:
- Bash

```
None
helm upgrade newrelic-bundle nri-bundle \
  --namespace newrelic \
  --reuse-values \
  --repo https://helm-charts.newrelic.com
```

Q2: helm upgrade hangs indefinitely with a blinking cursor

- **Cause:** The EC2 host lacked outbound internet access to contact helm-charts.newrelic.com, OR Kubernetes was waiting for webhook endpoints to respond during deletion routines.
- **Fix:** Cancel the process (Ctrl + C), pass --wait=false, or execute the direct kubectl webhook cleanup path.

Q3: Can I scale the deployment to 0 replicas directly without deleting the Mutating Webhook?

- **Resolution: Yes**, because the output confirmed newrelic-infra-operator.newrelic.com uses FailurePolicy: Ignore. However, if the webhook registration remains active while the operator pod is down, kube-apiserver will experience a 10–30 second timeout delay on every Pod creation attempt before falling back and deploying without the sidecar. Deleting the webhook registration prevents this latency.

6. Best Practices & Pitfalls

Best Practices

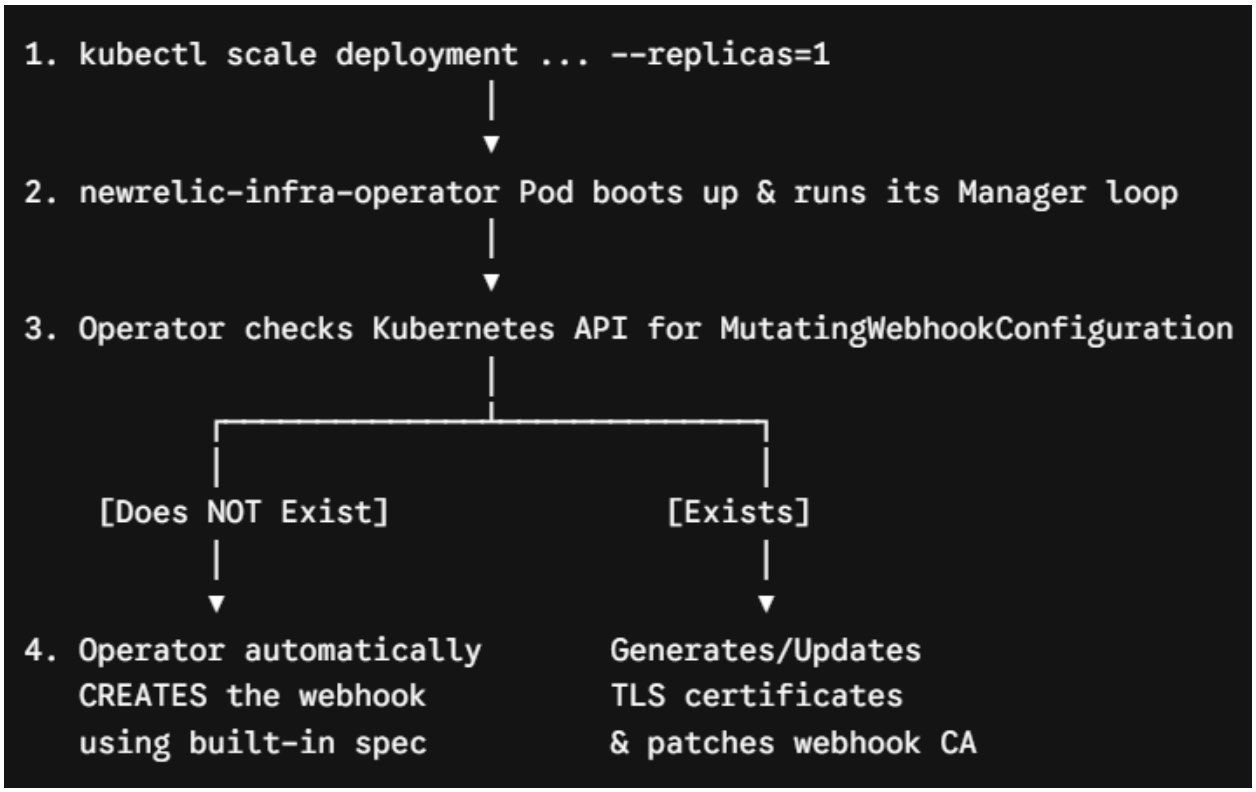
- **Verify failurePolicy First:** Always verify that third-party webhooks use failurePolicy: Ignore before scaling down operator deployments in production clusters.
- **Namespace Isolation:** Exclude system namespaces (kube-system, kube-public, kube-node-lease) from sidecar mutation to prevent bootstrapping cycles.
- **Clean Rollouts:** Remember that modifying webhooks or operators **only affects newly created Pods**. Existing Pods require a kubectl rollout restart to add or remove sidecars.

Pitfalls to Avoid

- **failurePolicy: Fail Locked Clusters:** Never scale an admission controller to 0 if its webhook uses failurePolicy: Fail. Doing so will block all new Pod creations cluster-wide.
- **Helm Locks (pending-upgrade):** Interrupting helm upgrade prematurely can leave the release in pending-upgrade. Resolve this by running helm rollback <release-name> <revision> to return to the last stable state.
- **Fargate Image Incompatibility:** Ensure sidecars targeted for AWS EKS Fargate use rootless/unprivileged tags (newrelic/infrastructure-k8s:*-unprivileged), as Fargate forbids privileged containers and host volume mounts.

Q) When you scale the operator back to `replicas=1`, the creation of the `MutatingWebhookConfiguration` happens automatically via the Operator's internal controller reconcile loop.

How the Automatic Recreation Works



The Mechanism Step-by-Step

1. **Bootstrap Initialization:** As the `newrelic-infra-operator` container starts up, its Go-based controller loop executes an initialization phase.
2. **Resource Reconcile Check:** The operator queries the `kube-apiserver` to inspect its required resources (`MutatingWebhookConfiguration`, Secrets, TLS certificates).
3. **Automatic Creation & Registration:** Finding that the `MutatingWebhookConfiguration` object is missing, the operator generates it in-memory from its embedded manifests and issues a `POST` request to `/apis/admissionregistration.k8s.io/v1/mutatingwebhookconfigurations`.
4. **TLS CA-Bundle Injection:** The operator issues/refreshes the self-signed TLS certificates for the webhook service port and patches the `caBundle` field inside the newly created webhook manifest so `kube-apiserver` can securely call it over HTTPS.

How to Verify It Came Back

After scaling back up:

Bash

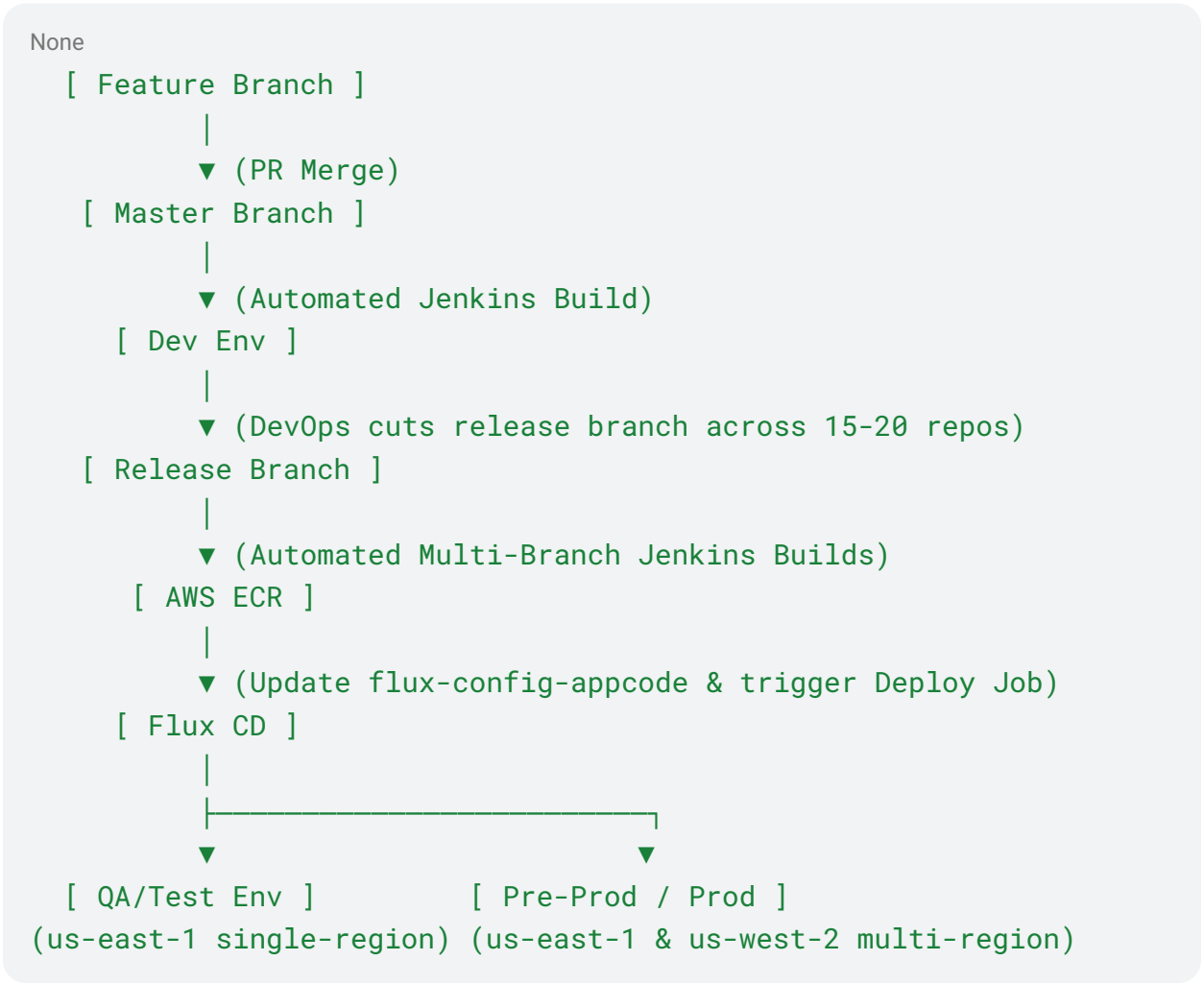
```
None
# 1. Scale operator back to 1
kubectl scale deployment -n newrelic -l
app.kubernetes.io/name=newrelic-infra-operator --replicas=1

# 2. Wait a few seconds for the operator pod to reach Running state
kubectl get pods -n newrelic -l
app.kubernetes.io/name=newrelic-infra-operator

# 3. Verify the MutatingWebhookConfiguration was automatically
recreated
kubectl get mutatingwebhookconfigurations -l
app.kubernetes.io/name=newrelic-infra-operator
```

Expected Output: The webhook will reappear automatically with a new age (e.g., `AGE: 5s`).

4. Software Delivery & Deployment Workflows



Microservice Release Branching Strategy & Workflow

The OSND platform comprises approximately 15 to 20 microservices, each maintained in its own dedicated Bitbucket repository. The release and deployment workflow across environments follows a structured pipeline:

1. Feature Development & Dev Deployments

- Developers implement new features and fixes in short-lived `feature/*` branches created from `master`.
- Upon completion, a Pull Request (PR) is opened against the `master` branch.
- Merging a PR into `master` automatically triggers a Jenkins pipeline that builds the application artifact and deploys it directly to the **Dev** environment.

2. Release Branching Procedure

Once developers complete testing and sign off on feature functionality in the Dev environment, the DevOps team is tasked with creating a unified release branch.

- **Branching Standard:** A single release branch name (e.g., `release/MMP3-PI-41-Sprint-24`) is applied consistently across all microservice repositories.
- **Automation Script:** To eliminate manual overhead, DevOps engineers execute the standardized branch-cutting script located at:

```
None
```
`flux-config-appcode/scripts/cut_branch.sh`

- **Execution Process:** The script reads the list of microservice repositories, clones or updates them, cuts the specified release branch from `master`, and pushes the new branch to origin in parallel.

4.1 Automated Branching Setup (cut_branch.sh)

When cutting release branches across all 15–20 microservices, we can use the automated script in flux-config-appcode. Or we can manually cut branches through the bit bucket UI.

1. Execute cut_branch.sh with the target release name (e.g., release/MMP3-PI-41-Sprint-24).

   Or

2. Manually create release branches for all micro-services

Note: MMP3-PI-41-Sprint-24 → branch name will be shared by `Sunil Yadawad` over a mail

Execution Walkthrough

Bash

```
None
# 1. Perform a dry-run first to inspect all steps
./scripts/cut_branch.sh -l scripts/repos.txt -r -s master -t
release/MMP3-PI-41-Sprint-24 -y -n

# 2. Execute branch creation locally and push to remote Stash
repositories
./scripts/cut_branch.sh -l scripts/repos.txt -r -s master -t
release/MMP3-PI-41-Sprint-24 -y -p
```

Script Flags Reference

Flag	Purpose
-l	Path to repository list file (scripts/repos.txt)
-r	Treat entries in repos.txt as remote Git URLs (automatically handles temp cloning)

-s	Source branch (defaults to master)
-t	Target release branch name (e.g., release/MMP3-PI-41-Sprint-24)
-y	Non-interactive mode (prevents double-prefixing branch names)
-n	Dry-run mode (echoes commands without modifying repos)
-p	Push created branches to origin

Important Gotcha: Avoid using the -B flag. The script's option parser contains a bug handling -B arguments. Passing -y and including the explicit release/ prefix in -t avoids interactive prompts and parser errors.

This operational reference documents the inner mechanics, options, known gotchas, and execution steps for the release branching automation script used across OSND microservice repositories.

1. Script Architecture & Execution Flow

The `cut_branch.sh` script automates cutting release or custom branches across all microservices listed in `scripts/repos.txt`.

Core Processing Workflow

1. **Argument Parsing & Validation:** Uses `getopts` to parse flags. Validates that the repository list (`-l`) and target branch (`-t`) are provided, and that the specified list file exists.
2. **Branch-Type Resolution:** If non-interactive mode (`-y`) is not set and no branch type (`-B`) is provided, the script interactively prompts to select a branch type (`release/`, `custom/`, or `none`).

3. **Repository Parsing:** Iterates line-by-line through `repos.txt`, ignoring blank lines and lines starting with `#`.
4. **Clone & Checkout Execution (Per Entry):**
    - **Remote URLs (`-r` enabled):** Clones the repository into a temporary workspace directory (`mktemp -d`).
    - **Local Paths (`-r` disabled):** Validates that the entry points to a valid local directory containing a `.git` folder.
    - **Branch Operations:** Fetches remote updates (`git fetch --all --prune`), verifies or checks out the source branch (`-s`), and creates the target branch (`-t`).
    - **Push (`-p` enabled):** Pushes the newly created branch back to `origin`.
5. **Completion:** Prints the path to the temporary execution directory and reports final status.

3. Known Issues & Operational Gotchas

> **Critical Parser Bug (`-B` Flag):**
>
> The internal `getopts` specification defines the flag without a colon parameter (`:l:s:t:pnhrBy:`). Consequently, `-B` does not consume an argument string. Passing `-B release` as shown in legacy documentation can cause silent failure or crash scripts relying on `set -u` (unbound variable).

Recommended Workaround

Do **not** pass the `-B` flag. Instead:

1. Provide the full prefixed branch name directly in the target flag: `-t release/<branch-name>`.
2. Pass the non-interactive flag `-y` to skip the interactive select menu and prevent double-prefixing.

4. Standard Operating Execution Steps

Step 1: Execute Dry-Run Verification

Run the script with the `-n` flag to simulate all Git commands and inspect output without making remote or local changes:

Bash

```
None
./scripts/cut_branch.sh \
  -l scripts/repos.txt \
  -r \
  -s master \
  -t release/MMP3-PI-41-Sprint-24 \
  -y \
  -n
```

Step 2: Perform Branch Creation & Push to Remote

After confirming the dry-run output, remove `-n` and add `-p` to create and push branches across all microservices:

Bash

```
None
./scripts/cut_branch.sh \
  -l scripts/repos.txt \
  -r \
  -s master \
  -t release/MMP3-PI-41-Sprint-24 \
  -y \
  -p
```

5. Post-Execution Checklist

- **Authentication:** Ensure local Git credentials (SSH key or PAT for `ctlabs.verizon.net`) are active before using `-p`.

- **Temp Workspace:** When using `-r`, the script clones into a system temporary directory (`/tmp/tmp.XXXXXXXXXX`). The script outputs this location at completion for manual inspection if needed.
- **Cleanup:** Temporary directories are retained post-execution and can be removed manually once branching is verified.

4.2 QA Environment Release & Deployment Standard Operating Procedure (SOP)

Follow this step-by-step operational guide to cut a release branch across all microservices and roll out the build to the **QA (Testing)** environment.

1.Cut Release Branches Across Microservices:
Pre-requisite: Verification of Dev sign-off.
Execute the branch-cutting script to create the requested release branch (e.g., release/MMP3-PI-41-Sprint-24) from master across all microservice repositories:

Bash

```
None
./scripts/cut_branch.sh -l scripts/repos.txt -r -s master -t
release/MMP3-PI-41-Sprint-24 -y -p
```

2.Verify & Configure Jenkins Multi-Branch Pipelines:

1. Navigate to the **OSND Multi-Branch Pipelines Portal**.
2. Ensure every microservice has an associated pipeline entry matching its repository name (e.g., repository → oslte-spog-admin
3. Pipeline → mb-oslte-spog-admin-pipeline-soe).
4. *If a microservice is new, manually create a corresponding pipeline job.*
5. Update the pipeline configuration for each microservice to recognize the new release branch name.
6. Saving the configuration will auto-trigger build jobs, compiling code and pushing the new container images to **AWS ECR**.

3.Update Flux Configuration Manifests:

1. Open the flux-config-appcode repository.
2. Navigate to test-config/auto-deploy.conf.
3. Update the branch parameter to match the release branch name (e.g., MMP3-PI-41-Sprint-24).
4. Commit and push changes directly to the remote master branch.

4.Trigger QA Deployment Pipeline:
Run the **OSND QA Deploy Jenkins Job**.

> **Note:** This job automatically updates the deployment YAML manifests for all microservices in <env>-config with the latest ECR image tags and commits the changes back to flux-config-appcode. **Flux CD** will detect these repository updates and automatically reconcile the resources on the QA EKS cluster.

5.Execute Manual Configuration & Database Scripts:
Refer to the **Confluence Development Configuration Changes Document** and perform the required release steps:

- Execute all documented database scripts.
- Apply required API gateway and environment configurations.
- **Exclusion:** Skip the **ENM CBRS** configuration steps (these are owned and handled separately by an external team).

6.Send Stakeholder Notification:
Once Flux completes deployment and manual configuration steps are verified, send an official email broadcast to the project stakeholders confirming that the latest release builds are live in the QA environment.

4.3 Pre-Production Promotion Standard Operating Procedure (SOP)

Follow this step-by-step operational guide when promoting a validated release build (e.g., MMP3-PI-41-Sprint-24) from the QA environment into the **Pre-Production (Staging)** multi-region environment.

1.Update Pre-Prod Auto-Deploy Configuration:

Pre-requisite: Successful verification in QA environment.

1. Open the flux-config-appcode repository.
2. Navigate to preprod-config/auto-deploy.conf.
3. Update the branch parameter to match the verified release branch name (e.g., MMP3-PI-41-Sprint-24).
4. Commit and push the updated configuration to the remote master branch.

2.Trigger Pre-Production Deployment Job:

Execute the **OSND Pre-Prod Deploy Jenkins Job**.

> **Mechanism:** This job fetches the target ECR image tags corresponding to the release branch, updates the Kubernetes deployment manifests in preprod-config, and merges the updated manifests back into the master branch of flux-config-appcode.

3.Verify Automated Multi-Region Rollout via Flux CD:

Monitor **Flux CD** as it detects the committed configuration changes in flux-config-appcode.

- **Multi-Region Scope:** Flux will reconcile the manifests and automatically roll out the updated microservice images across EKS clusters in both active regions:
    - us-east-1 (Primary)
    - us-west-2 (Secondary)

4.Execute Manual Configuration & Database Scripts:

Access the **Confluence Development Configuration Changes Document** and carry out the required updates for Pre-Prod:

- Execute all documented database migration scripts.
- Apply required API gateway routing and environment settings.
- **Exclusion:** Skip **ENM CBRS** configuration steps (owned and managed separately by an external team).

5.Send Promotion Notification Email:

After verifying multi-region pod status and completing manual configurations, broadcast an email notification to stakeholders confirming that the build has been successfully promoted to Pre-Production.

Technical Debt / Known Issues:

Issue 1: High Execution Time in Release Branching Automation

- **Component:** Release Branch Cutting Script (`flux-config-appcode/scripts/cut_branch.sh`)
- **Impact:** High deployment delay / Manual workaround required
- **Description:**
  The automated branch-cutting script currently processes repositories sequentially, performing full network clones, fetches, and pushes for each service one by one. Across 15 to 20 microservices, this sequential network dependency causes severe performance overhead. The script execution takes longer to complete than manually navigating Bitbucket to create the branches across all repositories, defeating the purpose of the automation.

Issue 2: Limitations in Jenkins Multi-Branch Pipeline Configuration

- **Component:** Jenkins Multi-Branch Pipelines (`SOE-AWS-OSND/OSND multi-branch pipelines/`)
- **Impact:** Maintenance overhead and inefficient resource utilization
- **Description:**
  - **Manual Configuration Overhead:** Pipeline configurations currently require manual updates to hardcode the exact release branch name for every new sprint or release cycle.
  - **Broad Wildcard Inefficiency:** Applying a broad wildcard rule (e.g., `release/*`) avoids manual updates, but it causes Jenkins to discover, scan, and track all historical release branches simultaneously. This leads to redundant pipeline indexing, unnecessary resource consumption, and potential accidental triggers on legacy release branches.

Issue 3: Terraform

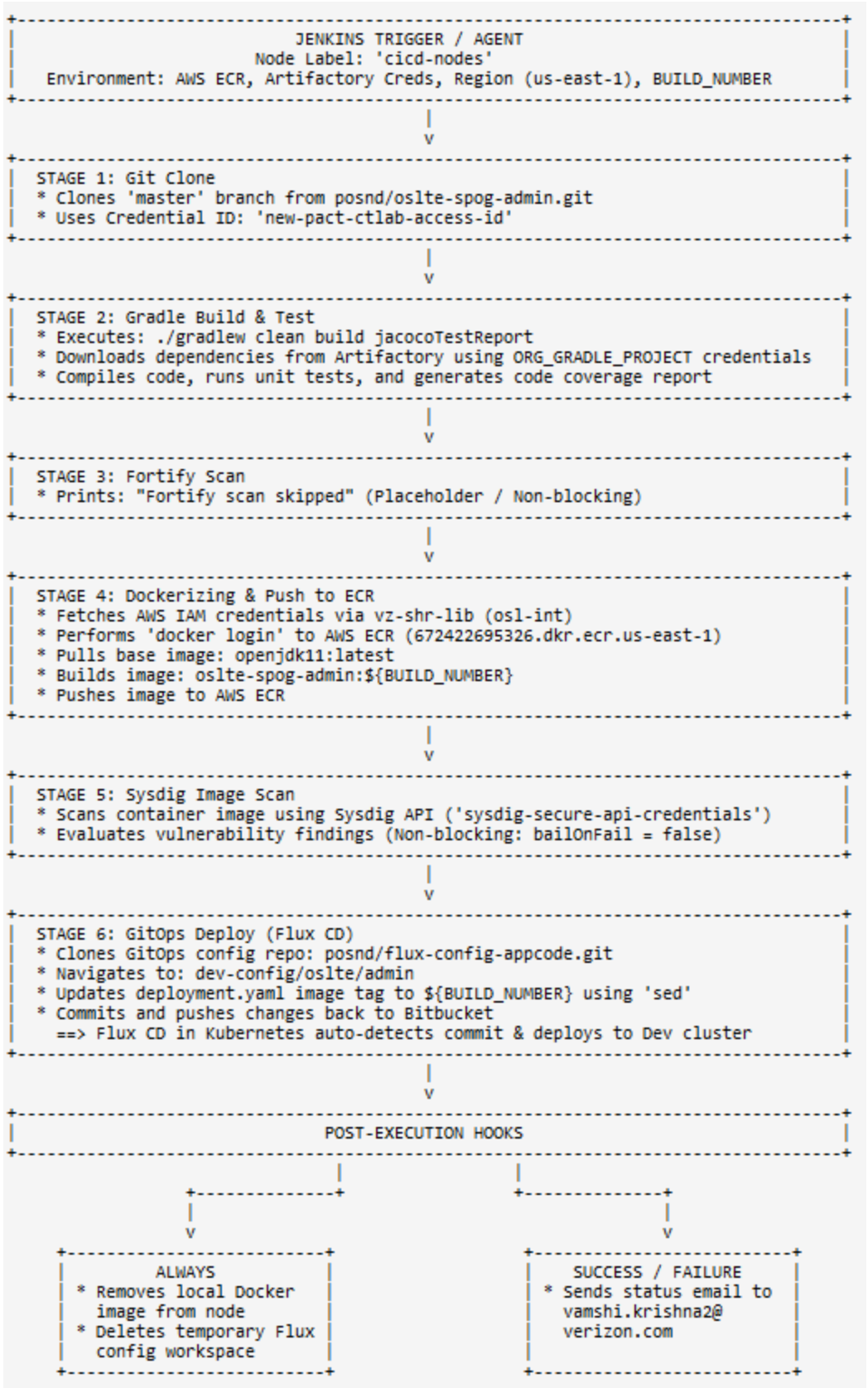
- **No modular structure:** Infrastructure code lacks modularization.
- **No remote state management:** State files are not centrally managed.
- **Code Duplication:** Environment configurations are duplicated across repository paths rather than parametrized.

Jenkins Pipeline for micro services:

```
None
@Library('vz-shr-lib') _
pipeline {
    agent { node { label 'cicd-nodes' } }
    environment {
        BUILD_NUMBER                        = "${currentBuild.getNumber()}"
        OSND_DOCKER_REGISTRY                =
"672422695326.dkr.ecr.us-east-1.amazonaws.com"
        AWS_DEFAULT_REGION                  = "us-east-1"
        CI                                  = false
        ARTIFACTORY_CRED                    = credentials('artifactory-user-id')
        ORG_GRADLE_PROJECT_readUser         = "${env.ARTIFACTORY_CRED_USR}"
        ORG_GRADLE_PROJECT_readPwd          = "${env.ARTIFACTORY_CRED_PSW}"
        ORG_GRADLE_PROJECT_ARTIFACTORY_URL =
"https://oneartifactoryci.verizon.com/artifactory/I5NV_OSND-prod"
    }
    stages {
        stage('git clone oslte-spog-admin') {
            steps {
                git branch: 'master',
                    credentialsId: 'new-pact-ctlab-access-id',
                    url:
'https://ctlabs.verizon.net/stash/scm/posnd/oslte-spog-admin.git'
            }
        }
        stage('oslte-spog-admin build') {
            steps {
                sh './gradlew clean build jacocoTestReport'
            }
        }
        stage('Fortify Scan') {
            steps {
                echo 'Fortify scan skipped'
            }
        }
        stage('Dockerizing oslte-spog-admin') {
            steps {
                script {
                    def utils = new com.vzc.devops.Utils()
                    def awsCreds = utils.getAwsCredsForExport('osl-int',
env.AWS_DEFAULT_REGION)
                    sh """
```

```
                        set -e
                        ${awsCreds} aws ecr get-login-password --region
${AWS_DEFAULT_REGION} | docker login --username AWS --password-stdin
${OSND_DOCKER_REGISTRY}
                        docker pull ${OSND_DOCKER_REGISTRY}/oslte/openjdk11:latest
                        docker build -t
${OSND_DOCKER_REGISTRY}/oslte/oslte-spog-admin:${BUILD_NUMBER} --build-arg
OPEN_JDK_IMAGE=${OSND_DOCKER_REGISTRY}/oslte/openjdk11:latest .
                        ${awsCreds} docker push
${OSND_DOCKER_REGISTRY}/oslte/oslte-spog-admin:${BUILD_NUMBER}
                    """
                }
            }
        }
        stage('Sysdig Image Scan') {
            steps {
                script {
                    sysdigImageScan engineCredentialsId:
'sysdig-secure-api-credentials',
                        imageName:
"${OSND_DOCKER_REGISTRY}/oslte/oslte-spog-admin:${BUILD_NUMBER}",
                        bailOnFail: false
                }
            }
        }
        stage('Deploy oslte-spog-admin') {
            steps {
                withCredentials([
                    gitUsernamePassword(credentialsId: 'new-pact-ctlab-access-id',
gitToolName: 'Default')
                ]) {
                    sh """
                        set -e
                        git config --global user.email "AWS-Factories@amazon.com"
                        git config --global user.name "AWS-Factories"
                        git clone
https://ctlabs.verizon.net/stash/scm/posnd/flux-config-appcode.git
flux-config-appcode-admin-${BUILD_NUMBER}
                        cd
flux-config-appcode-admin-${BUILD_NUMBER}/dev-config/oslte/admin
                        sed -i "s|\\([[:space:]]*\\)image:
${OSND_DOCKER_REGISTRY}/oslte/oslte-spog-admin:.*|\\1image:
${OSND_DOCKER_REGISTRY}/oslte/oslte-spog-admin:${BUILD_NUMBER}|" deployment.yaml
                        git diff --quiet || (git add deployment.yaml && git commit
-m "Deploy oslte-spog-admin build# ${BUILD_NUMBER} to Dev environment.")
```

```
                        git config pull.rebase false
                        git pull --no-edit
                        git push origin master
                    """
                }
            }
        }
    }
    post {
        always {
            sh """
                docker rmi -f
${OSND_DOCKER_REGISTRY}/oslte/oslte-spog-admin:${BUILD_NUMBER} || true
                rm -rf ${WORKSPACE}/flux-config-appcode-admin-${BUILD_NUMBER} ||
true
            """
        }
        success {
            emailext subject: "SUCCESS: oslte-spog-admin build ${BUILD_NUMBER}",
                body: "Build ${BUILD_NUMBER} succeeded.",
                to: "vamshi.krishna2@verizon.com",
                from: "donotreply@verizon.com"
        }
        failure {
            emailext subject: "FAILURE: oslte-spog-admin build ${BUILD_NUMBER}",
                body: "Build ${BUILD_NUMBER} failed.",
                to: "vamshi.krishna2@verizon.com",
                from: "donotreply@verizon.com"
        }
    }
}
```


JENKINS TRIGGER / AGENT
Node Label: 'cicd-nodes'
Environment: AWS ECR, Artifactory Creds, Region (us-east-1), BUILD_NUMBER
STAGE 1: Git Clone
* Clones 'master' branch from posnd/oslte-spog-admin.git
* Uses Credential ID: 'new-pact-ctlab-access-id'
STAGE 2: Gradle Build & Test
* Executes: ./gradlew clean build jacocoTestReport
* Downloads dependencies from Artifactory using ORG_GRADLE_PROJECT credentials
* Compiles code, runs unit tests, and generates code coverage report
STAGE 3: Fortify Scan
* Prints: "Fortify scan skipped" (Placeholder / Non-blocking)
STAGE 4: Dockerizing & Push to ECR
* Fetches AWS IAM credentials via vz-shr-lib (osl-int)
* Performs 'docker login' to AWS ECR (672422695326.dkr.ecr.us-east-1)
* Pulls base image: openjdk11:latest
* Builds image: oslte-spog-admin:${BUILD_NUMBER}
* Pushes image to AWS ECR
STAGE 5: Sysdig Image Scan
* Scans container image using Sysdig API ('sysdig-secure-api-credentials')
* Evaluates vulnerability findings (Non-blocking: bailOnFail = false)
STAGE 6: GitOps Deploy (Flux CD)
* Clones GitOps config repo: posnd/flux-config-appcode.git
* Navigates to: dev-config/oslte/admin
* Updates deployment.yaml image tag to ${BUILD_NUMBER} using 'sed'
* Commits and pushes changes back to Bitbucket
==> Flux CD in Kubernetes auto-detects commit & deploys to Dev cluster
POST-EXECUTION HOOKS
ALWAYS
* Removes local Docker image from node
* Deletes temporary Flux config workspace
SUCCESS / FAILURE
* Sends status email to vamshi.krishna2@verizon.com

This Jenkinsfile defines an automated CI/CD pipeline for building, scanning, and deploying the microservice **`oslte-spog-osnd-admin`** using a GitOps deployment pattern (via **Flux CD**).

Here is a step-by-step breakdown of how the pipeline operates:

Header & Environment Setup

- **`@Library('vz-shr-lib') _`**: Imports a shared Jenkins Groovy library (`vz-shr-lib`) containing custom utility classes (used later for fetching AWS credentials).
- **`agent { node { label 'cicd-nodes' } }`**: Directs Jenkins to execute this pipeline on a worker node tagged with the label `cicd-nodes`.
- **`environment { ... }`**: Defines global variables:
  - **`OSND_DOCKER_REGISTRY`**: Points to an AWS Elastic Container Registry (ECR) repository (`us-east-1`).
  - **`ARTIFACTORY_CRED`**: Pulls Artifactory credentials using Jenkins credential ID `artifactory-user-id` and sets up variables (`readUser`, `readPwd`, URL) for Gradle dependencies.

Pipeline Stages

Stage 1: `git clone oslte-spog-osnd-admin`

- **Action:** Checks out the source code from the `master` branch of `[https://ctlabs.verizon.net/stash/scm/posnd/oslte-spog-osnd-admin.git](https://ctlabs.verizon.net/stash/scm/posnd/oslte-spog-osnd-admin.git)` using stored credentials (`new-pact-ctlab-access-id`).

Stage 2: `Dockerizing oslte-spog-osnd-admin`

- **Action:** Builds and pushes the Docker container image.
- **Process:**
  1. Instantiates `com.vzc.devops.Utils()` from the shared library to retrieve dynamic AWS credentials for the `osl-int` environment.
  2. Authenticates Docker to AWS ECR via `aws ecr get-login-password`.
  3. Builds the Docker image and tags it with the current Jenkins build number: $OSND_{D}OCKER_{R}EGISTRY/oslte/oslte - spog - ai - ops$:{BUILD_NUMBER}
  4. Pushes the newly tagged image to AWS ECR.

Stage 3: `Sysdig Scan`

- **Action:** Performs security vulnerability scanning on the newly built container image.

- **Process:**
    1. Writes the image name and tag into a file (`sysdig_secure_images`).
    2. Runs `sysdigImageScan` against Sysdig Cloud (`app.us3.sysdig.com`).
    3. Uses `bailOnFail: false` and a `try-catch` block so security scan warnings **do not block/break** the pipeline execution if vulnerabilities are detected.

Stage 4: `Deploy oslte-spog-osnd-admin` (GitOps Deployment)

- **Action:** Triggers deployment via GitOps by updating the application's Flux configuration repository.
- **Process:**
    1. Clones the GitOps repository `flux-config-appcode.git` into a workspace folder named `flux-config-appcode-spog-osnd-admin-${BUILD_NUMBER}`.
    2. Navigates to the Dev configuration path: `dev-config/oslte/spog-osnd-admin`.
    3. Uses `sed` to update `deployment.yaml`, replacing the existing image tag with the new `${BUILD_NUMBER}` tag.
    4. Commits the modified `deployment.yaml` back to Git with the message: `Deploy oslte-spog-osnd-admin build# ${BUILD_NUMBER} to Dev environment.`
    5. Pulls latest remote changes and pushes the commit back to Bitbucket.
    6. **Flux CD** watching that repository will automatically detect the Git commit and sync the updated deployment into the Kubernetes cluster.
    7. Cleans up the temporary GitOps folder.

Post Actions (`post`)

- **`always`**:
    - **`cleanWs()`**: Deletes all checked-out code and temporary files from the Jenkins workspace.
    - Removes the locally cached Docker image (`docker rmi -f ...`) from the Jenkins runner node to conserve disk space.
- **`success`**:
    - Sends an email notification to `vamshi.krishna2@verizon.com`, developers, and the build requester indicating the build succeeded.
- **`failure`**:
    - Sends an email notification indicating the build failed, with a direct link to the console log (`${env.BUILD_URL}/console`).

list of all pre-requisites for this pipeline to work

For this Jenkins pipeline to execute successfully from start to finish, the following system, infrastructure, network, and credential pre-requisites must be configured:

1. Jenkins Infrastructure & Agent Nodes

- **cicd-nodes Agent:** A Jenkins worker node configured with the label cicd-nodes.
- **Runtime Tools Installed on the Agent Node:**
  - **Git CLI** (configured and accessible in system PATH).
  - **Docker Engine** (running daemon with permissions for the jenkins user to run docker commands without sudo).
  - **AWS CLI v2** (installed for aws ecr get-login-password).
  - **Standard Unix Utilities:** sed, grep, bash, and rm.

2. Jenkins Global & Shared Libraries

- **Shared Library Configuration:** The pipeline shared library vz-shr-lib must be registered in Jenkins Global Pipeline Libraries (Manage Jenkins → System → Global Pipeline Libraries).
- **Groovy Utility Class:** The library must expose the package/class com.vzc.devops.Utils with a working getAwsCredsForExport(String envName, String region) method to generate AWS environment export scripts.

3. Jenkins Stored Credentials

The following credential IDs must exist in the Jenkins Credentials Store with appropriate access permissions:

Credential ID	Type	Used For
artifactory-user-id	Username with Password	Accessing Gradle dependencies from oneartifactoryci.verizon.com

new-pact-ctlab-access-id	Username with Password / SSH Key / PAT	Checkout from source Bitbucket (posnd/oslte-spog-osnd-mcp.git) and commit/push access to posnd/flux-config-appcode.git
sysdig-osnd-credential-id	Secret Text / API Token	Authenticating with Sysdig Secure Cloud (https://app.us3.sysdig.com)

4. Network & Firewall Access

The Jenkins agent node must have network/firewall connectivity to:

- **Bitbucket / Stash:** https://ctlabs.verizon.net (HTTPS access to Git repos).
- **AWS ECR Registry:** 672422695326.dkr.ecr.us-east-1.amazonaws.com (HTTPS port 443).
- **Artifactory:** https://oneartifactoryci.verizon.com (HTTPS port 443).
- **Sysdig Engine:** https://app.us3.sysdig.com (HTTPS port 443).

5. Cloud & External System Permissions

- **AWS IAM Permissions:** The credentials resolved by utils.getAwsCredsForExport('osl-int', ...) must have IAM policies allowing:
  - ecr:GetAuthorizationToken
  - ecr:BatchCheckLayerAvailability
  - ecr:InitiateLayerUpload
  - ecr:UploadLayerPart
  - ecr:CompleteLayerUpload
  - ecr:PutImage
- **AWS ECR Repository:** The repository oslte/oslte-spog-ai-ops must exist in AWS ECR account 672422695326 in us-east-1.
- **Bitbucket Repositories & Permissions:**
  - Read access to https://ctlabs.verizon.net/stash/scm/posnd/oslte-spog-osnd-mcp.git.
  - Read/Write (Push) access to https://ctlabs.verizon.net/stash/scm/posnd/flux-config-appcode.git.
- **Flux Config Directory Structure:** The GitOps repo (flux-config-appcode.git) must contain the exact path and file:

    - Path: dev-config/oslte/spog-osnd-mcp/deployment.yaml
    - File contents: deployment.yaml must contain a matching string pattern for image: 672422695326.dkr.ecr.us-east-1.amazonaws.com/oslte/oslte-spog-ai-ops:<tag>.

6. Jenkins Plugins Installed

- **Pipeline Shared Groovy Libraries Plugin**
- **Git Plugin**
- **Credentials Binding Plugin**
- **Sysdig Secure Plugin** (sysdigImageScan step)
- **Email Extension Plugin** (emailext step) with an SMTP server configured under Jenkins Global Mailer settings.

Standard Operating Procedure (SOP) & Technical Documentation

Topic: CloudBees/Jenkins Dynamic Kubernetes Agents Architecture

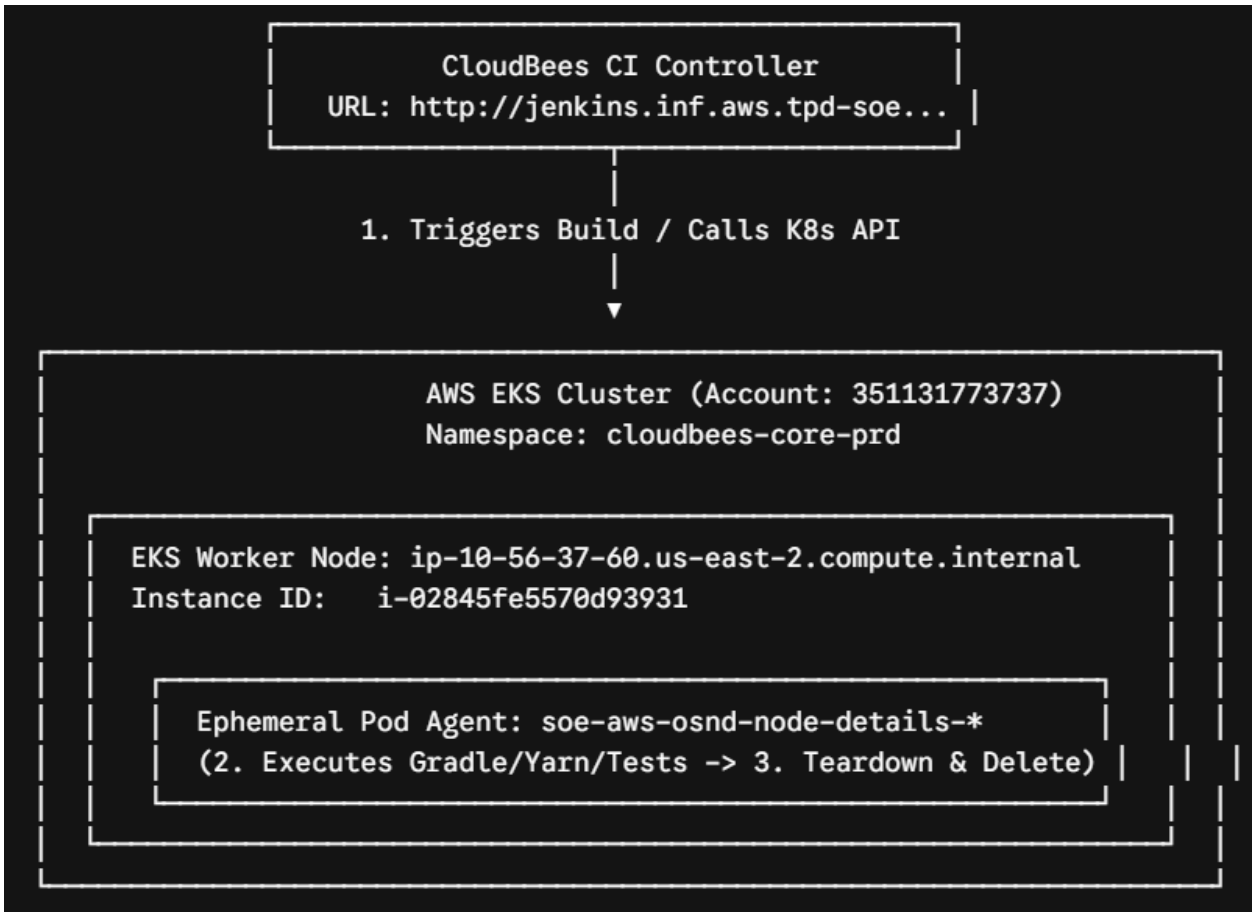
Target Environment: AWS EKS / CloudBees CI (osl Controller)

URL: https://jenkins.inf.aws.tpd-soe.net/osl/job/SOE-AWS-OSND

1. Architectural Overview

Our CI/CD setup utilizes the **Containerized Dynamic Jenkins Agents** paradigm powered by the **Jenkins Kubernetes Plugin**.

Instead of routing pipeline builds to permanent, static virtual machines (EC2 instances) that sit idle, our CloudBees Managed Controller (osl) communicates directly with an Amazon Elastic Kubernetes Service (EKS) cluster to provision short-lived **Pod Agents** on demand.



Build Lifecycle Flow:

1. **Trigger:** A developer pushes code or manually runs a Jenkins pipeline.
2. **Provisioning:** The CloudBees controller (osl) requests a new Kubernetes Pod from the EKS API server in the cloudbees-core-prd namespace.
3. **Execution:** The agent pod starts, pulls container images (e.g., JNLP agent, JDK, Node/Yarn dependencies), checks out source code, and runs build steps.
4. **Teardown:** Upon build completion (Success, Failure, or Abort), the Kubernetes plugin automatically terminates and deletes the agent pod, releasing CPU/RAM resources back to the cluster.

2. Verified AWS Infrastructure & Account Directory

To avoid confusion when submitting Cloud Infrastructure tickets or inspecting AWS consoles, refer to this account mapping:

Active Jenkins CI/CD Infrastructure (Where Jobs Execute)

- **AWS Account ID:** 351131773737
- **AWS Region:** us-east-2 (US East - Ohio)
- **Availability Zone:** us-east-2b
- **Deployment Model:** CloudBees CI on EKS Cluster
- **Kubernetes Namespace:** cloudbees-core-prd
- **Service Account:** jenkins-agents
- **Current EKS Worker Node Instance:** i-02845fe5570d93931 (ip-10-56-37-60.us-east-2.compute.internal)
- **Default Agent Container Image:** 351131773737.dkr.ecr.us-east-2.amazonaws.com/cloudbees/cloudbees-core-agent:2.555.3.36983

Application Deployment Account (Different / Separate Account)

- **AWS Account ID:** 672422695326 (pact-oslte-spog-depl)
- **AWS Region:** us-east-1 (US East - N. Virginia)
- **EC2 Instance:** pact-jenkins-agentnode0 (i-0dfdc90a49465bb58)
- **Usage:** Application hosting / Deployment targets. *(Note: Pipeline builds do **not** run on this instance).*

3. Key Benefits of This Architecture

- **Isolated Environments:** Every build executes in a fresh, pristine container environment. Previous build artifacts or stale caches cannot cause false positives/negatives.
- **Cost Efficiency:** EKS worker nodes auto-scale down when no builds are active, saving cloud compute costs.
- **Concurrency:** Parallel builds automatically spawn parallel pods without queue bottlenecks.

- **Infrastructure as Code (IaC):** Agent resource requirements (CPU, RAM, storage, sidecars) are declared directly in git repositories inside the Jenkinsfile.

4. Standard Jenkinsfile Template for Teams

To ensure new pipelines run reliably within this environment, copy and use this standardized Declarative Pipeline pattern:

Groovy

```
None
pipeline {
    agent {
        kubernetes {
            yaml '''
apiVersion: v1
kind: Pod
metadata:
  labels:
    component: build-agent
spec:
  nodeSelector:
    kubernetes.io/os: "linux"
  serviceAccountName: "jenkins-agents"
  containers:
  - name: jnlp
    image:
"351131773737.dkr.ecr.us-east-2.amazonaws.com/cloudbees/cloudbees-c
ore-agent:2.555.3.36983"
    resources:
      requests:
        cpu: "1000m"
        memory: "4Gi"
        ephemeral-storage: "10Gi"
      limits:
        cpu: "2000m"
        memory: "8Gi"
        ephemeral-storage: "20Gi"
    env:
    - name: K8S_NODE_NAME
```

```
      valueFrom:
        fieldRef:
          fieldPath: spec.nodeName
'''
        }
    }
    stages {
        stage('Environment Audit') {
            steps {
                script {
                    echo "Executing on K8s Node:
${env.K8S_NODE_NAME}"
                    echo "Working Directory:  ${env.WORKSPACE}"
                }
            }
        }
        stage('Build & Test') {
            steps {
                // Example compilation steps
                sh './gradlew clean build --no-daemon'
            }
        }
    }
}
```

5. Frequently Asked Questions (FAQ) & Troubleshooting

Q1: Why did my pipeline fail with AgentOfflineException or Eviction API: evicting?

- **Cause:** The underlying EKS worker node ran out of ephemeral disk space or RAM during heavy compilation/dependency downloads (yarn install, Gradle builds), prompting Kubernetes to evict the pod.
- **Fix:** Define explicit ephemeral-storage requests and limits (e.g., requests: ephemeral-storage: "10Gi") in your Jenkinsfile podTemplate.

Q2: Why did sh 'hostname' fail with exit code 127?

- **Cause:** The base CloudBees agent container is built on a minimal Linux distribution that omits core utilities like hostname.
- **Fix:** Use native environment variables like ${HOSTNAME} or read /etc/hostname directly instead of running external commands.

Q3: Can I run custom tools (e.g., kubectl, helm, aws-cli) in my pipeline?

- **Cause:** Public Docker images (like bitnami/kubectl:latest) are blocked by network policies/firewalls.
- **Fix:** Pull approved internal container images hosted in ECR (351131773737.dkr.ecr.us-east-2.amazonaws.com/...) or Artifactory (artifactory.inf.aws.vz-connect.net/...).

Regex pattern for multi-branch pipeline

```
None
^(?!.*obsolete)release/MMP3-PI-(4[0-9]|[5-9][0-9]|\d{3,})-.*
```

How this single pattern works:

- ^(?!.*obsolete) → **The Exclude Check:** Rejects any branch immediately if the word obsolete appears anywhere in the branch name (e.g., release/MMP3-PI-41-Sprint-24-obsolete).
- release/MMP3-PI- → Matches your standard branch prefix.
- (4[0-9]|[5-9][0-9]|\d{3,}) → **The Include Check:** Matches PI numbers **40–49**, **50–99**, or **100+** automatically.
- -.* → Matches the rest of the branch name.

Updated Regex to include “`release/MMP3-PI-39-40-Sprint-20-21-22-23`”

```
None
^(?!.*obsolete)release/MMP3-PI-(4[0-9]|[5-9][0-9]|\d{3,})-.*|^release/MMP3-PI-39-40-Sprint-20-21-22-23$
```

Failover Script

Directory Structure:

```
/home/ec2-user/edn-connection-check/
├── k8s_connectivity_failover.sh       <-- Main 5-min health check & scaling script
├── rotate_k8s_logs.sh                 <-- Nightly 15-day log cleanup script
├── k8s_applications_scaled_zero.flag <-- State flag (created ONLY when scaled to 0)
└── logs/                              <-- Dedicated logs directory
    ├── k8s_failover_2026-09-30.log   <-- Today's active log file (appended every 5 mins)
    ├── k8s_failover_2026-09-29.log   <-- Yesterday's historical log
    ├── k8s_failover_2026-09-28.log   <-- Historical log
    ├── ...                            <-- Auto-purged after 15 days
    └── rotation.log                   <-- Output log for rotate_k8s_logs.sh execution
```

1. mkdir -p edn-connection-check
2. cd edn-connection-check
3. touch `k8s_connectivity_failover.sh rotate_k8s_logs.sh`
4. mkdir -p logs

Grant execution permissions to both scripts

1) Run below commands on your EC2 instance to set execution permissions and assign folder ownership to `ec2-user`:

```
None
chmod +x /home/ec2-user/edn-connection-check/k8s_connectivity_failover.sh

chmod +x /home/ec2-user/edn-connection-check/rotate_k8s_logs.sh
```

2) Ensure ec2-user owns the entire directory structure and files

```
None
sudo chown -R ec2-user:ec2-user /home/ec2-user/edn-connection-check
```

Verify Permissions

Run this command to inspect file modes:

None

```
ls -la /home/ec2-user/edn-connection-check/*.sh
```

Expected output:

None

```
-rwxr-xr-x 1 ec2-user ec2-user ...
/home/ec2-user/edn-connection-check/k8s_connectivity_failover.sh
-rwxr-xr-x 1 ec2-user ec2-user ...
/home/ec2-user/edn-connection-check/rotate_k8s_logs.sh
```

K8s_conntectivty_failover.sh:

Shell

```
#!/usr/bin/env bash
# ==============================================================================
# Script Name : k8s_connectivity_failover.sh
# Location    : /home/ec2-user/edn-connection-check/k8s_connectivity_failover.sh
# Purpose     : Monitors downstream FQDN endpoints via proxy-service.
#               Scales down k8s deployments if ALL fail; restores if ANY passes.
# Schedule    : Crontab (every 5 minutes)
# ==============================================================================

# --- ENFORCE RUNNING AS ec2-user ---
TARGET_USER="ec2-user"
if [ "$(whoami)" != "$TARGET_USER" ]; then
    echo "Switching execution context to $TARGET_USER..."
    exec sudo -u "$TARGET_USER" PATH="$PATH"
KUBECONFIG="${KUBECONFIG:-/home/ec2-user/.kube/config}" "$0" "$@"
fi

set -euo pipefail

# --- ENVIRONMENT ISOLATION FOR CRON ---
export PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
export KUBECONFIG="${KUBECONFIG:-/home/ec2-user/.kube/config}"

# --- CONFIGURATION & DIRECTORY SETUP ---
NAMESPACE="applications"
```

```
BASE_DIR="/home/ec2-user/edn-connection-check"
LOG_DIR="${BASE_DIR}/logs"
STATE_FILE="${BASE_DIR}/k8s_applications_scaled_zero.flag"
CHECK_TIMEOUT=7 # Maximum allowed execution seconds per connection check

# Dynamic Daily Log File (e.g., logs/k8s_failover_2026-09-30.log)
TODAY=$(date '+%Y-%m-%d')
LOG_FILE="${LOG_DIR}/k8s_failover_${TODAY}.log"

# Ensure log directory exists
mkdir -p "$LOG_DIR"

log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | tee -a "$LOG_FILE"
}

# --- TARGET ENDPOINTS ---
# Format: "HOST:PORT:USE_PROXY"
ENDPOINTS=(
    # Direct CUDB Checks First (Fastest path to PASS)
    "blrck6030071-o-cudb-01-fe.vnoe.vzwnet.com:636:false"
    "blrck6030071-o-cudb-02-fe.vnoe.vzwnet.com:636:false"
    "holtl6717827-o-cudb-01-fe.vnoe.vzwnet.com:636:false"
    "holtl6717827-o-cudb-02-fe.vnoe.vzwnet.com:636:false"
    "vavit6247086-o-cudb-01-fe.vnoe.vzwnet.com:636:false"
    "vavit6247086-o-cudb-02-fe.vnoe.vzwnet.com:636:false"

    # EDA Checks (Proxy Required)
    "blrck6030071-o-eda-01.vnoe.vzwnet.com:8181:true"
    "blrck6030071-o-eda-01.vnoe.vzwnet.com:8383:true"
    "blrck6030071-o-eda-02.vnoe.vzwnet.com:8181:true"
    "blrck6030071-o-eda-02.vnoe.vzwnet.com:8383:true"

    "holtl6717827-o-eda-01.vnoe.vzwnet.com:8181:true"
    "holtl6717827-o-eda-01.vnoe.vzwnet.com:8383:true"
    "holtl6717827-o-eda-02.vnoe.vzwnet.com:8181:true"
    "holtl6717827-o-eda-02.vnoe.vzwnet.com:8383:true"

    "vavit6247086-o-eda-01.vnoe.vzwnet.com:8181:true"
    "vavit6247086-o-eda-01.vnoe.vzwnet.com:8383:true"
    "vavit6247086-o-eda-02.vnoe.vzwnet.com:8181:true"
    "vavit6247086-o-eda-02.vnoe.vzwnet.com:8383:true"
)

# --- EXECUTION LOOP ---
```

```
any_endpoint_healthy=false
PROXY_ADDR=""

log "Starting downstream health checks..."

for ep in "${ENDPOINTS[@]}"; do
    IFS=":" read -r host port use_proxy <<< "$ep"

    if [ "$use_proxy" = "true" ]; then
        # Lazy proxy resolution: resolves ONCE only if a proxy endpoint is
evaluated
        if [ -z "$PROXY_ADDR" ]; then
            PROXY_IP=$(kubectl get svc proxy-service -n proxy -o
jsonpath='{.spec.clusterIP}' 2>/dev/null || true)

            if [ -z "$PROXY_IP" ]; then
                log "WARNING: Could not fetch proxy-service ClusterIP. Falling
back to Pod IP lookup..."
                PROXY_IP=$(kubectl get pods -n proxy -l app=proxy
--field-selector=status.phase=Running -o jsonpath='{.items[0].status.podIP}'
2>/dev/null || true)
            fi

            if [ -z "$PROXY_IP" ]; then
                log "CRITICAL: Unable to resolve any Proxy IP (Service or Pod).
Exiting."
                exit 1
            fi

            PROXY_ADDR="${PROXY_IP}:3128"
            log "Active proxy endpoint resolved: $PROXY_ADDR (Running as:
$(whoami))"
        fi

        if timeout "$CHECK_TIMEOUT" nc -zv -w 5 "$host" "$port" --proxy
"$PROXY_ADDR" &>/dev/null; then
            log "PASS: $host:$port (via proxy $PROXY_ADDR)"
            any_endpoint_healthy=true
            break # Short-circuit on first success
        fi
    else
        if timeout "$CHECK_TIMEOUT" nc -zv -w 5 "$host" "$port" &>/dev/null; then
            log "PASS: $host:$port (direct)"
            any_endpoint_healthy=true
            break # Short-circuit on first success
```

```
        fi
    fi
    log "FAIL: $host:$port"
done

# --- SCALE EVALUATION ---
if [ "$any_endpoint_healthy" = false ]; then
    log "ALERT: ALL downstream FQDN checks failed!"

    if [ ! -f "$STATE_FILE" ]; then
        log "Scaling DOWN deployments in namespace '$NAMESPACE' to 0..."
        if kubectl scale deploy --all --replicas=0 -n "$NAMESPACE" >> "$LOG_FILE"
2>&1; then
            touch "$STATE_FILE"
            log "Scale down successful. State file created."
        else
            log "ERROR: kubectl scale down command failed."
        fi
    else
        log "Deployments already scaled to 0. No action required."
    fi
else
    log "HEALTHY: Minimum required connectivity verified."

    if [ -f "$STATE_FILE" ]; then
        log "RECOVERY DETECTED: Scaling UP deployments in namespace '$NAMESPACE'
to 1..."
        if kubectl scale deploy --all --replicas=1 -n "$NAMESPACE" >> "$LOG_FILE"
2>&1; then
            rm -f "$STATE_FILE"
            log "Scale up successful. State file cleared."
        else
            log "ERROR: kubectl scale up command failed."
        fi
    fi
fi

log "Execution finished."
```

rotate_k8s_logs.sh:

Shell

```
#!/usr/bin/env bash
# ================================================================================
# Script Name : rotate_k8s_logs.sh
# Location    : /home/ec2-user/edn-connection-check/rotate_k8s_logs.sh
# Purpose     : Purges log files older than 15 days from the logs directory.
# Schedule    : Crontab (Daily at Midnight: 0 0 * * *)
# ================================================================================

set -euo pipefail

# --- CONFIGURATION ---
BASE_DIR="/home/ec2-user/edn-connection-check"
LOG_DIR="${BASE_DIR}/logs"
RETENTION_DAYS=10

echo "[$(date '+%Y-%m-%d %H:%M:%S')] Starting log cleanup routine..."

if [ -d "$LOG_DIR" ]; then
    # Find and remove log files modified more than 15 days ago
    DELETED_COUNT=$(find "$LOG_DIR" -type f -name "k8s_failover_*.log" -mtime
+"$RETENTION_DAYS" -print -delete | wc -l)
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] Cleaned up $DELETED_COUNT log file(s)
older than $RETENTION_DAYS days."
else
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] Log directory $LOG_DIR does not exist.
Skipping."
fi

echo "[$(date '+%Y-%m-%d %H:%M:%S')] Cleanup completed."
```

Here is the complete script with the email helper updated to use **AWS SNS CLI (aws sns publish)** instead of mailx.

It uses the AWS CLI to send JSON-formatted notification payloads to an SNS topic on failover and recovery transitions, with a fallback log entry if the AWS CLI call fails.

Bash

```
None
#!/usr/bin/env bash
#
================================================================
===========
# Script Name : k8s_connectivity_failover.sh
# Location    :
/home/ec2-user/edn-connection-check/k8s_connectivity_failover.sh
# Purpose     : Monitors downstream FQDN endpoints via
proxy-service.
#               Scales down k8s deployments if ALL fail; restores
if ANY passes.
# Schedule    : Crontab (every 5 minutes)
#
================================================================
===========

# --- ENFORCE RUNNING AS ec2-user ---
TARGET_USER="ec2-user"
if [ "$(whoami)" != "$TARGET_USER" ]; then
    echo "Switching execution context to $TARGET_USER..."
    exec sudo -u "$TARGET_USER" PATH="$PATH"
KUBECONFIG="${KUBECONFIG:-/home/ec2-user/.kube/config}" "$0" "$@"
fi

set -euo pipefail
```

```
# --- ENVIRONMENT ISOLATION FOR CRON ---
export
PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"
export KUBECONFIG="${KUBECONFIG:-/home/ec2-user/.kube/config}"

# --- CONFIGURATION & DIRECTORY SETUP ---
NAMESPACE="applications-1"
BASE_DIR="/home/ec2-user/edn-connection-check"
LOG_DIR="${BASE_DIR}/logs"
STATE_FILE="${BASE_DIR}/k8s_applications_scaled_zero.flag"
CHECK_TIMEOUT=7 # Maximum allowed execution seconds per connection
check

# AWS SNS Alert Configuration (Update with your SNS Topic ARN &
Region)
SNS_TOPIC_ARN="arn:aws:sns:us-east-1:123456789012:k8s-failover-aler
ts"
AWS_REGION="us-east-1"

# Dynamic Daily Log File (e.g., logs/k8s_failover_2026-09-30.log)
TODAY=$(date '+%Y-%m-%d')
LOG_FILE="${LOG_DIR}/k8s_failover_${TODAY}.log"

# Ensure log directory exists
mkdir -p "$LOG_DIR"

log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | tee -a "$LOG_FILE"
}

# --- AWS SNS NOTIFICATION HELPER ---
send_sns_alert() {
    local subject="$1"
    local message="$2"
```

```
    if command -v aws &>/dev/null; then
        if aws sns publish \
            --topic-arn "$SNS_TOPIC_ARN" \
            --subject "$subject" \
            --message "$message" \
            --region "$AWS_REGION" >> "$LOG_FILE" 2>&1; then
            log "NOTIFICATION: Sent AWS SNS alert successfully."
        else
            log "WARNING: Failed to publish message to AWS SNS
Topic."
        fi
    else
        log "WARNING: AWS CLI ('aws') is not installed or not in
PATH. SNS alert skipped."
    fi
}

# --- TARGET ENDPOINTS ---
# Format: "HOST:PORT:USE_PROXY"
ENDPOINTS=(
    # Direct CUDB Checks First (Fastest path to PASS)
    "blrck6030071-o-cudb-01-fe.vnoe.vzwnet.com:636:false"
    "blrck6030071-o-cudb-02-fe.vnoe.vzwnet.com:636:false"
    "holtl6717827-o-cudb-01-fe.vnoe.vzwnet.com:636:false"
    "holtl6717827-o-cudb-02-fe.vnoe.vzwnet.com:636:false"
    "vavit6247086-o-cudb-01-fe.vnoe.vzwnet.com:636:false"
    "vavit6247086-o-cudb-02-fe.vnoe.vzwnet.com:636:false"

    # EDA Checks (Proxy Required)
    "blrck6030071-o-eda-01.vnoe.vzwnet.com:8181:true"
    "blrck6030071-o-eda-01.vnoe.vzwnet.com:8383:true"
    "blrck6030071-o-eda-02.vnoe.vzwnet.com:8181:true"
    "blrck6030071-o-eda-02.vnoe.vzwnet.com:8383:true"

    "holtl6717827-o-eda-01.vnoe.vzwnet.com:8181:true"
    "holtl6717827-o-eda-01.vnoe.vzwnet.com:8383:true"
```

```
    "holtl6717827-o-eda-02.vnoe.vzwnet.com:8181:true"
    "holtl6717827-o-eda-02.vnoe.vzwnet.com:8383:true"

    "vavit6247086-o-eda-01.vnoe.vzwnet.com:8181:true"
    "vavit6247086-o-eda-01.vnoe.vzwnet.com:8383:true"
    "vavit6247086-o-eda-02.vnoe.vzwnet.com:8181:true"
    "vavit6247086-o-eda-02.vnoe.vzwnet.com:8383:true"
)

# --- EXECUTION LOOP ---
any_endpoint_healthy=false
PROXY_ADDR=""

log "Starting downstream health checks..."

for ep in "${ENDPOINTS[@]}"; do
    IFS=":" read -r host port use_proxy <<< "$ep"

    if [ "$use_proxy" = "true" ]; then
        # Lazy proxy resolution: resolves ONCE only if a proxy
endpoint is evaluated
        if [ -z "$PROXY_ADDR" ]; then
            PROXY_IP=$(kubectl get svc proxy-service -n proxy -o
jsonpath='{.spec.clusterIP}' 2>/dev/null || true)

            if [ -z "$PROXY_IP" ]; then
                log "WARNING: Could not fetch proxy-service
ClusterIP. Falling back to Pod IP lookup..."
                PROXY_IP=$(kubectl get pods -n proxy -l app=proxy
--field-selector=status.phase=Running -o
jsonpath='{.items[0].status.podIP}' 2>/dev/null || true)
            fi

            if [ -z "$PROXY_IP" ]; then
                log "CRITICAL: Unable to resolve any Proxy IP
(Service or Pod). Exiting."
```

```
                exit 1
            fi

            PROXY_ADDR="${PROXY_IP}:3128"
            log "Active proxy endpoint resolved: $PROXY_ADDR
(Running as: $(whoami))"
        fi

        if timeout "$CHECK_TIMEOUT" nc -zv -w 5 "$host" "$port"
--proxy "$PROXY_ADDR" &>/dev/null; then
            log "PASS: $host:$port (via proxy $PROXY_ADDR)"
            any_endpoint_healthy=true
            break # Short-circuit on first success
        fi
    else
        if timeout "$CHECK_TIMEOUT" nc -zv -w 5 "$host" "$port"
&>/dev/null; then
            log "PASS: $host:$port (direct)"
            any_endpoint_healthy=true
            break # Short-circuit on first success
        fi
    fi
    log "FAIL: $host:$port"
done

# --- SCALE EVALUATION & SNS NOTIFICATIONS ---
if [ "$any_endpoint_healthy" = false ]; then
    log "ALERT: ALL downstream FQDN checks failed!"

    if [ ! -f "$STATE_FILE" ]; then
        log "Scaling DOWN deployments in namespace '$NAMESPACE' to
0..."
        if kubectl scale deploy --all --replicas=0 -n "$NAMESPACE"
>> "$LOG_FILE" 2>&1; then
            touch "$STATE_FILE"
            log "Scale down successful. State file created."
```

```
            # --- SNS ALERT: OUTAGE & FAILOVER TRIGGERED ---
            send_sns_alert \
                "CRITICAL: Kubernetes Failover Triggered
[Namespace: $NAMESPACE]" \
                "All 18 downstream FQDN checks (EDA/CUDB) failed at
$(date).\n\nDeployments in namespace '$NAMESPACE' have been scaled
to 0 to initiate Route 53 DNS failover.\n\nLog File: $LOG_FILE"
        else
            log "ERROR: kubectl scale down command failed."
        fi
    else
        log "Deployments already scaled to 0. No action required."
    fi
else
    log "HEALTHY: Minimum required connectivity verified."

    if [ -f "$STATE_FILE" ]; then
        log "RECOVERY DETECTED: Scaling UP deployments in namespace
'$NAMESPACE' to 1..."
        if kubectl scale deploy --all --replicas=1 -n "$NAMESPACE"
>> "$LOG_FILE" 2>&1; then
            rm -f "$STATE_FILE"
            log "Scale up successful. State file cleared."

            # --- SNS ALERT: RECOVERY DETECTED ---
            send_sns_alert \
                "RECOVERED: Kubernetes Downstream Connectivity
Restored [Namespace: $NAMESPACE]" \
                "Downstream connectivity checks succeeded at
$(date).\n\nDeployments in namespace '$NAMESPACE' have been scaled
back up to 1.\n\nLog File: $LOG_FILE"
        else
            log "ERROR: kubectl scale up command failed."
        fi
    fi
```

```
fi

log "Execution finished."
```

Setup Instructions

1. **Update Topic Variables**: Replace SNS_TOPIC_ARN and AWS_REGION with your actual SNS Topic ARN and AWS Region on lines 28–29:
2. Bash

```
None
SNS_TOPIC_ARN="arn:aws:sns:us-east-1:123456789012:k8s-failover-aler
ts"
AWS_REGION="us-east-1"
```

3.
4. **IAM Permissions**: Ensure the IAM role attached to your EC2 bastion instance includes the sns:Publish permission for your target topic:
5. JSON

```
None
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": "sns:Publish",
            "Resource":
"arn:aws:sns:us-east-1:123456789012:k8s-failover-alerts"
        }
    ]
}
```

6.
7. **Test SNS Publishing**: Run a manual test command from your terminal to verify message delivery before deployment:
8. Bash

None

```
aws sns publish \
    --topic-arn
"arn:aws:sns:us-east-1:123456789012:k8s-failover-alerts" \
    --subject "Test Failover Alert" \
    --message "Testing SNS notification integration from EC2
bastion." \
    --region us-east-1
```

Cut Release Branch

1. Open **git bash**
2. Create shell script file (Ex: createBranch.sh) and copy paste below code
3. Update source and target branch names
4. Update token
5. Provide execute perm → chmod +x createBranch.sh
6. Execute the script → ./createBranch.sh

Shell

```
#!/bin/bash

# ================= Configuration =================
BITBUCKET_URL="https://ctlabs.verizon.net/stash"
PROJECT_KEY="POSND"
SOURCE_BRANCH="release/MMP3-PI-41-Sprint-24-25"
TARGET_BRANCH="release/MMP3-PI-41-Sprint-24-25-obsolete"

# Note: Rotate this token immediately if it was posted to public/shared channels
BEARER_TOKEN="BBDC-MjQzMjM3NjE4Njc5OrI9ZCfYpcBBQNptFS9pE2453xtt"

REPOS=(
  "oslte-spog-admin"
  "oslte-spog-api"
  "oslte-spog-api-first"
  "oslte-spog-cloud-connect"
  "oslte-spog-commons"
  "oslte-spog-device"
  "oslte-spog-ibc-kpi-processor"
  "oslte-spog-inventory"
  "oslte-spog-kpi-processor"
  "oslte-spog-ndac-kpi-processor"
  "oslte-spog-portal"
  "enm-kpi-script"
  "enm-script"
  "enm_cbrs"
  "oslte-spog-processor"
  "oslte-spog-scheduler"
```

```
  "oslte-spog-s360-ticket"
  "oslte-spog-inventory-post"
  "oslte-spog-wsevents"
  "oslte-spog-vmb-processor"
  "oslte-spog-scim"
  "oslte-spog-mongo-mcp"
  "oslte-spog-aiops"
  "oslte-spog-osnd-mcp"
  "oslte-spog-models"
  # "enm-pm-file-fetcher"
  # "oslte-spog-pm-file-manager"
  # "kafka-service"
  # "oslte-spog-link-monitor"
  # "oslte-spog-kpi-query"
  # "oslte-spog-kpi-rest"
)
# ================================================

echo "Starting branch creation for Project: ${PROJECT_KEY}..."

for REPO in "${REPOS[@]}"; do
  echo -n "Processing [${REPO}]... "


URL="${BITBUCKET_URL}/rest/api/1.0/projects/${PROJECT_KEY}/repos/${REPO}/branches"
  PAYLOAD=$(cat <<EOF
{
  "name": "${TARGET_BRANCH}",
  "startPoint": "${SOURCE_BRANCH}"
}
EOF
)

  # Using -k for corporate SSL, -S to show errors
  HTTP_STATUS=$(curl -k -s -S -o /tmp/bb_response.json -w "%{http_code}" -X POST
"$URL" \
    -H "Authorization: Bearer ${BEARER_TOKEN}" \
    -H "Content-Type: application/json" \
```

```
    -d "$PAYLOAD" 2>/tmp/curl_err.log)
  CURL_EXIT=$?

  if [ "$CURL_EXIT" -ne 0 ] || [ "$HTTP_STATUS" = "000" ]; then
    echo "FAILED (Network / Connection error, exit code: ${CURL_EXIT})"
    if [ -s /tmp/curl_err.log ]; then
      cat /tmp/curl_err.log
    fi
    echo ""
    continue
  fi

  if [ "$HTTP_STATUS" -eq 200 ] || [ "$HTTP_STATUS" -eq 201 ]; then
    echo "SUCCESS (Created)"
  elif [ "$HTTP_STATUS" -eq 400 ] || [ "$HTTP_STATUS" -eq 409 ]; then
    echo "WARNING (Branch may already exist or invalid source - HTTP
${HTTP_STATUS})"
  else
    echo "FAILED (HTTP ${HTTP_STATUS})"
    [ -f /tmp/bb_response.json ] && cat /tmp/bb_response.json
    echo ""
  fi
done

echo "------------------------------------------------"
echo "Completed!"
```

Delete branch script

1. Open **git bash**
2. Create shell script file (Ex: deleteBranch.sh) and copy paste below code
3. Update target branch name
4. Update token
5. Provide execute perm → chmod +x deleteBranch.sh
6. Execute the script → ./deleteBranch.sh

Shell

```
#!/bin/bash

BITBUCKET_URL="https://ctlabs.verizon.net/stash"
PROJECT_KEY="POSND"
TARGET_BRANCH="release/MMP3-PI-41-Sprint-24"
BEARER_TOKEN="BBDC-MjQzMjM3NjE4Njc5OrI9ZCfYpcBBQNptFS9pE2453xtt"

REPOS=(
  "oslte-spog-admin"
  "oslte-spog-api"
  "oslte-spog-api-first"
  "oslte-spog-cloud-connect"
  "oslte-spog-commons"
  "oslte-spog-device"
  "oslte-spog-ibc-kpi-processor"
  "oslte-spog-inventory"
  "oslte-spog-kpi-processor"
  "oslte-spog-link-monitor"
  "oslte-spog-ndac-kpi-processor"
  "oslte-spog-portal"
  "enm-kpi-script"
  "enm-script"
  "enm_cbrs"
  "oslte-spog-processor"
  "oslte-spog-scheduler"
  "oslte-spog-s360-ticket"
  "oslte-spog-inventory-post"
  "oslte-spog-wsevents"
  "oslte-spog-vmb-processor"
  "oslte-spog-scim"
  "oslte-spog-mongo-mcp"
  "oslte-spog-aiops"
  "oslte-spog-kpi-query"
  "oslte-spog-kpi-rest"
```

```
  "oslte-spog-osnd-mcp"
)

echo "Starting branch deletion for '${TARGET_BRANCH}'..."

for REPO in "${REPOS[@]}"; do
  echo -n "Deleting in [${REPO}]... "

  # Notice the endpoint uses 'branch-utils/1.0' instead of 'api/1.0'

URL="${BITBUCKET_URL}/rest/branch-utils/1.0/projects/${PROJECT_KEY}/repos/${REPO}/
branches"

  HTTP_STATUS=$(curl -s -o /dev/null -w "%{http_code}" -X DELETE "$URL" \
    -H "Authorization: Bearer ${BEARER_TOKEN}" \
    -H "Content-Type: application/json" \
    -d '{
      "name": "refs/heads/'"${TARGET_BRANCH}"'"
    }')

  if [ "$HTTP_STATUS" -eq 204 ] || [ "$HTTP_STATUS" -eq 200 ]; then
    echo "DELETED SUCCESSFULLY"
  elif [ "$HTTP_STATUS" -eq 404 ]; then
    echo "NOT FOUND"
  else
    echo "FAILED (HTTP ${HTTP_STATUS})"
  fi
done

echo "Done!"
```

AWS EDN failover Lambda function

High Availability (HA) Drill & Application Scaling Operational Runbook

This runbook documents the commands and step-by-step procedures required to safely scale down workloads, suspend Flux CD reconciliation, perform High Availability (HA) validation drills, and restore applications in target environments.

IMP Note:

1. **Option-1** works only when there are no replicas count directly added in the deployment yaml file and all deployments are scaled through HPA.
2. **Option-2** works in for both scenarios:
    a. Replicas count added directly in deployment yaml file
    b. Replicas scaled through HPA

Option-1: Standalone Application Scaling Commands (Recommended)

Use these bulk operations to scale all deployments within the applications namespace up or down.

Scale Down All Applications to 0 Replicas

> **Warning:** Executing this command terminates all running pods across the applications namespace.

Bash

```
None
kubectl scale deployment --all --replicas=0 -n applications

or

kubectl get deployments -n applications -o
custom-columns=NAME:.metadata.name --no-headers | xargs -I {}
kubectl scale deployment {} --replicas=0 -n applications
```

Scale Up All Applications to 1 Replica

Use this command to restore pod instances after completing maintenance or testing:

Bash

```
None
kubectl get deployments -n applications -o
custom-columns=NAME:.metadata.name --no-headers | xargs -I {}
kubectl scale deployment {} --replicas=1 -n applications
```

Option-2: Suspend Kustomization and Scaling commands

==========
PHASE 1: BRING DOWN REGION
==========

1. Check what Flux resources are managing the "applications" namespace:
   Identify Kustomizations. Check for Kustomizations across all namespaces.

```
None
flux get kustomizations -A
```

2. Suspend Flux Kustomization. Pause Flux so it stops overriding manual changes.
   ex: flux suspend kustomization prod-east-app -n flux-system

```
None
flux suspend kustomization <kustomization-name> -n <namespace>
```

3. Verify Suspension. Verify SUSPENDED = True
   ex: flux get kustomizations -n flux-system

```
None
flux get kustomizations -n <namespace>
```

4. Scale all deployments down to 0

```
None
kubectl scale deployment --all --replicas=0 -n applications
```

or

```
None
kubectl get deployments -n applications -o custom-columns=NAME:.metadata.name
--no-headers | xargs -I {} kubectl scale deployment {} --replicas=0 -n
applications
```

5. Verify all pods are deleted

```
None
kubectl get pods -n applications
```

==========
PHASE 2: QA TESTING WINDOW
==========
Hand off environment to QA for validation/failover testing.

==========
PHASE 3: BRING UP REGION (Post-Testing)
==========

Option A: Resume Flux (Recommended - Auto-Heals to Git State)

Since Flux stores the original replicas: 1 state from Git, simply resuming Flux forces it to auto-heal and scale all deployments back up automatically:

A.1: Resume Kustomization

```
None
flux resume kustomization <kustomization-name> -n <namespace>
```

A.2: Force an immediate sync from Git to speed it up

```
None
flux reconcile kustomization <kustomization-name> -n <namespace> --with-source
```

Option B: Manual Scale-Up First, Then Resume Flux

If you want to manually scale workloads up step-by-step before re-enabling Flux:

B.1: Manually scale specific deployments back to 1

```
None
kubectl scale deployment --all --replicas=1 -n applications
```

B.2: Verify pods are Running

```
None
kubectl get pods -n applications
```

B.3: Resume Flux to restore automated management

```
None
flux resume kustomization <kustomization-name> -n flux-system
```

==========

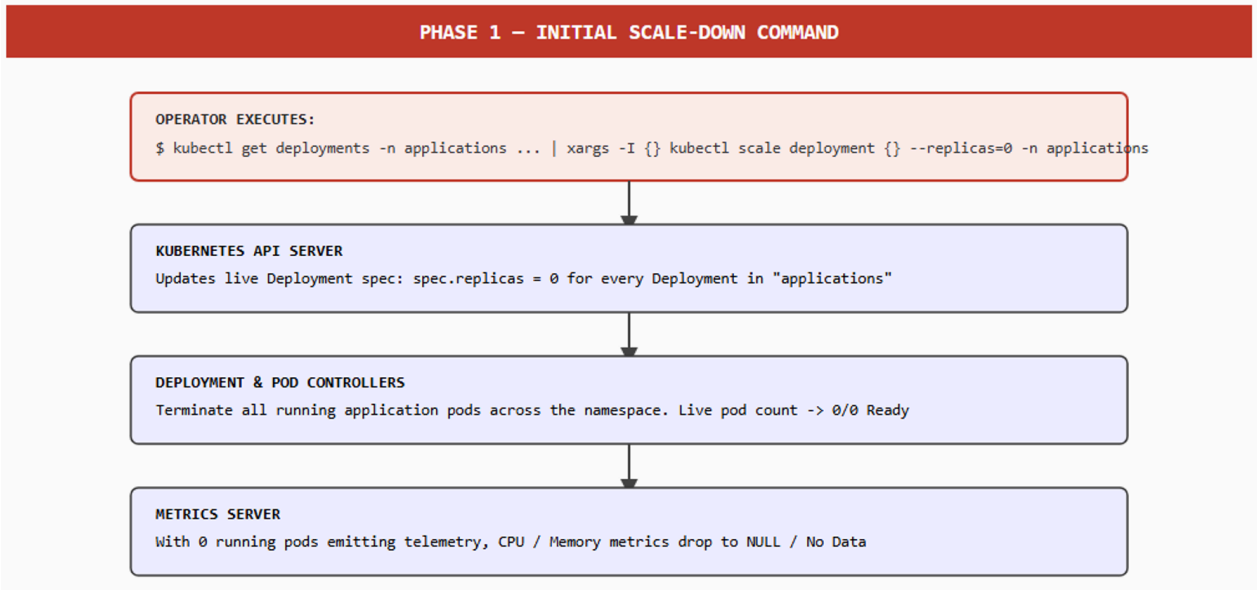
PHASE 4: Final Verification

==========

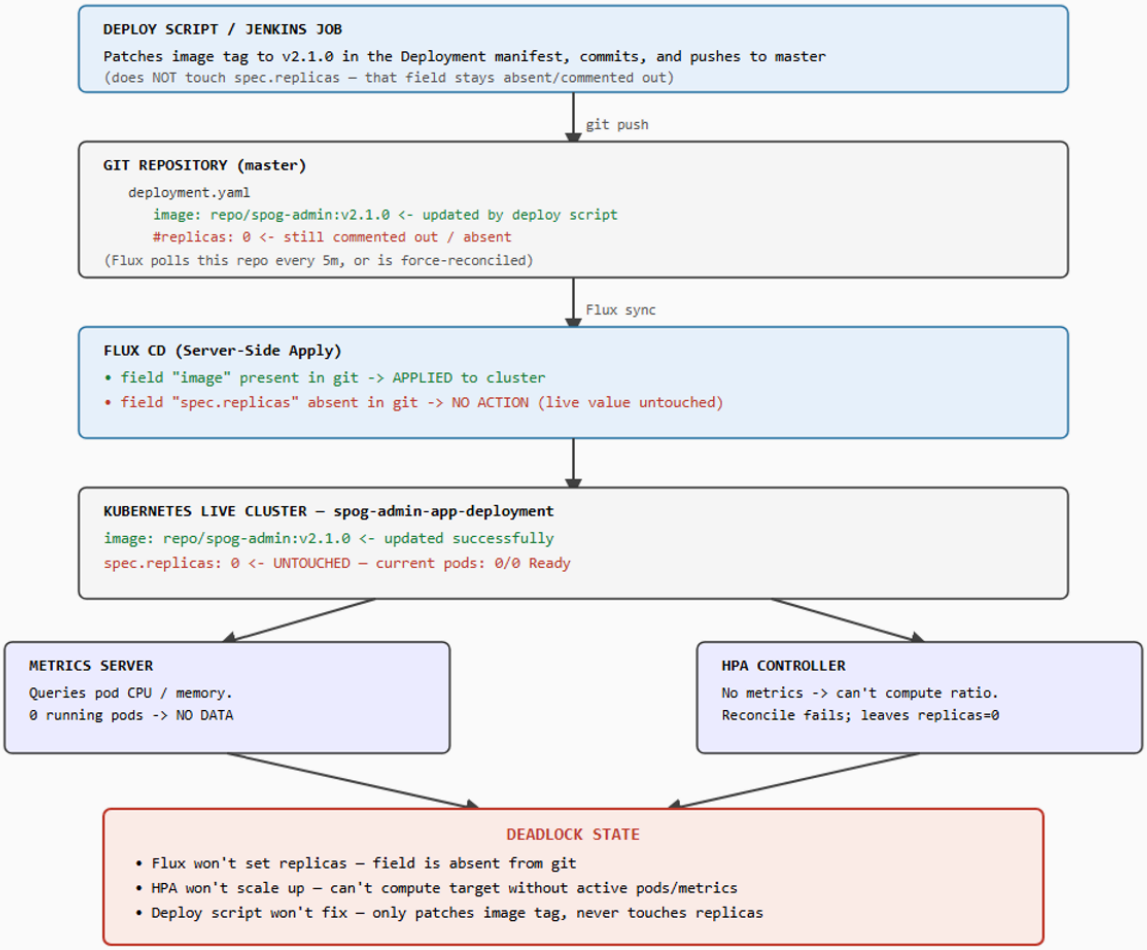
```
None
kubectl get pods -n applications
```

FYI:
Increase the screen size to have a better look at the below images. It might seem blurry with regular screen size.

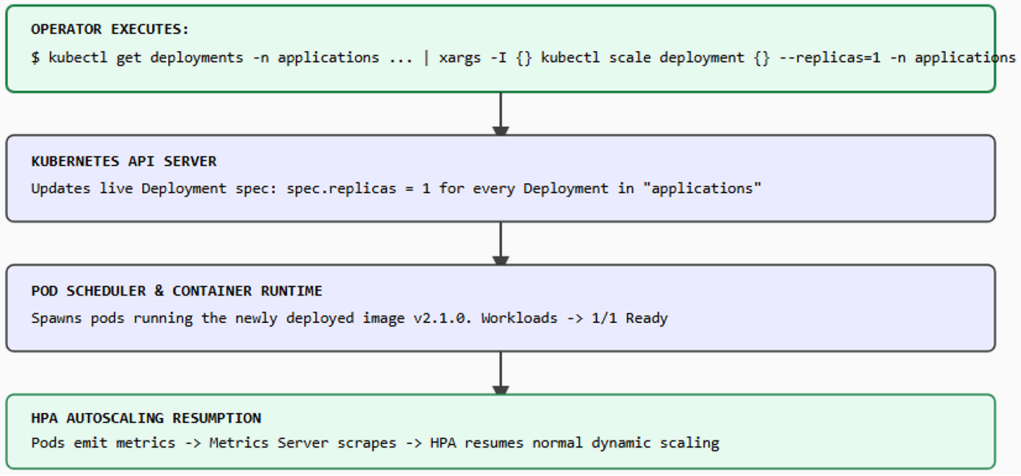
It’s important to understand how dead lock happens when you execute above commands.



PHASE 2 — GITOPS RECONCILIATION & DEPLOYMENT SCRIPT



PHASE 3 — MANUAL SCALE-UP RESOLUTION



High Availability (HA) Drill Standard Operating Procedure (SOP) (Optional)

Follow this sequence to ensure Flux CD does not automatically revert replica states during the HA drill.

1.Inspect & Suspend Flux Kustomization: Pre-requisite: Check live Flux state.
Inspect the active Flux configuration, then suspend reconciliation for the targeted environment (e.g., preprod-east-app) to prevent GitOps auto-healing during the drill:

Bash

```
Shell
# Inspect live manifest
kubectl get kustomization preprod-east-app -n flux-system -o yaml

# Suspend Flux reconciliation
flux suspend kustomization preprod-east-app -n flux-system

# Verify suspension status
kubectl get kustomizations.kustomize.toolkit.fluxcd.io -n
flux-system
```

2.Remove HPAs and Scale Down Applications: Prevent HPA auto-scaling conflicts.
Delete Horizontal Pod Autoscalers (HPAs) to prevent them from attempting to scale up zero-replica deployments, then scale all application workloads to 0:

Bash

```
Shell
# Remove all HPAs in the target namespace
kubectl delete hpa --all -n applications

# Scale down all deployments
kubectl get deployments -n applications -o
custom-columns=NAME:.metadata.name --no-headers | xargs -I {}
kubectl scale deployment {} --replicas=0 -n applications
```

3.Execute High Availability Drill:
Perform the required HA validation checks, regional failovers, or disaster recovery testing while application pods remain offline.

4.Resume Flux & Restore Workloads: Post-drill restoration.
Re-enable Flux CD reconciliation and restore deployment replicas back to active state:

Bash

```shell
# Resume Flux reconciliation
flux resume kustomization preprod-east-app -n flux-system

# Scale deployments back to 1 replica
kubectl get deployments -n applications -o
custom-columns=NAME:.metadata.name --no-headers | xargs -I {}
kubectl scale deployment {} --replicas=1 -n applications
```

Windows-Specific Recommendation

To configure Git so that **all repositories under the posnd project** automatically use your Personal Access Token without saving it in individual repo URLs, you can set a global URL rewrite rule in Git.

Step 1: Clean Your Repository URLs

First, ensure you are cloning repositories using standard HTTPS URLs **without** embedded tokens:

```
None
https://ctlabs.verizon.net/stash/scm/posnd/<repo-name>.git
```

If you already cloned `repo-name` with the token embedded, clean its local remote URL:

```
None
cd repo-name
git remote set-url origin https://ctlabs.verizon.net/stash/scm/posnd/repo-name.git
```

- **Verification:** Run `git remote -v`. The output should show the URL without `<user-name>:<Personal-Access-Token>@`.

Step 2: Set global URL rewrite for the posnd path

Run this command in Terminal to automatically inject your token into every request going to the `posnd` project path:

Bash

```
None
git config --global
url."https://<user-name>:<Personal-Access-Token>@ctlabs.verizon.net
/stash/scm/posnd/".insteadOf
"https://ctlabs.verizon.net/stash/scm/posnd/"
```

(Replace `<user-name>` and `<Personal-Access-Token>` with your actual credentials).

- **Verification:** Run `git config --global --get-regexp url` to confirm the rule appears in your global config.

How It Works

- Any time you run `git clone`, `git fetch`, or `git push` for **any repository** under `.../scm/posnd/`, Git automatically attaches your authentication token behind the scenes.
- Repositories outside of `posnd` will not use this rule.

macOS-Specific Recommendation

Instead of hardcoding the token into the global Git config, the recommended approach on macOS is to combine the **posnd** URL structure with **macOS Keychain**:

1. **Keep standard repo URLs** (no tokens embedded in global config):

```
None
git config --global credential.helper osxkeychain
```

2. **First time you interact with any repo in `posnd`**, Git will prompt you for:
    - **Username:** Your Bitbucket/Stash username
    - **Password:** Your Personal Access Token (PAT)
3. macOS Keychain securely stores this PAT once for `ctlabs.verizon.net`. Subsequent clones, pulls, and pushes across **all** repos under that host/domain will automatically reuse the token from Keychain.

Clone repo with specific branches only

1. Clone source repo and navigate into directory

```
None
git clone https://ctlabs.verizon.net/visp-stash/scm/open-qa/pdi_qa.git pdi_qa
```

or

clone repo by directly passing the credentials:

```
None
git -c credential.helper= clone
https://<user-name>:<Personal-Access-Token>@ctlabs.verizon.net/visp-stash/scm/open
qa/pdi_qa.git pdi_qa
```

```
None
cd pdi_qa
```

2. Add target remote using the correct SCM URL from your screenshot

```
None
git remote add target https://ctlabs.verizon.net/stash/scm/posnd/pdi_qa.git
```

3. Fetch all remote branches from source

```
None
git fetch origin
```

4. Checkout and track each branch locally which you want to migrate

```
None
git checkout develop
```

```
git checkout Re1_Develop_Branch_2026
git checkout Re1_Develop_Branch_2026_MongoDB
git checkout Q3_2026_Continuous_Monitoring_new
git checkout master
```

5. Push all checked-out branches to the target repository

```
None
git push target develop
git push target Re1_Develop_Branch_2026
git push target Re1_Develop_Branch_2026_MongoDB
git push target Q3_2026_Continuous_Monitoring_new
git push target master
```

Faster Alternative (1-Liner Push)
If you don't want to manually checkout all 5 branches one by one, you can push them directly after setting the remote:

```
Shell
git clone https://ctlabs.verizon.net/visp-stash/scm/open-qa/pdi_qa.git pdi_qa

cd pdi_qa

git remote add target https://ctlabs.verizon.net/stash/scm/posnd/pdi_qa.git
git fetch origin

git push target \
  origin/develop:refs/heads/develop \
  origin/Re1_Develop_Branch_2026:refs/heads/Re1_Develop_Branch_2026 \
  origin/Re1_Develop_Branch_2026_MongoDB:refs/heads/Re1_Develop_Branch_2026_MongoDB \
  origin/Q3_2026_Continuous_Monitoring_new:refs/heads/Q3_2026_Continuous_Monitoring_new \
  origin/master:refs/heads/master
```

Syncing Subsequent Changes (Delta Migration)

This document details the procedure for syncing post-migration updates from the legacy repository to the target repository.

Remote Repository Mapping

Remote Name	Description	Repository URL
origin	Source (Old Repository)	https://ctlabs.verizon.net/visp-stash/scm/openqa/pdi_qa.git
target	Destination (New Repository)	https://ctlabs.verizon.net/stash/scm/posnd/pdi_qa.git

Q) Since I had executed below cmd previously, when I execute git fetch origin or git pull origin Re1_Develop_Branch_2026, won't it refer to the new repo rather than the old repo?

```
None
git remote add target https://ctlabs.verizon.net/stash/scm/posnd/pdi_qa.git
```

No, it will still point to the old repo.

In Git, remotes are given distinct names. Adding target did **not** overwrite or replace origin.

Here is how your remotes are currently configured:

- **origin** → https://ctlabs.verizon.net/visp-stash/scm/openqa/pdi_qa.git (**Old Repo**)
- **target** → https://ctlabs.verizon.net/stash/scm/posnd/pdi_qa.git (**New Repo**)

When you run git fetch origin or git pull origin Re1_Develop_Branch_2026, Git explicitly queries **origin** (the old repo).

To interact with the new repo, you explicitly specify **target** (e.g., git push target ... or git fetch target).

Verification

You can run this command anytime to inspect which URLs your remotes are assigned to:

Bash

```
None
git remote -v
```

Expected output:

Plaintext

```
None
origin  https://ctlabs.verizon.net/visp-stash/scm/openqa/pdi_qa.git (fetch)
origin  https://ctlabs.verizon.net/visp-stash/scm/openqa/pdi_qa.git (push)
target  https://ctlabs.verizon.net/stash/scm/posnd/pdi_qa.git (fetch)
target  https://ctlabs.verizon.net/stash/scm/posnd/pdi_qa.git (push)
```

Syncing Subsequent Changes

When new commits are added to a branch in the old repository (`origin`), use either of the following approaches to sync them to the new repository (`target`).

Method A: Direct Remote-to-Remote Push (Recommended)

This approach updates the target repository directly from `origin` references without requiring a local branch checkout or merge step.

```
None
# 1. Fetch the latest changes from the old repository
git fetch origin

# 2. Push the updated branch from origin directly to target
git push target origin/Re1_Develop_Branch_2026:refs/heads/Re1_Develop_Branch_2026
```

Method B: Local Tracking & Inspection

Use this method if you need to inspect, build, or test the new changes locally before pushing to the target.

```
None
# 1. Switch to the local branch and pull latest changes from origin
git checkout Re1_Develop_Branch_2026
git pull origin Re1_Develop_Branch_2026

# 2. Push the updated local branch to target
git push target Re1_Develop_Branch_2026
```

Prompt to migrate existing terraform repo:

System / Task:
You are an expert Principal DevOps and Terraform Infrastructure Engineer. I will provide you with the source code of an existing Terraform project. Your task is to refactor this entire codebase into a clean, production-grade, highly reusable Terraform module structure.

Strict Rules & Requirements:
1. Complete Resource Coverage (Zero Omissions):
  - You MUST account for EVERY SINGLE resource, data source, local, output, file dependence (e.g., user-data scripts, JSON policies, certificates, keys), and import block present across all uploaded `.tf` files.
  - Do NOT abbreviate, summarize, truncate, or leave `...` placeholders in any generated `.tf`, `.json`, `.txt`, or key/cert files.

2. Module Architecture Design:
  - Organize the repository into logical reusable modules placed under `modules/<module_name>/`.
  - Place environment-specific root configurations under `environments/<env_name>/` (e.g., `environments/dev/`).
  - Standardize every module directory to contain `main.tf`, `variables.tf`, and `outputs.tf`.

3. Security Groups Refactoring:
  - Convert hardcoded `ingress` and `egress` blocks into standard, dynamic HCL `dynamic "ingress"` and `dynamic "egress"` configuration blocks driven by variable inputs.

4. Import & Lifecycle Preservations:
  - Preserve all `lifecycle { create_before_destroy = true }` directives.
  - Maintain all explicit `depends_on` relationships.

5. Zero-Downtime State Migration Output:
  - Provide a complete, explicit list of `terraform state mv` commands matching every original resource address to its new module path (e.g., `terraform state mv aws_security_group.eks-cluster-sg module.eks_cluster_sg.aws_security_group.this`) to ensure refactoring does not destroy or recreate any live cloud resources.

6. Shell Auto-Generation Script:
  - Output a single, standalone Bash script (`build_infrastructure.sh`) using EOF EOF blocks that, when executed in an empty directory, creates every folder and populates every single file completely with the exact code without requiring manual copy-pasting.
  - Use PowerShell-compatible compression at the end (`powershell -Command "Compress-Archive -Path terraform-refactored -DestinationPath dev.zip -Force"`) to generate the output zip file on Windows without requiring a native `zip` CLI tool.

Configure saml2aws for short live AWS access keys

https://confluence.verizon.com/spaces/SOEEXTKB/pages/772770820/TPD+SOE+AWS+-+saml2aws-vz+-+Verizon+Integration

https://confluence.verizon.com/spaces/SOEEXTKB/pages/772777716/TPD+SOE+AWS+-+saml2aws-vz+Troubleshooting

```
None
Path of .credentials file:    C:\Users\srivsa\.aws
Path of saml exe file:        C:\Tools
Path of .saml2aws file:       C:\Users\srivsa
Path of .bashrc:              C:\Users\srivsa
Path of .bash_profile:        C:\Users\srivsa
Path of .sso_pass:            C:\Users\srivsa
```

Windows system configuration steps:

1. download saml2aws
2. Copy saml2aws-vz.exe to a folder in your PATH (e.g. C:\Tools\) and
3. add the saml2aws folder to your system's Path environment variable

```
Shell
[Environment]::SetEnvironmentVariable("Path", $env:Path +
";C:\tools\saml2aws-vz.win.2026.08.26-vz1", "User")
```

4. verify that env variable is set correctly

```
Shell
Get-Command saml2aws-vz -ErrorAction SilentlyContinue |
Select-Object Name, Source
```

sample o/p:

```
Shell

Name                           Source

----                                ------

saml2aws-vz.exe
C:\Tools\saml2aws-vz.win.2026.08.26-vz1\saml2aws-vz.exe
```

5. Create the Profile File

```
Shell
New-Item -Path $PROFILE -ItemType File -Force
```

6. Open the Profile in Notepad

```
Shell
notepad $PROFILE
```

7. Add the following to your PowerShell profile (copy & paste, save)

```
Shell
$env:HTTPS_PROXY = "http://server.proxy.vzwcorp.com:9290"
```

8. Run this once to configure the tool:

```
Shell
saml2aws-vz configure --idp-account osnd-dev   --session-duration=28800
--profile osnd-dev

saml2aws-vz configure --idp-account osnd-test  --session-duration=28800
--profile osnd-test

saml2aws-vz configure --idp-account osnd-preprod --session-duration=28800
--profile osnd-preprod

saml2aws-vz configure --idp-account osnd-prod  --session-duration=28800
--profile osnd-prod
```

Note: --session-duration=28800 extends your session time to 8 hours (our current maximum for any console role)

You will be prompted for three things:

```
Shell

? AWS Profile  saml      ← press Enter

? Username     _         ← type your Verizon username (e.g. jsmith)

? Meta Alias   _         ← press Enter unless your org uses a
different SSO integration (see below)
```

That's it. Setup is done.

Meta Alias — Selecting Your Org's SSO Integration (v2026.08.19-vz1+)

Different Verizon orgs have their own AWS SSO integration in ForgeRock, each identified by a distinct **Meta Alias**. The `configure` prompt defaults to the standard TPD SOE integration — **press Enter to accept it unless your platform team has told you to use a different one.**

Meta Alias	Used By
/employee/VZIDP_28054_JGBV_TPDSOEAWSSOE	PACT Applications
/employee/VZIDP_28054_JGBV_TPDSOEAWSCON	Default
/employee/VZIDP_28054_JGBV_TPDSOEAWSCDI	CDI Applications

Existing profiles created before this version are unaffected — they keep using the default integration automatically, no action needed. To change your org's integration later, re-run:

```
None

saml2aws-vz configure --profile saml
```

and enter the new value at the Meta **Alias** prompt. (It's stored in `~/.saml2aws` as `meta_alias = ...` under your profile's section if you ever need to edit it by hand.)

9. Open saml2aws file and update meta alias to pact applications

```
Shell
notepad $HOME\.saml2aws
```

Copy and paste “/employee/VZIDP_28054_JGBV_TPDSOEAWSSOE” for “meta_alias”.
Save and exit.

10. Login to individual profile. **Make sure you are connected to TAMPA gateway VPN** (India gateways don’t work)

```
Shell
saml2aws-vz login --profile osnd-dev
saml2aws-vz login --profile osnd-test
saml2aws-vz login --profile osnd-preprod
saml2aws-vz login --profile osnd-prod
```

OR

```
Shell
saml2aws-vz login --idp-account osnd-dev --profile osnd-dev
saml2aws-vz login --idp-account osnd-test --profile osnd-test
saml2aws-vz login --idp-account osnd-preprod --profile osnd-preprod
saml2aws-vz login --idp-account osnd-prod --profile osnd-prod
```

For Force login(rotate credentials forcefully):

```
Shell
saml2aws-vz login --profile osnd-dev --force
OR
saml2aws-vz login --idp-account osnd-dev --profile osnd-dev --force
```

11. verify your current AWS authentication state and identity for the `osnd-dev` profile while explicitly bypassing SSL certificate verification

```
Shell
aws sts get-caller-identity --no-verify-ssl --profile osnd-dev
```

12. List s3 buckets while explicitly bypassing SSL certificate verification

```
Shell
aws s3 ls --no-verify-ssl --profile osnd-dev
```

Automate Login process:

Instead of entering login profile cmd, system credentials, DUO passcode and selecting the required role for each and every aws profile, I have automated the process as below. Now I just have to run the **aws-login-all** cmd which invokes a method/script stored in .bashrc file. This script will read the system credentials from the .sso_pass file. I have to enter the DUO passcode manually. Saml2aws.exe will automatically store the generated access keys in AWS .credentials file.

```shell
aws-login-all
or
aws-login-all --force
```

Sample Execution:

```shell
SRIVSA@VIN0019ALKKW6E6 MINGW64 ~
$ aws-login-all --force
Force flag detected: Re-authenticating all environments...
Enter 6-digit DUO Passcode: <enter DUO passcode>

----------------------------------------
Authenticating: osnd-dev
----------------------------------------
Using IdP Account osnd-dev to access VerizonSSO https://ssologin.verizon.com
Authenticating as srivsa ...
Selected role: arn:aws:iam::520949192567:role/vz-console-soe-administrator
Requesting AWS credentials using SAML assertion.
Logged in as:
arn:aws:sts::520949192567:assumed-role/vz-console-soe-administrator/sachin.s@one.v
erizon.com

Your new access key pair has been stored in the AWS configuration.
Note that it will expire at 2026-10-17 05:39:08 +0530 IST
To use this credential, call the AWS CLI with the --profile option (e.g. aws
--profile osnd-dev ec2 describe-instances).
----------------------------------------
Authenticating: osnd-test
----------------------------------------
Using IdP Account osnd-test to access VerizonSSO https://ssologin.verizon.com
Selected role: arn:aws:iam::157520333689:role/vz-console-soe-administrator
```

```
Requesting AWS credentials using SAML assertion.
Logged in as:
arn:aws:sts::157520333689:assumed-role/vz-console-soe-administrator/sachin.s@one.v
erizon.com

Your new access key pair has been stored in the AWS configuration.
Note that it will expire at 2026-10-17 05:39:11 +0530 IST
To use this credential, call the AWS CLI with the --profile option (e.g. aws
--profile osnd-test ec2 describe-instances).
----------------------------------------
Authenticating: osnd-preprod
----------------------------------------
Using IdP Account osnd-preprod to access VerizonSSO https://ssologin.verizon.com
Selected role: arn:aws:iam::837767801504:role/vz-console-soe-administrator
Requesting AWS credentials using SAML assertion.
Logged in as:
arn:aws:sts::837767801504:assumed-role/vz-console-soe-administrator/sachin.s@one.v
erizon.com

Your new access key pair has been stored in the AWS configuration.
Note that it will expire at 2026-10-17 05:39:14 +0530 IST
To use this credential, call the AWS CLI with the --profile option (e.g. aws
--profile osnd-preprod ec2 describe-instances).
----------------------------------------
Authenticating: osnd-prod
----------------------------------------
Using IdP Account osnd-prod to access VerizonSSO https://ssologin.verizon.com
Selected role: arn:aws:iam::282329364436:role/vz-console-soe-administrator
Requesting AWS credentials using SAML assertion.
Logged in as:
arn:aws:sts::282329364436:assumed-role/vz-console-soe-administrator/sachin.s@one.v
erizon.com

Your new access key pair has been stored in the AWS configuration.
Note that it will expire at 2026-10-17 05:39:16 +0530 IST
To use this credential, call the AWS CLI with the --profile option (e.g. aws
--profile osnd-prod ec2 describe-instances).
----------------------------------------
All 4 profiles processed!
```

.sso_pass file contents

<entered my system login pwd here>

.bashrc file contents

```
export PATH=$PATH:/c/Tools
export PATH="HOME/.local/bin:PATH"
export HTTPS_PROXY=http://server.proxy.vzwcorp.com:9290
export https_proxy=http://server.proxy.vzwcorp.com:9290
export AWS_PROFILE=saml
export PATH="$PATH:C:\Program Files\Amazon\AWSCLIV2"
export SAML2AWS_DISABLE_KEYCHAIN=false
export SAML2AWS_PASSWORD="Verizon*0"
alias saml2aws-vz='winpty saml2aws-vz'


aws-login-all() {
  local CACHE_PATH="$HOME/.aws/saml2aws/shared_vz_saml_cache"
  local ENVS=("osnd-dev" "osnd-test" "osnd-preprod" "osnd-prod")
  local FORCE_FLAG=""

  if [[ "1" == " - f"||"1" == "--force" ]]; then
    FORCE_FLAG="--force"
  fi

  # Read password from restricted file dynamically
  if [[ -f ~/.sso_pass ]]; then
    export SAML2AWS_PASSWORD=$(cat ~/.sso_pass)
  else
    echo "Error: Password file ~/.sso_pass not found."
    return 1
  fi

  read -p "Enter 6-digit DUO Passcode: " DUO_CODE
  echo ""

  for env in "${ENVS[@]}"; do
    echo "Authenticating: $env"
    \saml2aws-vz login \
      --idp-account "$env" \
      --profile "$env" \
      --duo-mfa-option "Passcode" \
      --mfa-token "$DUO_CODE" \
      --cache-saml \
      --cache-file "$CACHE_PATH" \
      --skip-prompt \
      --disable-keychain \
      $FORCE_FLAG
  done

  # Wipe from memory after authentication
  unset SAML2AWS_PASSWORD
}
```

.saml2aws file contents

```
[default]
name                  = default
app_id                =
url                   = https://ssologin.verizon.com
username              = srivsa
provider              = VerizonSSO
mfa                   = Auto
mfa_ip_address        =
skip_verify           = false
proxy                 = http://server.proxy.vzwcorp.com:9290
timeout               = 0
aws_urn               = urn:amazon:webservices
aws_session_duration  = 28800
aws_profile           = osnd
resource_id           =
subdomain             =
role_arn              =
policy_file           =
policy_arn_list       =
region                =
http_attempts_count   =
http_retry_delay      =
credentials_file      =
saml_cache            = false
saml_cache_file       =
target_url            =
disable_remember_device = false
disable_sessions      = false
download_browser_driver = false
headless              = false
prompter              =
kc_broker             =
meta_alias            = /employee/VZIDP_28054_JGBV_TPDSOEAWSSOE

[osnd-dev]
name                  = osnd-dev
app_id                =
url                   = https://ssologin.verizon.com
username              = srivsa
provider              = VerizonSSO
mfa                   = Auto
mfa_ip_address        =
skip_verify           = false
proxy                 = http://server.proxy.vzwcorp.com:9290
timeout               = 0
aws_urn               = urn:amazon:webservices
aws_session_duration  = 28800
aws_profile           = osnd-dev
```

```
resource_id            =
subdomain              =
role_arn               = arn:aws:iam::520949192567:role/vz-console-soe-administrator
policy_file            =
policy_arn_list        =
region                 =
http_attempts_count    =
http_retry_delay       =
credentials_file       =
saml_cache             = false
saml_cache_file        =
target_url             =
disable_remember_device = false
disable_sessions       = false
download_browser_driver = false
headless               = false
prompter               =
kc_broker              =
meta_alias             = /employee/VZIDP_28054_JGBV_TPDSOEAWSSOE

[osnd-test]
name                   = osnd-test
app_id                 =
url                    = https://ssologin.verizon.com
username               = srivsa
provider               = VerizonSSO
mfa                    = Auto
mfa_ip_address         =
skip_verify            = false
proxy                  = http://server.proxy.vzwcorp.com:9290
timeout                = 0
aws_urn                = urn:amazon:webservices
aws_session_duration   = 28800
aws_profile            = osnd-test
resource_id            =
subdomain              =
role_arn               = arn:aws:iam::157520333689:role/vz-console-soe-administrator
policy_file            =
policy_arn_list        =
region                 =
http_attempts_count    =
http_retry_delay       =
credentials_file       =
saml_cache             = false
saml_cache_file        =
target_url             =
disable_remember_device = false
disable_sessions       = false
download_browser_driver = false
headless               = false
```

```
prompter              =
kc_broker             =
meta_alias            = /employee/VZIDP_28054_JGBV_TPDSOEAWSSOE

[osnd-preprod]
name                  = osnd-preprod
app_id                =
url                   = https://ssologin.verizon.com
username              = srivsa
provider              = VerizonSSO
mfa                   = Auto
mfa_ip_address        =
skip_verify           = false
proxy                 = http://server.proxy.vzwcorp.com:9290
timeout               = 0
aws_urn               = urn:amazon:webservices
aws_session_duration  = 28800
aws_profile           = osnd-preprod
resource_id           =
subdomain             =
role_arn              = arn:aws:iam::837767801504:role/vz-console-soe-administrator
policy_file           =
policy_arn_list       =
region                =
http_attempts_count   =
http_retry_delay      =
credentials_file      =
saml_cache            = false
saml_cache_file       =
target_url            =
disable_remember_device = false
disable_sessions      = false
download_browser_driver = false
headless              = false
prompter              =
kc_broker             =
meta_alias            = /employee/VZIDP_28054_JGBV_TPDSOEAWSSOE

[osnd-prod]
name                  = osnd-prod
app_id                =
url                   = https://ssologin.verizon.com
username              = srivsa
provider              = VerizonSSO
mfa                   = Auto
mfa_ip_address        =
skip_verify           = false
proxy                 = http://server.proxy.vzwcorp.com:9290
timeout               = 0
aws_urn               = urn:amazon:webservices
```

```
aws_session_duration    = 28800
aws_profile             = osnd-prod
resource_id             =
subdomain               =
role_arn                = arn:aws:iam::282329364436:role/vz-console-soe-administrator
policy_file             =
policy_arn_list         =
region                  =
http_attempts_count     =
http_retry_delay        =
credentials_file        =
saml_cache              = false
saml_cache_file         =
target_url              =
disable_remember_device = false
disable_sessions        = false
download_browser_driver = false
headless                = false
prompter                =
kc_broker               =
meta_alias              = /employee/VZIDP_28054_JGBV_TPDSOEAWSSOE
```

1. Add to User PATH via PowerShell (No Admin Required)

You do not need Administrator rights or access to the System Control Panel to update your personal User `PATH` variable.

1. Open **PowerShell**.
2. Run the following command to permanently add `C:\Tools` (or wherever your `saml2aws-vz.exe` is located) to your user environment:

```shell
[Environment]::SetEnvironmentVariable("Path", $env:Path + ";C:\tools\saml2aws-vz.win.2026.08.26-vz1", "User")
```

3. Close and reopen PowerShell, then verify:

```shell
saml2aws-vz --version
```

Verification: If it returns a version number (e.g., `2026.08.26-vz1`), the command was successful.

2. One-Time Configuration

Run the one-time configuration command to create your AWS profile configuration:

```shell
saml2aws-vz configure --session-duration=28800 --profile osnd-dev
```

When prompted:

- **AWS Profile:** Press **Enter** (defaults to `saml`).
- **Username:** Type your Verizon SSO username (e.g., `jsmith`).
- **Meta Alias:** Press **Enter** to accept the default TPD SOE integration, unless your platform team specified a custom Meta Alias string for your org.

View configured profiles:

```shell
notepad $HOME\.saml2aws
```

3. Configure Corporate Proxy (PowerShell Profile)

Verizon SSO authentication and AWS CLI operations require routing through the corporate network proxy.

1. Open your PowerShell profile in Notepad:

```shell
notepad $PROFILE
```

*(If prompted to create a new file, click **Yes**.)*

2. Add the following line to the file, save, and exit:

```shell
$env:HTTPS_PROXY = "http://server.proxy.vzwcorp.com:9290"
```

3. Apply the updated profile to your current PowerShell session:

```shell
. $PROFILE
```

4. Authenticate & Obtain Credentials

Ensure you are connected to the corporate VPN or on-net before running the login command:

```shell
saml2aws-vz login --profile osnd-dev
```

1. Enter your Verizon SSO password when prompted.
2. Approve the DUO Push notification on your mobile device.
3. Use the arrow keys to select your designated AWS IAM Role from the interactive list and press **Enter**.

This step stores temporary AWS credentials (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_SESSION_TOKEN`) into `%USERPROFILE%\.aws\credentials` under the `[saml]` profile.

> **Tip to reduce DUO prompts:** Add the `--cache-saml` flag during subsequent logins within the same session to reuse your active SAML assertion without triggering a new DUO push:
>
> PowerShell

```shell
saml2aws-vz login --profile saml --cache-saml
--duo-mfa-option "Duo Push"
```

Fix Configuration Overwrites for Profiles:

Critical Config Fix.
In your logs, every time you ran saml2aws-vz configure --profile <name>, it printed:

Configuration saved for IDP account: default

Because the --idp-account flag was not specified, each command **overwrote the default account** rather than creating separate profiles.

To fix this, pass -a <profile-name> so each environment gets its own isolated config:

```
Shell
saml2aws-vz configure -a osnd-dev --profile osnd-dev --session-duration=28800

saml2aws-vz configure -a osnd-test --profile osnd-test --session-duration=28800

saml2aws-vz configure -a osnd-preprod --profile osnd-preprod
--session-duration=28800

saml2aws-vz configure -a osnd-prod --profile osnd-prod --session-duration=28800
```

Verification: Check %USERPROFILE%\.saml2aws in Notepad to ensure four separate [osnd-dev], [osnd-test], [osnd-preprod], and [osnd-prod] sections exist.

Log In to Target Environment:

Authenticate.
Once VPN is active and the proxy is loaded in your current terminal session, run login with matching IDP account and profile flags:

PowerShell

```
None
saml2aws-vz login -a osnd-test --profile osnd-test
```

Verification: You will be prompted for your password, receive a DUO push on your phone, and see your temporary access keys saved to %USERPROFILE%\.aws\credentials.

NewRelic Implementation:

```
Shell
kubectl get mutatingwebhookconfigurations
```

Sample o/p:

NAME	WEBHOOKS	AGE
0500-amazon-eks-fargate-mutation.amazonaws.com	2	5y261d
aws-load-balancer-webhook	2	5y261d
newrelic-newrelic-infra-operator	1	3y258d
newrelic-nri-metadata-injection	1	3y258d
pod-identity-webhook	1	5y261d
vpc-resource-mutating-webhook	1	5y261d

The output confirms that **`newrelic-newrelic-infra-operator`** is the mutating webhook injecting the sidecar into your pods.

Since Helm is not managing these resources, the proxy setting is defined directly inside the Operator's configuration (either in its Deployment, ConfigMap, or Custom Resources) in your cluster.

Follow these steps to find where the proxy URL/IP is defined and update it:

Step 1: Check the Operator's Deployment and ConfigMaps

Find the namespace and deployment running `newrelic-infra-operator`:

```
Shell
kubectl get deployment -A | grep newrelic
```

Once you identify the namespace (e.g., `newrelic` or `default`), check the deployment spec for proxy environment variables:

Namespace = newrelic-int

```shell
kubectl get deployment -n newrelic-int -l app.kubernetes.io/name=newrelic-infra-operator -o yaml | grep -iE 'proxy|pact'
```

Also check any ConfigMaps in that namespace:

```shell
kubectl get configmaps -n newrelic-int -o yaml | grep -iE 'proxy|pact'
```

Step 2: Check New Relic Custom Resources (CRDs)

The `newrelic-infra-operator` often injects sidecars based on Custom Resources (such as `NewRelicInfrastructure` or `ControlPlane`).

List all custom resources managed by New Relic:

```shell
kubectl get crds | grep newrelic
```

If you see custom resources like `instrumentations.newrelic.com` or similar, check their manifests for proxy overrides:

```shell
kubectl get instrumentations.newrelic.com -A -o yaml | grep -iE 'proxy|pact'
```

Step 3: Identify the exact ConfigMaps containing the proxy setting

Run this command to print the specific names of the ConfigMaps that contain `proxy.pact.verizon.com`:

```shell
kubectl get configmaps -n newrelic-int -o jsonpath='{range .items[*]}{.metadata.name}{"\n"}{.data}{"\n---\n"}{end}' | grep -B 2 -A 5 "proxy.pact.verizon.com"
```

Alternatively, list all ConfigMaps in `newrelic-int` to inspect them directly:

```shell
kubectl get configmaps -n newrelic-int
```

Here are the exact 4 ConfigMaps in the `newrelic-int` namespace that are injecting the `proxy.pact.verizon.com:8080` setting into your New Relic sidecars and agents:

1. **`newrelic-nri-kube-events-agent-config`**
2. **`newrelic-nrk8s-agent-controlplane`**
3. **`newrelic-nrk8s-agent-ksm`**
4. **`newrelic-nrk8s-agent-kubelet`**

Squid-proxy:

```none
http://proxy-service.proxy:3128/
```

Step 1: Edit/Update proxy in configmaps

```shell
kubectl edit configmap <configmap name> -n <newrelic namespace>
```

```
kubectl edit configmap newrelic-nri-kube-events-agent-config -n newrelic-int
kubectl edit configmap newrelic-nrk8s-agent-controlplane -n newrelic-int
kubectl edit configmap newrelic-nrk8s-agent-ksm -n newrelic-int
kubectl edit configmap newrelic-nrk8s-agent-kubelet -n newrelic-int
```

Step 2: Restart New Relic Deployments

Restart the New Relic operator and controller pods so they reload the updated ConfigMaps:

Bash

None

```
kubectl rollout restart deployment newrelic-newrelic-infra-operator
-n newrelic-int
kubectl rollout restart deployment newrelic-nri-metadata-injection
-n newrelic-int
kubectl rollout restart deployment newrelic-nri-kube-events -n
newrelic-int
```

Step 3: Restart all of Your Application Deployment

Because sidecar injection occurs during pod creation, restart your application deployment so the mutating webhook injects the updated sidecar without proxy parameters:

None

```
kubectl rollout restart deployment spog-admin-app-deployment -n
<your-app-namespace>
```

Step 4: Verify Direct Outbound Traffic

1. Check the logs of the newly created sidecar container:

None

```
kubectl logs spog-admin-app-deployment-<new-pod-id> -c
newrelic-infrastructure-sidecar -n <your-app-namespace>
```

2. Verify that connection errors to `162.247.243.20:443` (`infra-api.newrelic.com`) disappear, and traffic no longer hits the `SPOG-OSND` proxy policy.

Edit the Deployment Spec

1. Open the deployment in your editor:

```
None
kubectl edit deployment newrelic-nri-prometheus -n newrelic-int
```

2. Locate `EMITTER_PROXY` under `env:` and update it:

YAML

```
None
- name: EMITTER_PROXY
  value: http://proxy-service.proxy:3128/
```

3. Save and exit (`:wq`).

Identify and list pact proxy usage:

```
Shell
echo "=== 1. CONFIGMAPS (PROXY HOST SETTINGS) ==="
kubectl get configmaps -A -o json | jq -r '
  .items[] |
  select(.data | tostring | test("proxy\\.pact\\.verizon\\.com"; "i")) |
  "Namespace: " + .metadata.namespace + " | Kind: ConfigMap | Name: " +
.metadata.name'

echo "=== 2. DEPLOYMENTS (CONTAINER ENV VARS) ==="
kubectl get deployments -A -o json | jq -r '
  .items[] |
  select(.spec.template.spec.containers[].env[]?.value? | strings |
test("proxy\\.pact\\.verizon\\.com"; "i")) |
  "Namespace: " + .metadata.namespace + " | Kind: Deployment | Name: " +
.metadata.name'
```

```
echo "=== 3. DAEMONSETS & STATEFULSETS ==="
kubectl get daemonsets,statefulsets -A -o json | jq -r '
  .items[] |
  select(.spec.template.spec.containers[].env[]?.value? | strings |
test("proxy\\.pact\\.verizon\\.com"; "i")) |
  "Namespace: " + .metadata.namespace + " | Kind: " + .kind + " | Name: " +
.metadata.name'

echo "=== 4. RUNNING PODS (ACTIVE INJECTED / RUNTIME ENV) ==="
kubectl get pods -A -o json | jq -r '
  .items[] |
  select(.spec.containers[].env[]?.value? | strings |
test("proxy\\.pact\\.verizon\\.com"; "i")) |
  "Namespace: " + .metadata.namespace + " | Kind: Pod | Name: " + .metadata.name'

echo "=== 5. SECRETS ==="
kubectl get secrets -A -o json | jq -r '
  .items[] |
  select(.data | tostring | test("proxy\\.pact\\.verizon\\.com"; "i")) |
  "Namespace: " + .metadata.namespace + " | Kind: Secret | Name: " +
.metadata.name'

echo "=== 6. MUTATING WEBHOOK CONFIGURATIONS ==="
kubectl get mutatingwebhookconfigurations -o json | jq -r '
  .items[] |
  select(. | tostring | test("proxy\\.pact\\.verizon\\.com"; "i")) |
  "Kind: MutatingWebhookConfiguration | Name: " + .metadata.name'
```

Identify and list pact proxy usage:

Shell

```
echo "=== 1. CONFIGMAPS (PROXY HOST SETTINGS) ==="
kubectl get configmaps -A -o json | jq -r '
  .items[] |
  select(.data | tostring | test("proxy\\.pact\\.verizon\\.com"; "i")) |
  "Namespace: " + .metadata.namespace + " | Kind: ConfigMap | Name: " +
.metadata.name'

echo "=== 2. DEPLOYMENTS (CONTAINER ENV VARS) ==="
kubectl get deployments -A -o json | jq -r '
  .items[] |
  select(.spec.template.spec.containers[].env[]?.value? | strings |
test("proxy\\.pact\\.verizon\\.com"; "i")) |
  "Namespace: " + .metadata.namespace + " | Kind: Deployment | Name: " +
.metadata.name'

echo "=== 3. DAEMONSETS & STATEFULSETS ==="
kubectl get daemonsets,statefulsets -A -o json | jq -r '
  .items[] |
  select(.spec.template.spec.containers[].env[]?.value? | strings |
test("proxy\\.pact\\.verizon\\.com"; "i")) |
  "Namespace: " + .metadata.namespace + " | Kind: " + .kind + " | Name: " +
.metadata.name'

echo "=== 4. RUNNING PODS (ACTIVE INJECTED / RUNTIME ENV) ==="
kubectl get pods -A -o json | jq -r '
  .items[] |
  select(.spec.containers[].env[]?.value? | strings |
test("proxy\\.pact\\.verizon\\.com"; "i")) |
  "Namespace: " + .metadata.namespace + " | Kind: Pod | Name: " + .metadata.name'

echo "=== 5. SECRETS ==="
kubectl get secrets -A -o json | jq -r '
  .items[] |
  select(.data | tostring | test("proxy\\.pact\\.verizon\\.com"; "i")) |
  "Namespace: " + .metadata.namespace + " | Kind: Secret | Name: " +
.metadata.name'

echo "=== 6. MUTATING WEBHOOK CONFIGURATIONS ==="
kubectl get mutatingwebhookconfigurations -o json | jq -r '
  .items[] |
  select(. | tostring | test("proxy\\.pact\\.verizon\\.com"; "i")) |
  "Kind: MutatingWebhookConfiguration | Name: " + .metadata.name'
```

```
Dart (unsupported)
=== 1. CONFIGMAPS (PROXY HOST SETTINGS) ===
Namespace: newrelic-int | Kind: ConfigMap | Name:
newrelic-nri-kube-events-agent-config
Namespace: newrelic-int | Kind: ConfigMap | Name:
newrelic-nrk8s-agent-controlplane
Namespace: newrelic-int | Kind: ConfigMap | Name: newrelic-nrk8s-agent-ksm
Namespace: newrelic-int | Kind: ConfigMap | Name: newrelic-nrk8s-agent-kubelet

=== 2. DEPLOYMENTS (CONTAINER ENV VARS) ===
Namespace: confluent | Kind: Deployment | Name: confluent-operator
Namespace: newrelic-int | Kind: Deployment | Name: newrelic-nri-prometheus

=== 3. DAEMONSETS & STATEFULSETS ===

=== 4. RUNNING PODS (ACTIVE INJECTED / RUNTIME ENV) ===
Namespace: confluent | Kind: Pod | Name: confluent-operator-7cf5d9cd57-7hhhg
Namespace: newrelic-int | Kind: Pod | Name:
newrelic-nri-prometheus-7b59f46bcb-jnqjf
Namespace: newrelic-int | Kind: Pod | Name:
newrelic-nri-prometheus-7ddff755d9-9hcql

=== 5. SECRETS ===

=== 6. MUTATING WEBHOOK CONFIGURATIONS ===
```

Proxy:
http://proxy-service.proxy:3128

1) **Namespace** → newrelic-int
   ConfigMaps:
   - Newrelic-nri-kube-events-agent-config (3)
   - Newrelic-nrk8s-agent-controlplane
   - Newrelic-nrk8s-agent-ksm
   - Newrelic-nrk8s-agent-kubelet

   Deployment → Newrelic-nri-prometheus
   EMITTER_PROXY=http://proxy-service.proxy:3128/

2) **Namespace** → confluent
   Deployment → confluent-operator
   HTTPS_PROXY=http://proxy-service.proxy:3128

Restart Deployments:

```shell
# 1. Restart New Relic Operator & Injector (reloads ConfigMaps for future sidecars)
kubectl rollout restart deployment newrelic-newrelic-infra-operator -n newrelic-int

kubectl rollout restart deployment newrelic-nri-metadata-injection -n newrelic-int

# 2. Restart New Relic Prometheus
kubectl rollout restart deployment newrelic-nri-prometheus -n newrelic-int

# 3. Restart Confluent Operator
kubectl rollout restart deployment confluent-operator -n confluent

# 4. Restart Application Deployment (re-injects clean sidecar container)
kubectl rollout restart deployment spog-inventory-app-deployment -n applications
```

New Relic Sidecar ImagePullBackOff — Test Environment (oslte cluster)

Executive Summary

Starting mid-September, the New Relic infrastructure monitoring sidecar failed to start on several application pods in the test EKS cluster,
blocking New Relic metrics collection for those pods (application traffic itself was largely unaffected). Investigation traced the cause to
a network egress failure to Docker Hub occurring somewhere beyond our VPC's Transit Gateway, which is owned and managed by a separate
central networking account — not by anything in our application configuration. Root cause has been isolated and escalated to that team; a
permanent mitigation (removing our dependency on Docker Hub entirely) is planned in parallel.

Problem Statement

Original issue: The newrelic-infrastructure-sidecar container — auto-injected into every pod by New Relic's Kubernetes infra-operator — was
stuck in ImagePullBackOff on multiple deployments (spog-portal-app, spog-osnd-mcp, and later spog-wsevents-app after a pod restart) in the
applications namespace of the test cluster.

Observed symptoms:
- Repeated Failed to pull image "newrelic/infrastructure-k8s:2.13.15-unprivileged" events, citing rpc error: code = DeadlineExceeded and
  dial tcp <ip>:443: i/o timeout against registry-1.docker.io, with the destination IP changing each retry.
- kubectl describe pod showed the sidecar cycling through Pulling → ErrImagePull → ImagePullBackOff indefinitely.
- All other containers in the same pods (application container, fluentd) pulled successfully from our private ECR and reached Running/Ready
  in seconds.
- One affected deployment (spog-osnd-mcp) additionally showed a high restart count (300+) on a different container — flagged as a likely
  separate, unrelated issue for follow-up.

Impact:
- Primary impact: loss of New Relic infrastructure metrics (CPU/memory/network telemetry) for affected pods — an observability gap, not a
  confirmed application outage, since application and logging containers remained healthy in most affected pods.
- Secondary/self-inflicted impact: deleting a previously healthy pod (spog-wsevents-app) during troubleshooting caused it to hit the same

failure on recreation, briefly extending the blast radius. This is now avoided pending root cause resolution.

Investigation & Key Findings

Steps taken:

1. Reviewed all deployment.yaml manifests in the oslte GitOps repo and confirmed the sidecar is not defined anywhere in our config — it's injected cluster-wide by New Relic's infra-operator (Helm release newrelic-bundle, chart nri-bundle-6.0.23, namespace newrelic) via a MutatingWebhookConfiguration.

```
[ec2-user@ip-63-106-228-51 bin]$ kubectl get mutatingwebhookconfigurations
NAME                                        WEBHOOKS   AGE
0500-amazon-eks-fargate-mutation.amazonaws.com   2          5y261d
aws-load-balancer-webhook                   2          5y261d
newrelic-newrelic-infra-operator            1          3y258d
newrelic-nri-metadata-injection             1          3y258d
pod-identity-webhook                        1          5y261d
vpc-resource-mutating-webhook               1          5y261d
```

2. Compared a failing pod against a healthy pod running the identical injected sidecar spec (same infra-operator.newrelic.com/agent-injected hash) — ruled out a config or image-version change as the cause.
3. Resolved each affected pod's ENI to its subnet/AZ via aws ec2 describe-network-interfaces and cross-referenced against healthy pods in the same subnets — ruled out a single bad subnet, since healthy and failing pods coexisted in the same subnet.
4. Deleted the healthy spog-wsevents-app pod as a live test: the replacement pod immediately hit the identical failure. This proved the issue is tied to when a pod is created, not where — any pod created after a certain point in time fails, regardless of AZ/subnet, while pods already running continue unaffected.
5. Narrowed the break window using last-known-good vs. first-known-bad pull timestamps: roughly Sep 14, 14:38 UTC – Sep 15, 12:00 UTC.
6. Confirmed no NAT Gateway exists in the VPC; inspected route tables for both affected subnets and found both route 0.0.0.0/0 to the same Transit Gateway (tgw-0f61e84327d8498b0), confirming a single shared egress path serves both affected AZs.
7. Inspected both security groups attached to the affected pods' ENIs (eks-cluster-sg-... and an AWS Firewall Manager–managed SG) — both explicitly permit outbound 443 to 0.0.0.0/0. Ruled out security groups as the blocker.
8. Queried the Transit Gateway directly and found it is owned by a separate AWS account (013337034102), described as "SOE Transit

Gateway-qat" — a centrally managed platform/network resource outside our team's account and visibility.

Root cause (as currently understood): Egress from our VPC to the public internet (specifically registry-1.docker.io / Docker Hub) is failing
at some point beyond our Transit Gateway, inside infrastructure owned by a central networking account we don't have visibility into. All
evidence (identical config, multiple AZs affected simultaneously, fully permissive local security groups and route tables, a clear
time-boxed onset) points to a change on that side — most likely a firewall, proxy, or domain-allowlist policy that stopped permitting
traffic to Docker Hub — rather than anything in our application, Kubernetes manifests, or local AWS networking.

Resolution Status & Planned Changes

No fix has been applied yet. This is currently at the escalation stage:

- Escalated to: the owning team for AWS account 013337034102 (Transit Gateway tgw-0f61e84327d8498b0), with the specific ask of checking for
  a firewall/proxy/Network Firewall policy change affecting Docker Hub reachability in the Sep 14 14:38 – Sep 15 12:00 UTC window.
- Planned mitigation (independent of the escalation outcome): mirror the image newrelic/infrastructure-k8s:2.13.15-unprivileged into our
  private ECR registry and update the infra-operator Helm release configuration to pull the sidecar image from ECR instead of Docker Hub —
  the same pattern already used for our fluentd and application images. This removes the dependency on public internet egress entirely and
  would prevent recurrence regardless of what the central network team finds.
- No changes have been made to test-config (the GitOps repo) as part of this investigation, since the sidecar is not defined there.

Verification & Testing

Not yet applicable — no fix has been deployed. Once the ECR mirror is in place, verification will consist of:
- Confirming the infra-operator config points at the ECR image path.
- Deleting/recreating an affected pod and confirming the sidecar reaches Running/Ready.
- Confirming New Relic infrastructure metrics resume for previously affected pods.
- If the network team identifies and fixes a policy on their side, verifying by recreating a previously-failing pod and confirming a
  successful pull directly from Docker Hub.

Lessons Learned & Future Recommendations

- Reduce reliance on public registries for cluster-critical/monitoring components. Our app and logging images are already mirrored to ECR
  for reliability; the New Relic sidecar was the one exception, and it's the one that broke. Recommend mirroring all third-party images used
  cluster-wide to ECR as a standing practice.
- Avoid deleting/recreating "healthy" pods during live troubleshooting of a suspected environmental issue until the blast radius is
  understood — doing so converted one working pod into a failing one during this investigation. Worth a quick team reminder.
- Limited visibility into shared network infrastructure (Transit Gateway, Firewall Manager policies) slowed the investigation. Consider
  requesting standing read access (or a documented contact/on-call path) to the central networking account for faster diagnosis of future
  egress issues.
- No alerting currently exists for sidecar ImagePullBackOff states. This was discovered manually. Recommend adding a basic alert (e.g., on
  ImagePullBackOff reason across the cluster) so similar issues surface proactively rather than being found during unrelated pod inspection.

Proxy

Identify the Deployments with the PACT Proxy Value

Run this command to get the exact **Namespace**, **Deployment Name**, and **Container / Env Var Name** where `proxy.pact.verizon.com` is configured:

Shell

```
kubectl get deployments -A -o json | jq -r '
  .items[] |
  select(.spec.template.spec.containers[].env[]?.value | strings | contains("proxy.pact.verizon.com")) |
  "Namespace: " + .metadata.namespace + " | Deployment: " + .metadata.name' | sort -u
```

our app is hosted in both east-1 and west-2 regions of aws account. I tried to access the app and it worked fine.

I brought down both regions and brought back up --> app access worked fine

I brought down east region alone and tried to access the app --> saw internal server error

I cleared my browser history include cache and cookies and retried to access the app --> access worked fine

Here is a complete technical breakdown of what occurred during your regional failover test, why the application broke, and how to fix it at both the application and infrastructure levels.

What Happened (Step-by-Step Scenario)

1. **Initial Baseline (Both Regions Active):** You accessed oslte-admin-preprod.pact.verizon.com. Traffic was routed (via AWS Route53 or your Global Load Balancer) to **us-east-1**. us-east-1 authenticated your session and set several HTTP cookies in your browser, including SESSION, XSRF-TOKEN, and load balancer stickiness cookies (NSC_... / SSOUATLB).
2. **First Failover Test (Both Down → Both Up):**
   When both regions restarted, no active sticky state was maintained improperly, and both regions came back up with identical state/caches. Request routing succeeded.
3. **The Failure Trigger (us-east-1 Brought Down):** When us-east-1 went down, your global load balancer redirected your browser's initial GET / document request to **us-west-2**. However, your browser automatically attached the existing us-east-1 session cookies (SESSION, XSRF-TOKEN, etc.).
4. **The HTTP 500 Crash: us-west-2** received the request, read the incoming SESSION / XSRF-TOKEN cookies, but failed to process them. Because the session data or encryption keys were tied exclusively to us-east-1, us-west-2 threw an uncaught exception during session validation or CSRF verification, sending back a raw 500 Internal Server Error page ({"message": "Internal server error"}).
5. **The "Missing Network Call" Illusion:** Because the initial HTML document request failed with HTTP status 500, no application JavaScript loaded into your browser. Without JavaScript running, no secondary Fetch/XHR API calls (such as /session or /token) were ever executed. When DevTools was filtered to Fetch/XHR, it appeared as though no API request had occurred.
6. **Clearing Browser History/Cookies Fixed It:** Wiping cookies deleted the invalid us-east-1 session markers. When you re-accessed the site, your browser sent a clean request to us-west-2 with no session headers. us-west-2 treated you as a new user, created a fresh session, and loaded successfully.

Root Cause Analysis

The root cause is a **lack of cross-region session synchronization and unhandled authentication exceptions during failover**:

- **Unsynchronized Session State / Memory Cache:** Session data created in us-east-1 was either kept in local instance memory or an isolated Redis/ElastiCache cluster in us-east-1. us-west-2 had no record of the session ID.
- **Mismatched Encryption Keys / CSRF Secrets:** If application secrets or CSRF signing keys differ between regions, us-west-2 cannot decrypt or verify tokens issued by us-east-1, leading to decryption failures.
- **Poor Exception Handling in Auth Middleware:** When the backend received a session cookie it could not find or decrypt, instead of cleanly invalidating the bad cookie and redirecting to login, the application threw an uncaught exception resulting in an HTTP 500.
- **Sticky Session / Load Balancer Persistence:** The presence of NSC_... cookies indicates a NetScaler or ALB stickiness policy attempting to pin your browser to us-east-1 infrastructure.

How to Fix This

To achieve seamless multi-region high availability without requiring users to clear browser cookies during an outage:

1. Code Fix: Graceful Auth Fallback (Immediate)

Update your Spring / Node / NGINX backend authentication middleware:

- Wrap session token decryption and lookup logic in a try/catch block.
- If a session ID is missing from the cache or a CSRF token cannot be decrypted, do **not** return a 500 Internal Server Error.
- Automatically invalidate/clear the incoming SESSION and XSRF-TOKEN cookies in the HTTP response headers and issue a 302 Redirect to the login route (or issue a fresh guest session).

2. Infrastructure Fix: Synchronized Global Session Store

- Transition session storage from regional/in-memory caches to a cross-region replicated storage engine, such as **AWS DynamoDB Global Tables** or **Amazon ElastiCache with Global Datastore**.
- Ensure application secret keys (e.g., JWT_SECRET, Spring Session encryption keys) are identical across both us-east-1 and us-west-2 deployment pipelines (via AWS Secrets Manager).

3. Routing Fix: Sticky Session Failover Policies

- Ensure load balancer / API Gateway health checks instantly strip stickiness headers (NSC_... or SSOUATLB) on target group failure so traffic routes cleanly to us-west-2 without attempting proxying back to us-east-1.

What We Learned From Your Kubernetes Output

1. **Backend Framework:** Your application (`oslte-spog-admin`) is a **Java / Spring Boot** microservice running on Java (indicated by `JVM_OPTS` and `actuator/health` probes).
2. **Configuration Source:** The app loads its configuration from a ConfigMap named `oslte-admin-spring-config` and mounts secrets from `oslte-admin-spring-secret` at `/etc/secrets`.
3. **No Distributed Session Store Configured:** There are **no environment variables or configuration entries for Redis, Memcached, or DynamoDB**.

Why the HTTP 500 Failure Happened

Because Spring Boot has no external Redis/database session provider configured, it defaults to **In-Memory Session Storage (In-Memory Tomcat HTTP Sessions)**.

1. **Session Isolation:** When you authenticated in `us-east-1`, your session was created directly in the RAM of the `spog-admin-app` pod running in `us-east-1`.
2. **Missing State in `us-west-2`:** When `us-east-1` went down and traffic failed over to `us-west-2`, your request arrived at the `us-west-2` pod carrying the `SESSION` cookie.
3. **The Uncaught Exception:** The Spring Boot pod in `us-west-2` searched its local memory for that session ID, found nothing, and threw an uncaught session/security exception (e.g., `InvalidSessionException` or `CsrfException`), returning the raw `{"message": "Internal server error"}` response.
   PNG

Step-by-Step Action Plan

Step 1: Inspect the Hidden Spring Boot Configuration

Run these two commands to inspect the exact Spring properties and secret values being passed to the Java application:

Bash

```
None
# 1. View the JVM options and Spring profile
kubectl get configmap oslte-admin-spring-config -n applications -o yaml

# 2. View the mounted secrets (e.g., database URLs or encryption keys)
```

```
kubectl get secret oslte-admin-spring-secret -n applications -o
yaml
```

Step 2: Immediate Code Fix (Spring Security Exception Handling)

Inform the development team to update the Spring Security configuration.

When Spring encounters an expired or invalid session ID/CSRF token, it should clear the cookie and redirect the user cleanly to the login page rather than throwing an unhandled exception:

Java

```
None
// Spring Security Configuration snippet
http
    .sessionManagement()
        .invalidSessionUrl("/login?expired=true")
    .and()
    .csrf()
        .accessDeniedHandler(customCsrfAccessDeniedHandler());
```

- **Verification:** Deploying this fix ensures that when `us-east-1` fails over, users will be redirected to the login screen instead of seeing an HTTP 500 error.
  PNG

Step 3: Infrastructure Fix (Cross-Region Redis Session Store)

To keep users logged in seamlessly across regions without requiring them to re-authenticate during a failover:

1. **Deploy AWS ElastiCache for Redis (Global Datastore):**
   - Have Network/Infra Ops create a Redis cluster with Global Datastore enabled between `us-east-1` (Primary Write) and `us-west-2` (Secondary Read/Write).
2. **Enable `spring-session-data-redis`:**
   - The dev team adds the `spring-session-data-redis` dependency to the Spring Boot project.
   - Update `oslte-admin-spring-config` to include:
   - YAML

```
None
spring.session.store-type: redis
spring.redis.host: <elasticache-global-endpoint>
spring.redis.port: 6379
```

    -

3. **Sync Secret Keys Across Regions:**
    - Ensure that `oslte-admin-spring-secret` in `us-east-1` and `us-west-2` share identical values for signing/encrypting session cookies and CSRF tokens. PNG+ 1

To help refine the exact Spring properties, what values do you see when you run `kubectl get configmap oslte-admin-spring-config -n applications -o yaml`?

Implementing Step 2 alone **will stop the application from throwing an HTTP 500 error**, but **it will not keep the user logged in** during a failover.

Here is the exact difference in user experience between implementing **Step 2 alone** versus implementing **both Step 2 and Step 3**:

Comparison of Solutions

Behavior / Experience	**Step 2 Alone (Graceful Fallback)**	**Step 2 + Step 3 (Global Session Store)**
Prevents HTTP 500 "Internal Server Error"?	**Yes** — Catches the exception and handles it cleanly.	**Yes** — The new region finds the session data immediately.
User Experience during Failover	**Logged out.** The user is seamlessly redirected to the /login page and must enter credentials again.	**Zero interruption.** The user stays logged in, and their active session continues uninterrupted.

Browser Cache Requirement	Users no longer need to clear cookies/cache to recover.	Users no longer need to clear cookies/cache to recover.
DevOps & Infra Complexity	**Low** — Application code change only; no new AWS infrastructure required.	**Medium/High** — Requires setting up AWS ElastiCache Redis Global Datastore or DynamoDB.

Is Step 2 Alone "Enough"?

- **Yes, for high-availability reliability (Minimum Viable Fix):** If your primary goal is to **stop site outages, prevent customer-facing HTTP 500 pages, and eliminate manual cookie clearing**, Step 2 is enough. It gracefully handles the missing session by sending the user to the login page.
- **No, for true active-passive / active-active seamless failover:**
  If your business requirement states that **users must remain logged in during an AWS region outage without re-authenticating**, then Step 2 alone is not enough. You need Step 3 to replicate session state across regions.

Route53 Mapping for https://oslte-admin-preprod.pact.verizon.com/

Shell

```
# cmd
dig oslte-admin-preprod.pact.verizon.com CNAME +short

# o/p --> displays cname record mapping
oslte-admin.osndpreprod.pact.verizon.com.
```

Shell

```
# cmd
dig oslte-admin.osndpreprod.pact.verizon.com +short

# o/p
63.106.192.186
63.106.192.156
63.106.192.203
```

TypeScript

```
1. User enters URL in Browser
   https://oslte-admin-preprod.pact.verizon.com
       |
       ▼
2. Corporate DNS / VPC Resolver
   Maps CNAME ——> oslte-admin.osndpreprod.pact.verizon.com
       |
       ▼
3. Route 53 Latency & Health Check Engine
   Hosted Zone: osndpreprod.pact.verizon.com
   Evaluates health of US-East (1a75f9de...) vs US-West (ec7613f2...)
   Selects ——> va-oslte-admin.osndpreprod.pact.verizon.com (US-East)
       |
       ▼
4. Route 53 Regional Alias A Record
   Resolves va-oslte-admin.osndpreprod.pact.verizon.com
   Alias to ——> dualstack.internal-k8s-nginx...us-east-1.elb.amazonaws.com
       |
       ▼
5. Final IP Resolution
   Returns Private IPs directly to client:
   • 63.106.192.186
```

```
   • 63.106.192.156
   • 63.106.192.203
       |
       ▼
6. Request Lands on NGINX Load Balancer (us-east-1)
   Routes packet through EKS Ingress ——> spog-admin-app Pod (Port 8080)
```

Final Key Insights for Your Failover Issue

1. **DNS Resolution is Dynamic:**
   When us-east-1 goes down, Step 3 in Route 53 flips the selection to or-oslte-admin.osndpreprod.pact.verizon.com (US-West) once the health check fails.
2. **Why the Error Occurred:** The failure you saw was not a DNS failure—it was caused by **stale browser session cookies (NSC_... / SESSION)** pinning your browser to us-east-1 state, coupled with **uncaught session exceptions** on the Spring Boot backend when traffic shifted to us-west-2.
3. **Your Next Steps as DevOps:**
   - **Application:** Have the dev team add graceful Spring Security handling for invalid/missing session tokens so it redirects to /login instead of throwing an HTTP 500.
   - **Infrastructure:** Ensure Route 53 health check intervals are set to fast (10s) to minimize failover detection delay.

Q) when we bring down east then how come post clearing the cache all the request goes to west only and not east?

The reason all requests go to us-west-2 after clearing the cache—even though the nodes in us-east-1 are still running—comes down to **how your ingress and load balancer use cookies to make routing decisions**.

When both regions are up, your global routing infrastructure uses **Sticky Sessions (Session Affinity)** to ensure a user stays connected to the exact same server region.

Here is the exact step-by-step mechanism of what happens behind the scenes:

Step 1: Before Clearing Cache (Traffic Stays Pinched to us-east-1)

- Your browser sent the initial request with a netscaler/load-balancer stickiness cookie (NSC_... or SSOUATLB) that pointed directly to us-east-1.
- Even though the pods in us-east-1 were down, the **Load Balancer / Proxy read that cookie first**.
- Because the cookie explicitly instructed: *"Route this user to us-east-1,"* the load balancer blindly forwarded the request to us-east-1.
- us-east-1 received the packet, found no pod listening on port 8080, and failed.

Step 2: After Clearing Cache & Cookies (Forced Failover)

- Clearing your browser history completely deleted the NSC_... and SSOUATLB stickiness cookies.
- When you typed the URL and hit Enter again, your browser sent a **"clean" HTTP GET request** with no routing markers attached.
- The Global Load Balancer / Route 53 received this un-stickied request and evaluated the targets:
    1. **Check us-east-1:** The load balancer tried to open a connection to the application port or health check endpoint in us-east-1. Because the pods were down, the probe failed or timed out.
    2. **Check us-west-2:** The load balancer probed us-west-2. The spog-admin-app pods were active and returned 200 OK on /actuator/health.
- Seeing that us-east-1 failed the application connection and us-west-2 was active, the load balancer routed 100% of new traffic to **us-west-2**.
- us-west-2 then issued a **new** stickiness cookie and SESSION cookie tied exclusively to us-west-2.

Key Takeaway for DevOps

The load balancer doesn't just look at whether the EC2/EKS node is alive; it relies on **cookie-based routing rules**.

- **With the old cookie:** The load balancer prioritized the cookie's instruction over target health, forcing your request into the offline us-east-1 pod path.
- **Without the cookie:** The load balancer had to make a fresh routing decision based strictly on active application health checks, which correctly directed you to us-west-2.

Q) how cookie explicitly instruct the load balancer to blindly forward requests to the east region?

The load balancer encodes routing information directly into the value of the stickiness cookie when it is first issued.

When a cookie-based sticky session is configured on an enterprise load balancer (like Citrix NetScaler, F5, or AWS ALB), it uses **opaque state encoding**:

1. How the Cookie Encodes the Target Region

When you first visited the site, the load balancer in us-east-1 generated the cookie named NSC_JOo0brl2c1pf50yenrndxdkl5jlnpct.

The cookie value ffffffff09457cd745525d5f4f58455e445a4a423b43 is an encrypted/hex-encoded string that contains:

- The exact IP address of the backend service in us-east-1.
- The server port number.
- Internal routing pool identifiers.

To your browser, it is just a random string of characters. But to the load balancer, it is a direct pointer to us-east-1.

2. The "Blind Forwarding" Execution Path

When you hit refresh *before* clearing your browser cache:

1. **Browser Behavior:** The browser inspects its cookie store, matches the domain oslte-admin-preprod.pact.verizon.com, finds NSC_JOo0brl2c1pf50yenrndxdkl5jlnpct, and attaches it to the HTTP Request header.
2. **Load Balancer Behavior:** The Global Load Balancer receives the HTTP header, extracts the NSC_... cookie, decodes the hex string, and extracts the target destination (us-east-1 backend IP).
3. **Routing Decision:** Instead of running a fresh health evaluation, the load balancer's sticky session algorithm takes priority. It forwards the packet straight to the us-east-1 backend IP encoded in the cookie.

4. **Connection Failure:** Because the us-east-1 pods were down, the target node failed to answer on port 8080, returning the 500 error or timing out.

Why Clearing the Cookie Breaks the Loop

When you cleared your browser cookies:

1. The next HTTP request was sent with **no NSC_... cookie attached**.
2. The load balancer inspected the incoming headers, found no stickiness instructions, and defaulted to **Health-Based Load Balancing**.
3. It checked active backend targets, saw that us-west-2 was answering healthy checks, routed your traffic to us-west-2, and issued a **brand new** stickiness cookie mapped to us-west-2.

How to Fix Load Balancer "Blind" Routing

To prevent a load balancer from honoring cookies for offline targets, the Network/Infrastructure Ops team must enable **Fallback / Health-Aware Session Stickiness** on the NetScaler or Application Load Balancer:

Plaintext

```
None
Sticky Session Override Rule:
IF (Target IP encoded in cookie is UNHEALTHY)
THEN ignore cookie -> re-route traffic to healthy region -> issue new stickiness cookie
```

When this configuration rule is enabled on the load balancer, traffic fails over automatically to us-west-2 even if the browser sends an old us-east-1 stickiness cookie

Q) how can this rule fix the issue since load balancers are region based?

Sticky Session Override Rule:
IF (Target IP encoded in cookie is UNHEALTHY)
THEN ignore cookie -> re-route traffic to healthy region -> issue new stickiness cookie

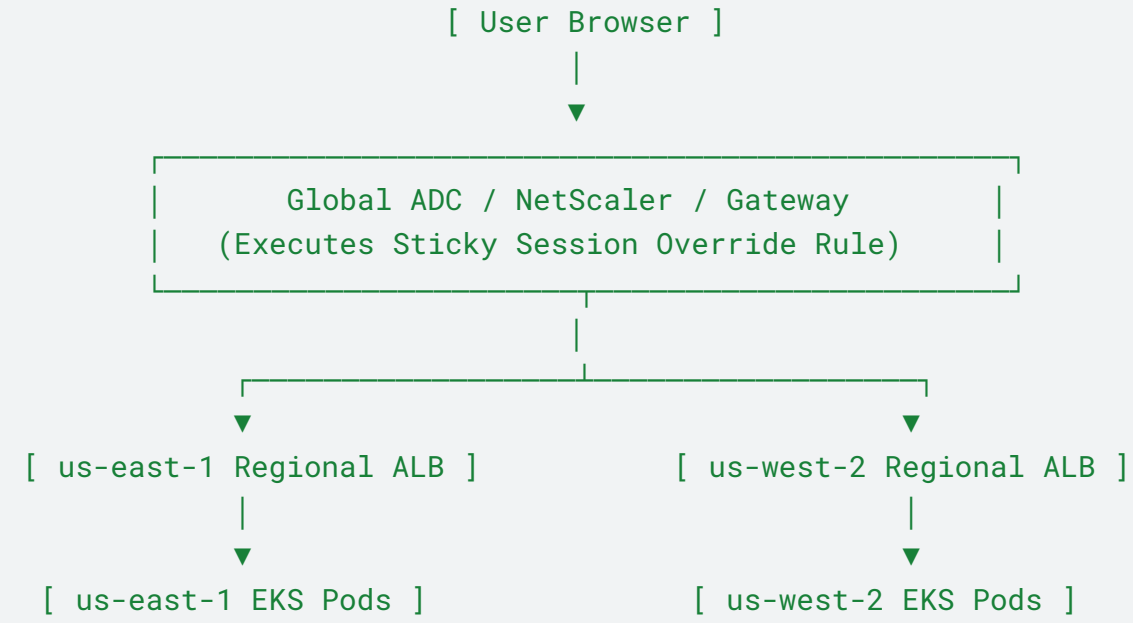
Answer:

To understand how that rule works, it helps to distinguish between **where the rule is executed** versus **where the actual load balancers live**.

That rule is **not executed inside a regional AWS ALB**. It is executed at your **Global Traffic Gateway** (e.g., Citrix NetScaler, F5 BIG-IP, or AWS Global Accelerator) that sits *above* both regions.

The Architecture: Global Proxy vs. Regional ALBs

None



Step-by-Step Execution of the Rule

When you sent the HTTP request with the NSC_... cookie before clearing your cache, here is how the Global Gateway processed it:

1. **Interception:** The Global Gateway received your request containing the cookie NSC_... pointing to the us-east-1 backend.
2. **Target Health Verification:** Before blindly forwarding the packet to us-east-1, the Global Gateway checked its real-time health table for the target IP encoded inside the cookie.
3. **Condition Evaluated (TRUE):** It detected that us-east-1 application pods/endpoints were **UNHEALTHY** (returning errors or connection refused).
4. **Action Executed:**
    - **Ignore Cookie:** The Global Gateway stripped/ignored the stale us-east-1 cookie.
    - **Re-route Traffic:** It selected the active, healthy target pool (us-west-2 Regional ALB).
    - **Issue New Cookie:** It forwarded the request to us-west-2 and returned a fresh us-west-2 stickiness cookie back to your browser.

Why It Broke in Your Test

In your current setup, your Global Gateway was configured with **strict persistence**. Instead of evaluating whether the destination IP encoded in the cookie was actually healthy, it blindly forwarded every request carrying that cookie to us-east-1.

Because us-east-1 was down, the connection failed. Clearing your browser cookies was the only way to manually strip that cookie and force the Global Gateway to perform a fresh health lookup to us-west-2.

Adding the **Sticky Session Override Rule** at the Global Gateway layer makes this failover automatic, so users don't have to clear their cookies

TPD SOE AWS - saml2aws-vz - Verizon Integration

Created by

Timothy Prichard, last updated on Jul 17, 2026 1 minute read

vz2aws — Getting AWS Credentials via Verizon SSO

This guide walks you through authenticating to AWS using your Verizon SSO credentials and DUO MFA.

Prerequisites

- The `saml2aws-vz` binary (request saml2aws-vz from AYS - https://atyourservice.verizon.com/ays)
- Your Verizon SSO username and password
- DUO Mobile enrolled on your phone
- Corporate network access (VPN or on-net)

Step 0 — Request the saml2aws-vz application from AYS

Travel to At Your Service (https://atyourservice.verizon.com/ays), select `Request Something`, select `Desktop Software`, search for and select `saml2aws-vz`, enter the business justification (e.g. needed to access AWS Resources using SAML), select Order Now

Once Approved and Completed go to "Workspace ONE Intelligent Hub" and look for saml2aws-vz and select install.

For those who have a greater level of access on their laptop and would like to perform the install manually

- macOS Download: https://artifactory.inf.aws.vz-connect.net/artifactory/saml2aws-vz/saml2aws-vz.mac.2026.06.05-vz1.zip
- Windows Download: https://artifactory.inf.aws.vz-connect.net/artifactory/saml2aws-vz/saml2aws-vz.win.2026.06.05-vz1.zip After downloading `saml2aws-vz` to your `~/Downloads` folder:

NEW VERSIONS

- macOS Download: https://artifactory.inf.aws.vz-connect.net/artifactory/saml2aws-vz/saml2aws-vz.mac.2026.08.26-vz1.zip
- Windows Download: https://artifactory.inf.aws.vz-connect.net/artifactory/saml2aws-vz/saml2aws-vz.win.2026.08.26-vz1.zip
- Linux Download: https://artifactory.inf.aws.vz-connect.net/artifactory/saml2aws-vz/saml2aws-vz.linux.2026.08.26-vz1.zip

```
None
unzip ~/Downloads/saml2aws-vz.mac.2026.06.05-vz1.zip
cp ~/Downloads/saml2aws-vz $HOME/.local/bin/saml2aws-vz
chmod +x $HOME/.local/bin/saml2aws-vz
xattr -rd com.apple.quarantine $(which saml2aws-vz)
```

The first command installs it. The second removes macOS's quarantine flag that blocks downloaded executables from running.

> **Windows users:** Copy `saml2aws-vz.exe` to a folder in your `PATH` (e.g. `C:\Tools\`) and add that folder to your system's Path environment variable.

Step 1 — One-Time Setup

Run this once to configure the tool:

```
None
saml2aws-vz configure --session-duration=28800 --profile saml
```

Note: --session-duration=28800 extends your session time to 8 hours (our current maximum for any console role)

You will be prompted for three things:

```
None
? AWS Profile  saml      ← press Enter
? Username     _         ← type your Verizon username (e.g. jsmith)
? Meta Alias   _         ← press Enter unless your org uses a
different SSO integration (see below)
```

That's it. Setup is done.

Meta Alias — Selecting Your Org's SSO Integration (v2026.08.19-vz1+)

Different Verizon orgs have their own AWS SSO integration in ForgeRock, each identified by a distinct **Meta Alias**. The `configure` prompt defaults to the standard TPD SOE integration — **press Enter to accept it unless your platform team has told you to use a different one.**

`/employee/VZIDP_28054_JGBV_TPDSOEAWSCON`	Default
`/employee/VZIDP_28054_JGBV_TPDSOEAWSCDI`	CDI Applications
`/employee/VZIDP_28054_JGBV_TPDSOEAWSSOE`	PACT Applications

Existing profiles created before this version are unaffected — they keep using the default integration automatically, no action needed. To change your org's integration later, re-run:

```
None
saml2aws-vz configure --profile saml
```

and enter the new value at the `Meta Alias` prompt. (It's stored in `~/.saml2aws` as `meta_alias = ...` under your profile's section if you ever need to edit it by hand.)

Shell Profile (one-time, Mac only)

Add the corporate proxy to your shell startup file so it persists across all terminal sessions and applies to `aws` CLI commands as well:

```
None
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.zshrc
echo 'export HTTPS_PROXY=http://server.proxy.vzwcorp.com:9290' >>
~/.zshrc
echo 'export https_proxy=http://server.proxy.vzwcorp.com:9290' >>
~/.zshrc
source ~/.zshrc
```

Windows users: Add the following to your PowerShell profile (`notepad $PROFILE`):

```
None
$env:HTTPS_PROXY = "http://server.proxy.vzwcorp.com:9290"
```

Step 2 — Log In

```
None
saml2aws-vz login --profile saml
```

What happens:

1. You are prompted for your Verizon password
2. A DUO push is sent to your phone — **approve it**
3. An interactive role list appears — use the **arrow keys** to select the role you want and press **Enter**
4. Temporary AWS credentials are written to `~/.aws/credentials` under the `saml` profile

Successful output looks like:

```
None
Authenticating as jsmith ...
Waiting for DUO push approval...
DUO: Pushed a login request to your device...

? Please choose the role [Use arrows to move, type to filter]
> Account: 637423515472
    vz-console-soe-administrator
    vz-console-soe-developer
  Account: 123456789012
    vz-console-soe-readonly

Logged in as:
arn:aws:iam::637423515472:assumed-role/vz-console-soe-administrator/js
mith
Your new access key pair has been stored in the AWS configuration.
Note that it will expire at 2026-06-01 19:00:00 +0000 UTC
```

Step 3 — Use Your Credentials

Run AWS CLI commands using `--profile saml`:

```
None
aws --profile saml ec2 describe-instances --region us-east-2
```

Or set it as your default profile for the session:

```
None
export AWS_PROFILE=saml
```

Then run `aws` commands normally:

```
None
aws ec2 describe-instances --region us-east-2
aws s3 ls
```

Credentials Expiry

Credentials expire after **1 hour**. When they expire, re-run Step 2.

If your credentials are still valid but you want to switch roles or refresh early, add `--force`:

```
None
saml2aws-vz login --profile saml --force
```

Verify Installation

To confirm the tool is installed and check your version:

```
None
saml2aws-vz --version
```

Expected output: `2026.06.02-vz1` (or later). If you see an older date, ask your team for the latest binary.

---

saml2aws-vz — Options & Flags Reference

Since saml2aws-vz is locked to the VerizonSSO provider, some inherited flags from other IdP providers (OneLogin, Okta, Browser, Keycloak, AzureAD, F5APM) do not apply — they are accepted by the CLI (inherited from upstream saml2aws) but have zero effect at runtime. Each flag below is marked ✅ **Functional** or ❌ **No-op** for VerizonSSO, with an implementation trace through the actual code.

Quick Summary Table

`--idp-account`	✅ Yes — selects the account section
`--mfa`	✅ Yes, but constrained to `"Auto"` only
`--skip-verify`	✅ Yes — read directly in `verizon.New()`
`--url`	✅ Yes — required, drives every request
`--username` / `--password`	✅ Yes
`--role`	✅ Yes (generic role-resolution logic)
`--policyfile` / `--policyarns`	✅ Yes (generic AssumeRoleWithSAML logic)
`--aws-urn`	❌ No-op — Verizon hardcodes `urn:amazon:webservices`
`--skip-prompt`	✅ Yes (generic)
`--session-duration`	✅ Yes (generic, with a caveat — see below)
`--disable-keychain`	✅ Yes (generic; **defaults to** `true` in this fork)

`--region`	✅ Yes (generic STS call + saved credentials region)
`--verbose` / `--quiet`	✅ Yes — Verizon's logger uses shared logrus
`--config`	✅ Yes (generic)
`--prompter`	✅ Yes — used for DUO passcode/menu prompts
`--kc-broker`	❌ No-op — Keycloak-only
`--idp-provider`	✅ Yes — selects Verizon in the provider switch
`--browser-type` / `--browser-executable-path` / `--browser-autofill`	❌ No-op — Browser-provider only
`--duo-mfa-option`	✅ Yes — directly read by `resolveDuoOption()`
`--force`	✅ Yes (generic)
`--credential-process`	✅ Yes (generic)
`--try-no-prompt`	✅ Yes for username/password stage; does not suppress DUO prompts
`--credentials-file`	✅ Yes (generic)
`--cache-saml` / `--cache-file`	✅ Yes (generic; especially useful for skipping repeat DUO pushes)
`--client-id` / `--client-secret` / `--mfa-ip-address`	❌ No-op — OneLogin-only

`--download-browser-driver`	❌ No-op — Browser-provider only
`--disable-sessions` / `--disable-remember-device`	❌ No-op — Okta-only
`--app-id`	❌ No-op — OneLogin/AzureAD-only
`--subdomain`	❌ No-op — OneLogin-only
`--resource-id`	❌ No-op — F5APM-only
`--profile`	✅ Yes — fully generic
`--exec-profile`	✅ Yes — fully generic (exec/console only)
`--link`	✅ Yes — fully generic (console only)
`--shell`	✅ Yes — fully generic (script only)

Architecture Notes (read this first)

- `cmd/saml2aws/main.go` defines all kingpin flags, binding them to `flags.CommonFlags` (shared) or `flags.LoginExecFlags` (login/exec-specific).
- `flags.ApplyFlagOverrides()` (`pkg/flags/flags.go`) copies non-empty `CommonFlags` values onto the loaded `cfg.IDPAccount` from `~/.saml2aws`. **Any flag not wired into this function only exists in memory for that single run** — it's never persisted.
- `resolveLoginDetails()` (`cmd/saml2aws/commands/login.go`) builds the `creds.LoginDetails` struct passed into `provider.Authenticate(loginDetails)`. Only fields that exist on that struct are visible to the Verizon provider code at authentication time.
- The Verizon provider lives at `pkg/provider/verizon/verizon.go`, registered under the name `"VerizonSSO"`.

Commands

`configure`	Set up a new profile (AWS profile + username)
`login`	Authenticate and get temporary AWS credentials
`exec`	Run a command with AWS credentials injected as env vars
`console`	Open the AWS web console after logging in
`list-roles`	List available role ARNs without a full interactive login
`script`	Emit shell commands to export AWS credential env vars

Global Flags

`--idp-account` / `-a`

Env: `SAML2AWS_IDP_ACCOUNT` · **Default:** **`default`**

Selects which `[section]` of `~/.saml2aws` to load via `LoadIDPAccount()`. This is the account-selection key for **every** command, not just login. If the named section doesn't exist, no error is raised immediately — an all-default `IDPAccount` is returned instead, which then fails later (and less obviously) at **`account.Validate()`** with something like "URL empty in idp account" rather than "account not found."

`--mfa`

Env: `SAML2AWS_MFA`

For VerizonSSO the *only* accepted value is `"Auto"` — anything else is rejected before authentication even starts (`invalidMFA` check against `MFAsByProvider["VerizonSSO"]`). Interestingly, `"Auto"` itself doesn't match any case in **`resolveDuoOption()`**'s switch, so it falls through to the **interactive** DUO push-vs-passcode prompt at runtime. In practice, non-interactive MFA selection requires `--duo-mfa-option` instead.

`--skip-verify` / `-s`

Env: `SAML2AWS_SKIP_VERIFY`

Disables TLS certificate verification. Mutates the process-wide `http.DefaultTransport` in `main.go` (affecting *any* code using `http.DefaultClient`, including the `console` command's federation call to `signin.aws.amazon.com`), **and** is read a second time directly inside `verizon.New()` to build Verizon's own dedicated transport. Once set to `true` in the account config file, it cannot be forced back to `false` purely via omitting the flag on a later run — the account file's `true` sticks.

`--url`

Env: `SAML2AWS_URL`

The IdP base URL. Required and URL-validated. Feeds `loginDetails.URL`, which becomes `baseURL` for every ForgeRock/DUO request the Verizon client makes. Also the lookup key for the OS keychain credential entry (irrelevant here since keychain is disabled by default — see `--disable-keychain`).

`--username` / `--password`

Env: `SAML2AWS_USERNAME` / `SAML2AWS_PASSWORD`

Feed `loginDetails.Username`/`Password`, sent to ForgeRock as `IDToken1`/`IDToken2` in `submitCredentials()`. Precedence order: account-file value → keychain lookup (skipped by default) → flag/env value → interactive prompt. Password is never persisted to `~/.saml2aws` (no `Password` field exists on `cfg.IDPAccount`) — it only goes to the OS keychain, and only if `--disable-keychain=false` is explicitly passed.

`--role`

Env: `SAML2AWS_ROLE`

Pre-selects a role ARN, skipping the interactive picker. If multiple roles are asserted and this is set, HTML-scraping/interactive prompting is bypassed entirely and `LocateRole()` is called directly — it errors with no fuzzy matching if the ARN doesn't exactly match one of the roles asserted for that user. This is the key lever for non-interactive automation with multiple available roles.

`--policyfile` / `--policyarns`

Env: `SAML2AWS_POLICY_FILE` / `SAML2AWS_POLICY_ARNS`

Supplemental IAM session policy (inline JSON file, or comma-separated managed policy ARNs) passed to the `AssumeRoleWithSAML` call. Both can be combined (AWS allows inline + managed simultaneously). **Caveat:** neither applies to the secondary role assumption triggered by `--exec-profile` — only the first SSO-derived role assumption is constrained.

`--aws-urn`

Env: `SAML2AWS_AWS_URN`

❌ **No-op for VerizonSSO.** This flag is used by other providers (ADFS, PingFed, Shibboleth) to build their SP-initiated SSO request URL. The Verizon provider hardcodes `vzSpEntityID = "urn:amazon:webservices"` as a package constant — this flag has zero effect on Verizon logins even though the CLI accepts it.

`--skip-prompt`

No env var.

All-or-nothing bypass of interactive prompting for login credentials. If username/password end up empty (no flag, no keychain hit), authentication fails immediately with an empty-credential validation error rather than prompting. Implicitly enabled by `--credential-process` even without passing it explicitly. Also used by `configure` to skip the interactive setup wizard entirely.

`--session-duration`

Env: `SAML2AWS_SESSION_DURATION` · **Default:** `3600` (1 hour)

Sets `DurationSeconds` on the `AssumeRoleWithSAML` call.

Gotcha: there are two independent consumption paths using two different sources of truth. The primary SAML-based login uses **`account.SessionDuration`** (config-file value applies if the flag/env is unset). The `--exec-profile` secondary/chained assumption instead uses the *raw* flag value directly, not the account-merged value — so if you only set `aws_session_duration` in `~/.saml2aws` (not via flag/env), the primary login honors it but a chained `--exec-profile` assumption silently falls back to the AWS SDK default (~1 hour) instead.

`--disable-keychain`

Env: `SAML2AWS_DISABLE_KEYCHAIN` · **Default:** `true` ⚠️

This fork inverts upstream saml2aws behavior — the OS keychain is disabled by default and must be explicitly re-enabled with `--disable-keychain=false`. Out of the box, this tool never reads or

writes the OS keychain; every run needs `--username`/`--password` flags, env vars, or interactive prompting. This was a deliberate corporate-fork choice.

`--region` / `-r`

Env: `SAML2AWS_REGION`

Sets the STS API endpoint region for the `AssumeRoleWithSAML` call (relevant for GovCloud/China partitions), and is written as the `region=` line in the saved `~/.aws/credentials` profile entry. **Does not** set `AWS_REGION`/`AWS_DEFAULT_REGION` env vars for `exec`/`console`/`script`'s spawned child process — a downstream `aws` CLI call still needs its own region resolution unless it reads the `region` key from the credentials file directly (recent AWS CLI versions support this).

`--verbose`

No env var.

Raises the global logrus level to Debug (surfacing every `logger.Debug(...)` call across the codebase, including Verizon's own `"auth chain initiated"`, `"DUO host: ..."`, and `"SAMLResponse prefix: ..."` debug lines), and switches the final error format from `%v` to `%+v` (full wrapped error chain via `pkg/errors`). All logging goes to stderr (stdout on Windows), so it's safe to pipe stdout output (e.g. from `script`) while still seeing debug noise separately.

`--quiet`

No env var.

Redirects both the standard `log` package and logrus output to `io.Discard` — silences *everything*, including the final top-level error print on failure. A failing `login --quiet` exits non-zero with **no stderr output at all**. Does not affect `fmt.Println` calls used for actual program output (role listings, credential-process JSON, the `--link` URL) — only logging is silenced. Combining with `--verbose` effectively gives `--quiet` the win, since discarded output doesn't matter regardless of level.

`--config`

Env: `SAML2AWS_CONFIGFILE` · **Default:** `~/.saml2aws`

Path to the account config file. Loaded with `Loose: true`, meaning a missing/bad path is not an error — it silently proceeds with an empty config, then fails later (less obviously) at `account.Validate()`.

`--prompter`

No env var.

Selects the interactive-input mechanism. Empty/`"survey"`/`"default"` → normal CLI prompter. `pinentry`/`pinentry-mac`/`pinentry-gnome3`/`pinentry-tty` (matches regex `^pinentry(-.*)?$`) → shells out to that pinentry binary for secure input. Any other value → hard validation error that fails the command immediately, even for commands that wouldn't otherwise need to prompt. Verizon's DUO flow uses this for both the passcode prompt and the push-vs-passcode selection menu.

`--kc-broker`

No env var.

❌ **No-op for VerizonSSO.** Keycloak-specific escape hatch for federated identity-broker setups (`KCFederationID := "#social-" + broker`). Never referenced anywhere in `verizon.go`.

`--idp-provider`

Env: `SAML2AWS_IDP_PROVIDER`

Selects which provider package handles authentication — for this fork, effectively always `VerizonSSO`. Note the interactive `configure` flow hardcodes `Provider = "VerizonSSO"` regardless of this flag, but a **non-interactive** `configure --skip-prompt --idp-provider=X` could theoretically write a different provider into the account file (since the hardcoding only happens in the interactive prompt path).

`--browser-type` / `--browser-executable-path` / `--browser-autofill`

Env: `SAML2AWS_BROWSER_TYPE` / `SAML2AWS_BROWSER_EXECUTABLE_PATH` / `SAML2AWS_BROWSER_AUTOFILL`

❌ **No-op for VerizonSSO.** Only consumed by `pkg/provider/browser/browser.go` (Playwright channel selection, custom executable path, and login-page autofill respectively). Never referenced in `verizon.go` — Verizon uses plain `net/http`, no browser automation at all.

`login`-specific Flags

`--profile` / `-p`

Env: `SAML2AWS_PROFILE` · **Default:** falls back to `"saml"` or the account's stored `aws_profile`

Determines which `[profile-name]` section of `~/.aws/credentials` is written to/read from. Behaves identically whether set on **`login`**, `exec`, `console`, or **`script`** — it's the same underlying mechanism (`ApplyFlagOverrides` → **`account.Profile`**) in every case.

`--duo-mfa-option`

Env: `SAML2AWS_DUO_MFA_OPTION` · **Enum:** `"Passcode"` or `"Duo Push"` (strict — kingpin rejects anything else)

Setting `"Duo Push"` sends a push notification and polls `/frame/status` every 3 seconds until approved. Setting `"Passcode"` prompts for (or uses `--mfa-token`, if set) a 6-digit DUO passcode instead. Without this flag resolving to a known value, the interactive push-vs-passcode menu appears — breaking non-interactive automation.

`--force`

No env var.

Bypasses the "credentials not expired, skip re-auth" shortcut in `Login()`. Without it, if the cached `~/.aws/credentials` entry for the profile isn't expired, the entire authentication flow (ForgeRock, DUO, STS) is skipped entirely and the command returns immediately (optionally re-printing cached creds if `--credential-process` is also set). This check is purely local/offline — it only looks at the cached expiry timestamp, not whether the underlying IdP session is still valid.

`--credential-process`

No env var.

Outputs credentials as `credential_process`-spec JSON to stdout (for AWS SDK/CLI's native `credential_process` integration) instead of the normal human-readable log messages. Implicitly enables `--skip-prompt` behavior. If using the *default* credentials file path, credentials are **not** also written to disk — only emitted to stdout, meaning every AWS API call that triggers this process will re-hit Verizon SSO/DUO unless combined with **`--try-no-prompt`** and a way to satisfy the DUO prompt. If a **custom** `--credentials-file` path is set, credentials are additionally persisted there to allow auto-refresh.

`--try-no-prompt`

No env var.

Skips the username/password interactive prompt *only if* both are already available (from keychain lookup or flags). **Does not suppress DUO/MFA prompts** — if `--duo-mfa-option` can't be resolved, you'll still get the interactive MFA-type menu, and if it resolves to Passcode with no `--mfa-token` supplied, you'll still get an interactive passcode prompt. Since `--disable-keychain` defaults to `true` in this fork, this flag is effectively a no-op unless you also supply `--username`/`--password` explicitly or re-enable the keychain.

`--credentials-file`

Env: `SAML2AWS_CREDENTIALS_FILE` · **Default:** `~/.aws/credentials`

Custom path for the AWS credentials output file. Also used to decide (via exact string comparison against the default path) whether `--credential-process` mode additionally persists credentials to disk. Auto-creates the file/parent directory (`0600` perms) with a stub `[profile]` section if it doesn't exist.

`--cache-saml`

Env: `SAML2AWS_CACHE_SAML`

Caches the raw base64 **SAML assertion** (not the AWS STS credentials) to skip the entire IdP round-trip — including the DUO push/passcode prompt — on subsequent runs, as long as the assertion's own expiry (`SubjectConfirmationData/@NotOnOrAfter` in the SAML XML) hasn't passed. AWS's `AssumeRoleWithSAML` STS call still happens on every run regardless. This is the most useful flag for reducing repeat DUO pushes when running `login`/`list-roles` multiple times in quick succession. If the cache file is missing/corrupt, it fails silently (just falls through to a real login) rather than erroring.

`--cache-file`

Env: `SAML2AWS_SAML_CACHE_FILE` · **Default:** `~/.aws/saml2aws/cache[_<account>]`

Custom path for the SAML assertion cache. Only has any effect if `--cache-saml` is also enabled. File is written with `0600` permissions (parent dir `0700`) since it contains a sensitive, replayable bearer credential.

`--client-id` / `--client-secret` / `--mfa-ip-address` (login context)

Env: `ONELOGIN_CLIENT_ID` / `ONELOGIN_CLIENT_SECRET` / `ONELOGIN_MFA_IP_ADDRESS`

❌ **No-op for VerizonSSO.** OneLogin-only — used for OAuth2 basic auth (`client-id`/`client-secret`) and IP-based MFA allow-listing (`mfa-ip-address`). Never referenced in `verizon.go`.

`--download-browser-driver`

Env: `SAML2AWS_AUTO_BROWSER_DOWNLOAD`

❌ **No-op for VerizonSSO.** Triggers a Playwright browser binary download for the `Browser` provider only.

`--disable-sessions` / `--disable-remember-device`

Env: `SAML2AWS_OKTA_DISABLE_SESSIONS` / `SAML2AWS_OKTA_DISABLE_REMEMBER_DEVICE`

❌ **No-op for VerizonSSO.** Okta-only session-cookie/remember-device controls. Note: when `--disable-keychain` is at its (default) `true` value and `account.Provider` happens to be `"okta"`, these get force-enabled as a side effect in `resolveLoginDetails()` — but this side effect is itself gated on the provider being Okta, so it's irrelevant here.

`configure`-specific Flags

`--app-id`

Env: `ONELOGIN_APP_ID`

❌ **No-op for VerizonSSO.** Required only for OneLogin/AzureAD providers.

`--subdomain`

Env: `ONELOGIN_SUBDOMAIN`

❌ **No-op for VerizonSSO.** OneLogin-only.

`--resource-id`

Env: `SAML2AWS_F5APM_RESOURCE_ID`

❌ **No-op for VerizonSSO.** F5APM-only.

`--profile` (configure context)

Env: `SAML2AWS_PROFILE`

Same field/mechanism as the login-context `--profile` — determines the AWS credentials profile name that gets written into the saved `~/.saml2aws` account definition (default `"saml"`).

Note on `configure` overall: running with `--skip-prompt` bypasses the interactive setup wizard (`PromptForConfigurationDetails`) entirely — including the hardcoded `Provider = "VerizonSSO"` assignment that path performs. Also, **`storeCredentials()`** will only persist a password to the OS keychain if `--disable-keychain=false` is explicitly passed (default is keychain disabled).

`exec`-specific Flags

`--profile` / `-p`

Env: `SAML2AWS_PROFILE`

Same generic profile-selection mechanism described above — determines which credentials are read for the command being executed.

`--exec-profile`

Env: `SAML2AWS_EXEC_PROFILE`

Chains a **second** role assumption on top of the credentials already obtained via SSO. Reads a separate profile from `~/.aws/config` (with `role_arn` + `source_profile` pointing back at your SSO profile) and performs an internal `AssumeRole` using that config, then runs the command with the final chained-role credentials. Example:

```
None
[profile deploy-role]
role_arn = arn:aws:iam::123456789012:role/vz-deploy-role
source_profile = saml
region = us-east-2
```

```
None
saml2aws-vz exec --profile saml --exec-profile deploy-role -- aws s3
ls
```

Caveats: `--policyfile`/`--policyarns` do not apply to this secondary assumption. `--session-duration` also behaves differently here — see the `--session-duration` entry above. When `--exec-profile` is used, `AWS_PROFILE`/`AWS_DEFAULT_PROFILE` env vars are deliberately *not* set for the spawned child process (only the raw access key/secret/ token/expiration vars are), so downstream tools fall back to whatever `AWS_PROFILE` they already had in the environment, if any.

`--credentials-file`

Env: `SAML2AWS_CREDENTIALS_FILE`

Same generic mechanism as in the login context.

`command` (positional argument)

The command to execute, e.g. `saml2aws-vz exec --profile saml -- aws s3 ls`. Everything after the recognized flags is greedily captured as a `[]string` — on macOS/Linux this is executed directly via `os/exec` with **no shell interpretation** (no pipes, no globbing, no &&). To run a shell pipeline, explicitly invoke a shell: `saml2aws-vz exec -- bash -c "aws s3 ls | grep foo"`. On Windows, the whole command line is instead prefixed with `cmd /C`, so `cmd.exe` operators *are* available there.

`Exec()` only triggers a fresh interactive login if a live STS `GetCallerIdentity` probe against the cached credentials fails — otherwise it's fast and silent, reusing existing credentials.

`console`-specific Flags

`--exec-profile`

Same mechanism as the `exec` command's flag — applies before the AWS console federation call.

`--profile` / `-p`

Same generic mechanism.

`--force`

No env var.

Behaves differently here than in `login`. In `console`, `--force` is checked *first* and unconditionally triggers a full re-login via `loginRefreshCredentials()`, regardless of whether cached credentials exist or are expired. Without `--force`, `console` does *more* diligence than `login` without `--force` — it actively probes AWS with a live `sts:GetCallerIdentity` call (not just checking the local cached expiry timestamp) before deciding whether a re-login is actually needed.

`--link`

No env var.

Prints the signed AWS Console federation URL to stdout instead of launching a browser (`open.Run()` on macOS/Linux/Windows via the OS's native "open" mechanism). Useful for headless/SSH sessions, or to pipe the URL elsewhere. The printed URL contains a one-time, short-lived `SigninToken` — treat it as a live authenticated bearer credential.

`--credentials-file`

Same generic mechanism.

`script`-specific Flags

`--profile` / `-p`

Same generic mechanism — determines which cached credentials to format for output. Note `script` never triggers a fresh login itself; if credentials don't exist or are expired, it errors out and expects you to have already run `login`/`exec` first.

`--shell`

Default: `bash` · **Enum:** `bash`, `/bin/sh`, `powershell`, `fish`, `env`

Selects the output template for exporting AWS credential environment variables to stdout, intended for `eval $(saml2aws-vz script --shell bash)`-style usage:

`bash`	`export VAR=value`	Unquoted values
`/bin/sh`	`export VAR=value`	Byte-for-byte identical template to `bash` — no functional difference currently
`fish`	`set -gx VAR value`	Expiration value is single-quoted
`powershell`	`$env:VAR='value'`	All values single-quoted
`env`	`VAR=value`	No `export`/`set` keyword; **omits the expiration variable entirely** — meant for `docker run --env-file`-style consumption

All five templates emit `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_SESSION_TOKEN`, `AWS_SECURITY_TOKEN`, and `SAML2AWS_PROFILE`; four of the five also emit `AWS_CREDENTIAL_EXPIRATION` (RFC3339-formatted) — `env` is the exception.

`--credentials-file`

Same generic mechanism.

`list-roles`-specific Flags

`--cache-saml` / `--cache-file`

Same generic caching mechanism described under the `login` section — `list-roles` uses the identical `SAMLCacheProvider` read/write pattern, and shares the same cache file with `login` if both are run against the same `--idp-account` with caching enabled. This is useful for listing roles repeatedly without repeatedly triggering DUO pushes.

Useful Combo — Fully Non-Interactive DUO Push Login

```
None
saml2aws-vz login --profile saml --duo-mfa-option "Duo Push"
--skip-prompt
```

This logs in, forces a DUO Push (no interactive MFA-type prompt), and — if `--role` and `--username` are also set via env or flags (and `--disable-keychain=false` with a previously-stored password, or `--password` is supplied) — requires zero interaction at all except approving the push on your phone.

Useful Combo — Skip Repeat DUO Pushes During a Work Session

```
None
saml2aws-vz login --profile saml --cache-saml --duo-mfa-option "Duo
Push"
```

The first run performs a full DUO push. Subsequent runs within the SAML assertion's validity window reuse the cached assertion and skip straight to the STS `AssumeRoleWithSAML` call — no repeat DUO push required.

Troubleshooting

No DUO prompt, exits immediately	Credentials still valid — add `--force` to re-authenticate
`connection reset by peer`	Not on corporate network — connect to VPN first
`DUO authentication rejected`	DUO push was denied — run the command again
`error completing DUO MFA: DUO host not found in TextOutputCallback script`	`bash rm -rf ~/.aws/saml2aws`, then run Step 1 and Step 2 again. If still an issue, try password reset and reboot - ** fixed in version 2026.08.26-vz1

Credentials expired	Re-run the login command (Step 2)
Wrong password	Just re-run the login command — password is always prompted
Issuer not present in specified provider	Make sure to use the latest version of the tool, and to use the correct Meta Alias for the desired Account

Local System Settings

Configure saml2aws for short live AWS access keys

https://confluence.verizon.com/spaces/SOEEXTKB/pages/772770820/TPD+SOE+AWS+-+saml2aws-vz+-+Verizon+Integration

https://confluence.verizon.com/spaces/SOEEXTKB/pages/772777716/TPD+SOE+AWS+-+saml2aws-vz+Troubleshooting

```
None
Path of .credentials file:    C:\Users\srivsa\.aws
Path of saml exe file:        C:\Tools
Path of .saml2aws file:       C:\Users\srivsa
Path of .bashrc:              C:\Users\srivsa
Path of .bash_profile:        C:\Users\srivsa
Path of .sso_pass:            C:\Users\srivsa
```

Windows system configuration steps:

13. download saml2aws
14. Copy saml2aws-vz.exe to a folder in your PATH (e.g. C:\Tools\) and
15. add the saml2aws folder to your system's Path environment variable

```
Shell
[Environment]::SetEnvironmentVariable("Path", $env:Path +
";C:\tools\saml2aws-vz.win.2026.08.26-vz1", "User")
```

16. verify that env variable is set correctly

```
Shell
Get-Command saml2aws-vz -ErrorAction SilentlyContinue |
Select-Object Name, Source
```

sample o/p:

```
Shell

Name                           Source

----                                ------

saml2aws-vz.exe
C:\Tools\saml2aws-vz.win.2026.08.26-vz1\saml2aws-vz.exe
```

17. Create the Profile File

```
Shell
New-Item -Path $PROFILE -ItemType File -Force
```

18. Open the Profile in Notepad

```
Shell
notepad $PROFILE
```

19. Add the following to your PowerShell profile (copy & paste, save)

```
Shell
$env:HTTPS_PROXY = "http://server.proxy.vzwcorp.com:9290"
```

20. Run this once to configure the tool:

```
Shell
saml2aws-vz configure --idp-account osnd-dev   --session-duration=28800
--profile osnd-dev

saml2aws-vz configure --idp-account osnd-test  --session-duration=28800
--profile osnd-test

saml2aws-vz configure --idp-account osnd-preprod --session-duration=28800
--profile osnd-preprod

saml2aws-vz configure --idp-account osnd-prod  --session-duration=28800
--profile osnd-prod
```

Note: --session-duration=28800 extends your session time to 8 hours (our current maximum for any console role)

You will be prompted for three things:

```
Shell

? AWS Profile  saml      ← press Enter

? Username     _         ← type your Verizon username (e.g. jsmith)

? Meta Alias   _         ← press Enter unless your org uses a
different SSO integration (see below)
```

That's it. Setup is done.

Meta Alias — Selecting Your Org's SSO Integration (v2026.08.19-vz1+)

Different Verizon orgs have their own AWS SSO integration in ForgeRock, each identified by a distinct **Meta Alias**. The `configure` prompt defaults to the standard TPD SOE integration — **press Enter to accept it unless your platform team has told you to use a different one.**

Meta Alias	Used By
/employee/VZIDP_28054_JGBV_TPDSOEAWSSOE	PACT Applications
/employee/VZIDP_28054_JGBV_TPDSOEAWSCON	Default
/employee/VZIDP_28054_JGBV_TPDSOEAWSCDI	CDI Applications

Existing profiles created before this version are unaffected — they keep using the default integration automatically, no action needed. To change your org's integration later, re-run:

```
None

saml2aws-vz configure --profile saml
```

and enter the new value at the Meta **Alias** prompt. (It's stored in `~/.saml2aws` as `meta_alias = ...` under your profile's section if you ever need to edit it by hand.)

21. Open saml2aws file and update meta alias to pact applications

```
Shell
notepad $HOME\.saml2aws
```

Copy and paste “/employee/VZIDP_28054_JGBV_TPDSOEAWSSOE” for “meta_alias”.
Save and exit.

22. Login to individual profile. **Make sure you are connected to TAMPA gateway VPN** (India gateways don’t work)

```
Shell
saml2aws-vz login --profile osnd-dev
saml2aws-vz login --profile osnd-test
saml2aws-vz login --profile osnd-preprod
saml2aws-vz login --profile osnd-prod
```

OR

```
Shell
saml2aws-vz login --idp-account osnd-dev --profile osnd-dev
saml2aws-vz login --idp-account osnd-test --profile osnd-test
saml2aws-vz login --idp-account osnd-preprod --profile osnd-preprod
saml2aws-vz login --idp-account osnd-prod --profile osnd-prod
```

For Force login(rotate credentials forcefully):

```
Shell
saml2aws-vz login --profile osnd-dev --force
OR
saml2aws-vz login --idp-account osnd-dev --profile osnd-dev --force
```

23. verify your current AWS authentication state and identity for the `osnd-dev` profile while explicitly bypassing SSL certificate verification

```
Shell
aws sts get-caller-identity --no-verify-ssl --profile osnd-dev
```

24. List s3 buckets while explicitly bypassing SSL certificate verification

```
Shell
aws s3 ls --no-verify-ssl --profile osnd-dev
```

Automate Login process:

Instead of entering login profile cmd, system credentials, DUO passcode and selecting the required role for each and every aws profile, I have automated the process as below. Now I just have to run the **aws-login-all** cmd which invokes a method/script stored in .bashrc file. This script will read the system credentials from the .sso_pass file. I have to enter the DUO passcode manually. Saml2aws.exe will automatically store the generated access keys in AWS .credentials file.

```shell
aws-login-all
or
aws-login-all --force
```

Sample Execution:

```shell
SRIVSA@VIN0019ALKKW6E6 MINGW64 ~
$ aws-login-all --force
Force flag detected: Re-authenticating all environments...
Enter 6-digit DUO Passcode: <enter DUO passcode>

----------------------------------------
Authenticating: osnd-dev
----------------------------------------
Using IdP Account osnd-dev to access VerizonSSO https://ssologin.verizon.com
Authenticating as srivsa ...
Selected role: arn:aws:iam::520949192567:role/vz-console-soe-administrator
Requesting AWS credentials using SAML assertion.
Logged in as:
arn:aws:sts::520949192567:assumed-role/vz-console-soe-administrator/sachin.s@one.v
erizon.com

Your new access key pair has been stored in the AWS configuration.
Note that it will expire at 2026-10-17 05:39:08 +0530 IST
To use this credential, call the AWS CLI with the --profile option (e.g. aws
--profile osnd-dev ec2 describe-instances).
----------------------------------------
Authenticating: osnd-test
----------------------------------------
Using IdP Account osnd-test to access VerizonSSO https://ssologin.verizon.com
Selected role: arn:aws:iam::157520333689:role/vz-console-soe-administrator
```

```
Requesting AWS credentials using SAML assertion.
Logged in as:
arn:aws:sts::157520333689:assumed-role/vz-console-soe-administrator/sachin.s@one.v
erizon.com

Your new access key pair has been stored in the AWS configuration.
Note that it will expire at 2026-10-17 05:39:11 +0530 IST
To use this credential, call the AWS CLI with the --profile option (e.g. aws
--profile osnd-test ec2 describe-instances).
----------------------------------------
Authenticating: osnd-preprod
----------------------------------------
Using IdP Account osnd-preprod to access VerizonSSO https://ssologin.verizon.com
Selected role: arn:aws:iam::837767801504:role/vz-console-soe-administrator
Requesting AWS credentials using SAML assertion.
Logged in as:
arn:aws:sts::837767801504:assumed-role/vz-console-soe-administrator/sachin.s@one.v
erizon.com

Your new access key pair has been stored in the AWS configuration.
Note that it will expire at 2026-10-17 05:39:14 +0530 IST
To use this credential, call the AWS CLI with the --profile option (e.g. aws
--profile osnd-preprod ec2 describe-instances).
----------------------------------------
Authenticating: osnd-prod
----------------------------------------
Using IdP Account osnd-prod to access VerizonSSO https://ssologin.verizon.com
Selected role: arn:aws:iam::282329364436:role/vz-console-soe-administrator
Requesting AWS credentials using SAML assertion.
Logged in as:
arn:aws:sts::282329364436:assumed-role/vz-console-soe-administrator/sachin.s@one.v
erizon.com

Your new access key pair has been stored in the AWS configuration.
Note that it will expire at 2026-10-17 05:39:16 +0530 IST
To use this credential, call the AWS CLI with the --profile option (e.g. aws
--profile osnd-prod ec2 describe-instances).
----------------------------------------
All 4 profiles processed!
```

.sso_pass file contents

<entered my system login pwd here>

.bashrc file contents

```
export PATH=$PATH:/c/Tools
export PATH="HOME/.local/bin:PATH"
export HTTPS_PROXY=http://server.proxy.vzwcorp.com:9290
export https_proxy=http://server.proxy.vzwcorp.com:9290
export AWS_PROFILE=saml
export PATH="$PATH:C:\Program Files\Amazon\AWSCLIV2"
export SAML2AWS_DISABLE_KEYCHAIN=false
export SAML2AWS_PASSWORD="Verizon*0"
alias saml2aws-vz='winpty saml2aws-vz'


aws-login-all() {
  local CACHE_PATH="$HOME/.aws/saml2aws/shared_vz_saml_cache"
  local ENVS=("osnd-dev" "osnd-test" "osnd-preprod" "osnd-prod")
  local FORCE_FLAG=""

  if [[ "1" == " - f"||"1" == "--force" ]]; then
    FORCE_FLAG="--force"
  fi

  # Read password from restricted file dynamically
  if [[ -f ~/.sso_pass ]]; then
    export SAML2AWS_PASSWORD=$(cat ~/.sso_pass)
  else
    echo "Error: Password file ~/.sso_pass not found."
    return 1
  fi

  read -p "Enter 6-digit DUO Passcode: " DUO_CODE
  echo ""

  for env in "${ENVS[@]}"; do
    echo "Authenticating: $env"
    \saml2aws-vz login \
      --idp-account "$env" \
      --profile "$env" \
      --duo-mfa-option "Passcode" \
      --mfa-token "$DUO_CODE" \
      --cache-saml \
      --cache-file "$CACHE_PATH" \
      --skip-prompt \
      --disable-keychain \
      $FORCE_FLAG
  done

  # Wipe from memory after authentication
  unset SAML2AWS_PASSWORD
}
```

If using Windows Powershell (PROFILE file) instead of git bash (.bashrc file)

```
# Environment & PATH Variables
$env:PATH += ";C:\Tools"
env: PATH = "env:USERPROFILE\.local\bin;$env:PATH"
$env:PATH += ";C:\Program Files\Amazon\AWSCLIV2"

# Proxy Configuration
$env:HTTPS_PROXY = "http://server.proxy.vzwcorp.com:9290"
$env:https_proxy = "http://server.proxy.vzwcorp.com:9290"

# AWS & saml2aws defaults
$env:AWS_PROFILE = "saml"
$env:SAML2AWS_DISABLE_KEYCHAIN = "false"

# Function to authenticate all accounts
function aws-login-all {
    param(
        [Alias("f")]
        [switch]$Force
    )

    cachePath = "env:USERPROFILE\.aws\saml2aws\shared_vz_saml_cache"
    passFile = "env:USERPROFILE\.sso_pass"
    $envs      = @("osnd-dev", "osnd-test", "osnd-preprod", "osnd-prod")

    # Read password dynamically
    if (Test-Path $passFile) {
        $env:SAML2AWS_PASSWORD = (Get-Content $passFile -Raw).Trim()
    } else {
        Write-Error "Password file $passFile not found."
        return
    }

    # Prompt for Duo passcode
    $duoCode = Read-Host "Enter 6-digit DUO Passcode"

    foreach ($targetEnv in $envs) {
        Write-Host "Authenticating: $targetEnv" -ForegroundColor Cyan

        $params = @(
            "login",
            "--idp-account", $targetEnv,
            "--profile", $targetEnv,
            "--duo-mfa-option", "Passcode",
            "--mfa-token", $duoCode,
            "--cache-saml",
            "--cache-file", $cachePath,
            "--skip-prompt",
```

```
        "--disable-keychain"
    )

    if ($Force) {
        $params += "--force"
    }

    # Run saml2aws-vz (winpty is not needed in native PowerShell)
    & saml2aws-vz @params
  }

  # Wipe password from session memory
  Remove-Item Env:\SAML2AWS_PASSWORD
}
```

.saml2aws file contents

```
[default]
name                  = default
app_id                =
url                   = https://ssologin.verizon.com
username              = srivsa
provider              = VerizonSSO
mfa                   = Auto
mfa_ip_address        =
skip_verify           = false
proxy                 = http://server.proxy.vzwcorp.com:9290
timeout               = 0
aws_urn               = urn:amazon:webservices
aws_session_duration  = 28800
aws_profile           = osnd
resource_id           =
subdomain             =
role_arn              =
policy_file           =
policy_arn_list       =
region                =
http_attempts_count   =
http_retry_delay      =
credentials_file      =
saml_cache            = false
saml_cache_file       =
target_url            =
disable_remember_device = false
disable_sessions      = false
download_browser_driver = false
headless              = false
prompter              =
```

```
kc_broker             =
meta_alias            = /employee/VZIDP_28054_JGBV_TPDSOEAWSSOE

[osnd-dev]
name                  = osnd-dev
app_id                =
url                   = https://ssologin.verizon.com
username              = srivsa
provider              = VerizonSSO
mfa                   = Auto
mfa_ip_address        =
skip_verify           = false
proxy                 = http://server.proxy.vzwcorp.com:9290
timeout               = 0
aws_urn               = urn:amazon:webservices
aws_session_duration  = 28800
aws_profile           = osnd-dev
resource_id           =
subdomain             =
role_arn              = arn:aws:iam::520949192567:role/vz-console-soe-administrator
policy_file           =
policy_arn_list       =
region                =
http_attempts_count   =
http_retry_delay      =
credentials_file      =
saml_cache            = false
saml_cache_file       =
target_url            =
disable_remember_device = false
disable_sessions      = false
download_browser_driver = false
headless              = false
prompter              =
kc_broker             =
meta_alias            = /employee/VZIDP_28054_JGBV_TPDSOEAWSSOE

[osnd-test]
name                  = osnd-test
app_id                =
url                   = https://ssologin.verizon.com
username              = srivsa
provider              = VerizonSSO
mfa                   = Auto
mfa_ip_address        =
skip_verify           = false
proxy                 = http://server.proxy.vzwcorp.com:9290
timeout               = 0
aws_urn               = urn:amazon:webservices
aws_session_duration  = 28800
```

```
aws_profile            = osnd-test
resource_id            =
subdomain              =
role_arn               = arn:aws:iam::157520333689:role/vz-console-soe-administrator
policy_file            =
policy_arn_list        =
region                 =
http_attempts_count    =
http_retry_delay       =
credentials_file       =
saml_cache             = false
saml_cache_file        =
target_url             =
disable_remember_device = false
disable_sessions       = false
download_browser_driver = false
headless               = false
prompter               =
kc_broker              =
meta_alias             = /employee/VZIDP_28054_JGBV_TPDSOEAWSSOE

[osnd-preprod]
name                   = osnd-preprod
app_id                 =
url                    = https://ssologin.verizon.com
username               = srivsa
provider               = VerizonSSO
mfa                    = Auto
mfa_ip_address         =
skip_verify            = false
proxy                  = http://server.proxy.vzwcorp.com:9290
timeout                = 0
aws_urn                = urn:amazon:webservices
aws_session_duration   = 28800
aws_profile            = osnd-preprod
resource_id            =
subdomain              =
role_arn               = arn:aws:iam::837767801504:role/vz-console-soe-administrator
policy_file            =
policy_arn_list        =
region                 =
http_attempts_count    =
http_retry_delay       =
credentials_file       =
saml_cache             = false
saml_cache_file        =
target_url             =
disable_remember_device = false
disable_sessions       = false
download_browser_driver = false
```

```
headless               = false
prompter               =
kc_broker              =
meta_alias             = /employee/VZIDP_28054_JGBV_TPDSOEAWSSOE

[osnd-prod]
name                   = osnd-prod
app_id                 =
url                    = https://ssologin.verizon.com
username               = srivsa
provider               = VerizonSSO
mfa                    = Auto
mfa_ip_address         =
skip_verify            = false
proxy                  = http://server.proxy.vzwcorp.com:9290
timeout                = 0
aws_urn                = urn:amazon:webservices
aws_session_duration   = 28800
aws_profile            = osnd-prod
resource_id            =
subdomain              =
role_arn               = arn:aws:iam::282329364436:role/vz-console-soe-administrator
policy_file            =
policy_arn_list        =
region                 =
http_attempts_count    =
http_retry_delay       =
credentials_file       =
saml_cache             = false
saml_cache_file        =
target_url             =
disable_remember_device = false
disable_sessions       = false
download_browser_driver = false
headless               = false
prompter               =
kc_broker              =
meta_alias             = /employee/VZIDP_28054_JGBV_TPDSOEAWSSOE
```

1. Add to User PATH via PowerShell (No Admin Required)

You do not need Administrator rights or access to the System Control Panel to update your personal User `PATH` variable.

4. Open **PowerShell**.
5. Run the following command to permanently add `C:\Tools` (or wherever your `saml2aws-vz.exe` is located) to your user environment:

```shell
[Environment]::SetEnvironmentVariable("Path", $env:Path + ";C:\tools\saml2aws-vz.win.2026.08.26-vz1", "User")
```

6. Close and reopen PowerShell, then verify:

```shell
saml2aws-vz --version
```

Verification: If it returns a version number (e.g., `2026.08.26-vz1`), the command was successful.

2. One-Time Configuration

Run the one-time configuration command to create your AWS profile configuration:

```shell
saml2aws-vz configure --session-duration=28800 --profile osnd-dev
```

When prompted:

- **AWS Profile:** Press **Enter** (defaults to `saml`).
- **Username:** Type your Verizon SSO username (e.g., `jsmith`).
- **Meta Alias:** Press **Enter** to accept the default TPD SOE integration, unless your platform team specified a custom Meta Alias string for your org.

View configured profiles:

```
Shell
notepad $HOME\.saml2aws
```

3. Configure Corporate Proxy (PowerShell Profile)

Verizon SSO authentication and AWS CLI operations require routing through the corporate network proxy.

4. Open your PowerShell profile in Notepad:

```
Shell
notepad $PROFILE
```

*(If prompted to create a new file, click **Yes**.)*

5. Add the following line to the file, save, and exit:

```
Shell
$env:HTTPS_PROXY = "http://server.proxy.vzwcorp.com:9290"
```

6. Apply the updated profile to your current PowerShell session:

```
Shell
. $PROFILE
```

4. Authenticate & Obtain Credentials

Ensure you are connected to the corporate VPN or on-net before running the login command:

```
Shell
saml2aws-vz login --profile osnd-dev
```

4. Enter your Verizon SSO password when prompted.
5. Approve the DUO Push notification on your mobile device.
6. Use the arrow keys to select your designated AWS IAM Role from the interactive list and press **Enter**.

This step stores temporary AWS credentials (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_SESSION_TOKEN`) into `%USERPROFILE%\.aws\credentials` under the `[saml]` profile.

> **Tip to reduce DUO prompts:** Add the `--cache-saml` flag during subsequent logins within the same session to reuse your active SAML assertion without triggering a new DUO push:
>
> PowerShell

```shell
saml2aws-vz login --profile saml --cache-saml
--duo-mfa-option "Duo Push"
```

Fix Configuration Overwrites for Profiles:

Critical Config Fix.
In your logs, every time you ran saml2aws-vz configure --profile <name>, it printed:

Configuration saved for IDP account: default

Because the --idp-account flag was not specified, each command **overwrote the default account** rather than creating separate profiles.

To fix this, pass -a <profile-name> so each environment gets its own isolated config:

```
Shell
saml2aws-vz configure -a osnd-dev --profile osnd-dev --session-duration=28800

saml2aws-vz configure -a osnd-test --profile osnd-test --session-duration=28800

saml2aws-vz configure -a osnd-preprod --profile osnd-preprod
--session-duration=28800

saml2aws-vz configure -a osnd-prod --profile osnd-prod --session-duration=28800
```

Verification: Check %USERPROFILE%\.saml2aws in Notepad to ensure four separate [osnd-dev], [osnd-test], [osnd-preprod], and [osnd-prod] sections exist.

Log In to Target Environment:

Authenticate.
Once VPN is active and the proxy is loaded in your current terminal session, run login with matching IDP account and profile flags:

PowerShell

```
None
saml2aws-vz login -a osnd-test --profile osnd-test
```

Verification: You will be prompted for your password, receive a DUO push on your phone, and see your temporary access keys saved to %USERPROFILE%\.aws\credentials.

connect to ec2 from powershell Windows local system

Use User-Level CA Export (Zero Admin Rights Needed)

Because you have read permissions to your own user certificate store, you can export Windows root certificates directly into your user home folder (`$HOME`) and tell AWS CLI to use that bundle. This satisfies SSL verification and prevents the warning from being generated in the first place.

Run this once in standard PowerShell to generate your user bundle

Shell

```
$bundlePath = "$HOME\corporate-ca-bundle.pem"

Remove-Item $bundlePath -ErrorAction SilentlyContinue



# Export all root CAs available to your current user session

Get-ChildItem -Path Cert:\LocalMachine\Root, Cert:\CurrentUser\Root |
ForEach-Object {

    $bytes =
$_.Export([System.Security.Cryptography.X509Certificates.X509ContentType]:
:Cert)

    $base64 = [System.Convert]::ToBase64String($bytes,
[System.Base64FormattingOptions]::InsertLineBreaks)

    Add-Content -Path $bundlePath -Value "-----BEGIN
CERTIFICATE-----`n$base64`n-----END CERTIFICATE-----"

}
```

Shell

```
PS C:\windows\system32> aws configure set default.ca_bundle
"$HOME\corporate-ca-bundle.pem"

PS C:\windows\system32> Get-Content "$HOME\.aws\config"
[default]
ca_bundle = C:\Users\srivsa\corporate-ca-bundle.pem
```

```
PS C:\windows\system32> notepad "$HOME\.aws\config"
```

Add below content into config file

```
[default]
ca_bundle = C:\Users\srivsa\corporate-ca-bundle.pem

[osnd-dev]
ca_bundle = C:\Users\srivsa\corporate-ca-bundle.pem

[osnd-test]
ca_bundle = C:\Users\srivsa\corporate-ca-bundle.pem

[osnd-preprod]
ca_bundle = C:\Users\srivsa\corporate-ca-bundle.pem

[osnd-prod]
ca_bundle = C:\Users\srivsa\corporate-ca-bundle.pem
```

Here is the clean setup using the PowerShell `$PROFILE` helper functions approach. This keeps your Windows Terminal `settings.json` lightweight, readable, and easy to maintain.

Step 1: Add Functions to Your Local PowerShell `$PROFILE`

Open your PowerShell profile on your local Windows machine:

PowerShell

```
None
notepad $PROFILE
```

Paste the following suite of helper functions (including your updated Dev and all other environments/regions):

```
Shell
# Common helper function to display menu and connect
function Invoke-Ec2Session {
    param(
```

```
        [string]$EnvName,
        [string]$Profile,
        [string]$Region,
        [ConsoleColor]$HeaderColor,
        [array]$Instances
    )

    $env:AWS_CA_BUNDLE = "$HOME\corporate-ca-bundle.pem"
    chcp 65001 | Out-Null
    [Console]::OutputEncoding = [Console]::InputEncoding = [System.Text.Encoding]::UTF8
    $env:AWS_PROFILE = $Profile
    $env:AWS_DEFAULT_REGION = $Region

    Clear-Host
    Write-Host "=== $EnvName [$Region] INSTANCE SELECTOR ===" -ForegroundColor $HeaderColor

    for ($i = 0; $i -lt $Instances.Count; $i++) {
        Write-Host ("  [{0}] {1,-40} ({2})" -f ($i + 1), $Instances[$i].Name,
$Instances[$i].Id)
    }

    Write-Host ""
    $choice = Read-Host "Select instance [1-$($Instances.Count)] (Default: 1)"
    if (-not $choice) { $choice = 1 }

    $target = $Instances[[int]$choice - 1].Id
    Write-Host "Connecting to $target as ec2-user..." -ForegroundColor Green

    # Launch session and run 'sudo su - ec2-user' directly
    aws ssm start-session `
        --target $target `
        --document-name "AWS-StartInteractiveCommand" `
        --parameters 'command="sudo su - ec2-user"'
}


# --- 1. DEV (us-east-1) ---
function Connect-DevEast1 {
    $instances = @(
        @{ Id = "i-0aac15b5be76bb4bc"; Name = "pm-ec2-eks-mgmt-dev (Default)" },
        @{ Id = "i-046d2f81f6143dd8c"; Name = "pm-osltespog-eks-dev-ng (Node 1)" },
        @{ Id = "i-0818544251c3c9b50"; Name = "pm-osltespog-eks-dev-ng (Node 2)" }
    )
    Invoke-Ec2Session -EnvName "DEV" -Profile "osnd-dev" -Region "us-east-1" -HeaderColor
Cyan -Instances $instances
}

# --- 2. TEST (us-east-1) ---
function Connect-TestEast1 {
    $instances = @(
```

```
        @{ Id = "i-080caacb8bd4149ab"; Name = "pm-ec2-eks-mgmt-test (Default)" },
        @{ Id = "i-088cbf7b106d32ca0"; Name = "pm-osltespog-eks-test-ng (Node 1)" },
        @{ Id = "i-08a82ecdaaeb64e82"; Name = "pm-osltespog-eks-test-ng (Node 2)" },
        @{ Id = "i-0429b75ada9512470"; Name = "pm-osltespog-eks-test-ng (Node 3)" }
    )
    Invoke-Ec2Session -EnvName "TEST" -Profile "osnd-test" -Region "us-east-1" -HeaderColor
Yellow -Instances $instances
}

# --- 3. PREPROD (us-east-1) ---
function Connect-PreprodEast1 {
    $instances = @(
        @{ Id = "i-022ad7e93cd8fb7c8"; Name = "pm-ec2-eks-mgmt-pre-production (Default)" },
        @{ Id = "i-0d4807b6d3bac03aa"; Name = "pm-osltespog-eks-preprod-ng (Node 1)" },
        @{ Id = "i-0c8d9a47643795174"; Name = "pm-osltespog-eks-preprod-ng (Node 2)" },
        @{ Id = "i-00df6714ae45bc92c"; Name = "pm-osltespog-eks-preprod-ng (Node 3)" }
    )
    Invoke-Ec2Session -EnvName "PREPROD" -Profile "osnd-preprod" -Region "us-east-1"
-HeaderColor Magenta -Instances $instances
}

# --- 4. PREPROD (us-west-2) ---
function Connect-PreprodWest2 {
    $instances = @(
        @{ Id = "i-065cf481db2712ab9"; Name = "pm-ec2-eks-mgmt-pre-production (Default)" },
        @{ Id = "i-0a11f5db05119b368"; Name = "pm-osltespog-eks-preprod-ng (Node 1)" },
        @{ Id = "i-03d5831d5b0620c73"; Name = "pm-osltespog-eks-preprod-ng (Node 2)" },
        @{ Id = "i-0c96fea2607e669fb"; Name = "pm-osltespog-eks-preprod-ng (Node 3)" }
    )
    Invoke-Ec2Session -EnvName "PREPROD" -Profile "osnd-preprod" -Region "us-west-2"
-HeaderColor Magenta -Instances $instances
}

# --- 5. PROD (us-east-1) ---
function Connect-ProdEast1 {
    $instances = @(
        @{ Id = "i-0d1b9011288cb5825"; Name = "pm-ec2-eks-mgmt-prod (Default)" },
        @{ Id = "i-01992d6852e2e5af5"; Name = "pm-osltespog-eks-prod-ng (Node 1)" },
        @{ Id = "i-0c7f29750b1df8072"; Name = "pm-osltespog-eks-prod-ng (Node 2)" },
        @{ Id = "i-0e0182b372a7aa627"; Name = "pm-osltespog-eks-prod-ng (Node 3)" }
    )
    Invoke-Ec2Session -EnvName "PROD" -Profile "osnd-prod" -Region "us-east-1" -HeaderColor
Red -Instances $instances
}

# --- 6. PROD (us-west-2) ---
function Connect-ProdWest2 {
    $instances = @(
        @{ Id = "i-084e56e564ad38bb0"; Name = "pm-ec2-eks-mgmt-prod (Default)" },
        @{ Id = "i-0ec93f8fab42dd50b"; Name = "pm-osltespog-eks-prod-ng (Node 1)" },
```

```
        @{ Id = "i-0a3108c662e2e98f2"; Name = "pm-osltespog-eks-prod-ng (Node 2)" },
        @{ Id = "i-02af57c877245bb4c"; Name = "pm-osltespog-eks-prod-ng (Node 3)" }
    )
    Invoke-Ec2Session -EnvName "PROD" -Profile "osnd-prod" -Region "us-west-2" -HeaderColor
Red -Instances $instances
}
```

Save and close Notepad.

Step 2: Update Windows Terminal `settings.json` Profiles

Open `settings.json` (`Ctrl + Shift + ,`). Update the `"profiles" -> "list"` entries to call these functions directly via their `commandline`:

```
JSON
{
    "commandline": "powershell.exe -NoExit -Command \"Connect-DevEast1\"",
    "guid": "{0ddbd457-7185-576b-9010-ef97541b8925}",
    "name": "AWS - DEV (us-east-1)",
    "suppressApplicationTitle": true,
    "tabColor": "#38BDF8",
    "tabTitle": "[DEV] us-east-1"
},
{
    "commandline": "powershell.exe -NoExit -Command \"Connect-TestEast1\"",
    "guid": "{8d25a6ec-a3a9-5815-85e1-c19f939c5c8d}",
    "name": "AWS - TEST (us-east-1)",
    "suppressApplicationTitle": true,
    "tabColor": "#FACC15",
    "tabTitle": "[TEST] us-east-1"
},
{
    "commandline": "powershell.exe -NoExit -Command \"Connect-PreprodEast1\"",
    "guid": "{0c38da38-5c01-5164-9c6f-fa9261462488}",
    "name": "AWS - PREPROD (us-east-1)",
    "suppressApplicationTitle": true,
    "tabColor": "#C084FC",
    "tabTitle": "[PREPROD] us-east-1"
},
{
    "commandline": "powershell.exe -NoExit -Command \"Connect-PreprodWest2\"",
    "guid": "{99a5a3d6-71cb-5494-bc7e-40b9f772d131}",
    "name": "AWS - PREPROD (us-west-2)",
    "suppressApplicationTitle": true,
    "tabColor": "#A855F7",
    "tabTitle": "[PREPROD] us-west-2"
},
```

```
{
    "commandline": "powershell.exe -NoExit -Command \"Connect-ProdEast1\"",
    "guid": "{5a357215-1a62-557a-9eaa-f1b37bf68294}",
    "name": "AWS - PROD (us-east-1)",
    "suppressApplicationTitle": true,
    "tabColor": "#EF4444",
    "tabTitle": "[PROD] us-east-1"
},
{
    "commandline": "powershell.exe -NoExit -Command \"Connect-ProdWest2\"",
    "guid": "{b1d0b991-75d4-5da2-8dbc-177a45f6c497}",
    "name": "AWS - PROD (us-west-2)",
    "suppressApplicationTitle": true,
    "tabColor": "#DC2626",
    "tabTitle": "[PROD] us-west-2"
}
```

Save `settings.json`.

Step 3: Usage

Press any shortcut (`Ctrl + Shift + 1` to `6`):

- The corresponding tab opens with the exact color theme.
- You are presented with the numbered list of instances for that environment/region.
- Press **Enter** to connect directly to the **Default management instance**, or type `2`, `3`, or `4` to connect to a node.
- Encoding is locked to UTF-8 (`chcp 65001`), so running `k9s` will render clean table borders.

If ssl verification fails, then use the below block. It uses "**--no-verify-ssl**"

Shell

```
# Common helper function to display menu and connect
function Invoke-Ec2Session {
    param(
        [string]$EnvName,
        [string]$Profile,
        [string]$Region,
        [ConsoleColor]$HeaderColor,
        [array]$Instances
    )

    $env:PYTHONWARNINGS = 'ignore'
```

```
    chcp 65001 | Out-Null
    [Console]::OutputEncoding = [Console]::InputEncoding =
[System.Text.Encoding]::UTF8
    $env:AWS_PROFILE = $Profile
    $env:AWS_DEFAULT_REGION = $Region

    Clear-Host
    Write-Host "=== $EnvName [$Region] INSTANCE SELECTOR ===" -ForegroundColor
$HeaderColor

    for ($i = 0; $i -lt $Instances.Count; $i++) {
        Write-Host ("  [{0}] {1,-40} ({2})" -f ($i + 1), $Instances[$i].Name,
$Instances[$i].Id)
    }

    Write-Host ""
    $choice = Read-Host "Select instance [1-$($Instances.Count)] (Default: 1)"
    if (-not $choice) { $choice = 1 }

    $target = $Instances[[int]$choice - 1].Id
    Write-Host "Connecting to $target..." -ForegroundColor Green
    aws ssm start-session --target $target --no-verify-ssl
}
```

Complete settings.json file

JSON

```
{
    "$help": "https://aka.ms/terminal-documentation",
    "$schema": "https://aka.ms/terminal-profiles-schema",
"actions":
    [
        {
            "command":
            {
                "action": "newTab",
                "profile": "AWS - DEV (us-east-1)"
            },
            "id": "User.newTab.DevEast1"
        },
        {
            "command":
            {
```

```
            "action": "newTab",
            "profile": "AWS - TEST (us-east-1)"
        },
        "id": "User.newTab.TestEast1"
    },
    {
        "command":
        {
            "action": "newTab",
            "profile": "AWS - PREPROD (us-east-1)"
        },
        "id": "User.newTab.PreprodEast1"
    },
    {
        "command":
        {
            "action": "newTab",
            "profile": "AWS - PREPROD (us-west-2)"
        },
        "id": "User.newTab.PreprodWest2"
    },
    {
        "command":
        {
            "action": "newTab",
            "profile": "AWS - PROD (us-east-1)"
        },
        "id": "User.newTab.ProdEast1"
    },
    {
        "command":
        {
            "action": "newTab",
            "profile": "AWS - PROD (us-west-2)"
        },
        "id": "User.newTab.ProdWest2"
    },
    {
        "command":
        {
            "action": "splitPane",
            "profile": "AWS - PROD (us-east-1)",
            "split": "right"
        },
        "id": "User.splitPane.ProdEast1"
```

```
    }
],
"copyFormatting": "none",
"copyOnSelect": false,
"defaultProfile": "{61c54bbd-c2c6-5271-96e7-009a87ff44bf}",
"keybindings":
[
    {
        "id": "Terminal.CopyToClipboard",
        "keys": "ctrl+c"
    },
    {
        "id": "Terminal.PasteFromClipboard",
        "keys": "ctrl+v"
    },
    {
        "id": "Terminal.FindText",
        "keys": "ctrl+shift+f"
    },
    {
        "id": "Terminal.DuplicatePaneAuto",
        "keys": "alt+shift+d"
    },
    {
        "id": "User.newTab.DevEast1",
        "keys": "ctrl+shift+1"
    },
    {
        "id": "User.newTab.TestEast1",
        "keys": "ctrl+shift+2"
    },
    {
        "id": "User.newTab.PreprodEast1",
        "keys": "ctrl+shift+3"
    },
    {
        "id": "User.newTab.PreprodWest2",
        "keys": "ctrl+shift+4"
    },
    {
        "id": "User.newTab.ProdEast1",
        "keys": "ctrl+shift+5"
    },
    {
        "id": "User.newTab.ProdWest2",
```

```
            "keys": "ctrl+shift+6"
        },
        {
            "id": "User.splitPane.ProdEast1",
            "keys": "alt+shift+p"
        }
    ],
    "newTabMenu":
    [
        {
            "type": "remainingProfiles"
        }
    ],
    "profiles":
    {
        "defaults": {},
        "list":
        [
            {
                "commandline":
"%SystemRoot%\\System32\\WindowsPowerShell\\v1.0\\powershell.exe",
                "guid": "{61c54bbd-c2c6-5271-96e7-009a87ff44bf}",
                "hidden": false,
                "name": "Windows PowerShell"
            },
            {
                "commandline": "%SystemRoot%\\System32\\cmd.exe",
                "guid": "{0caa0dad-35be-5f56-a8ff-afceeeaa6101}",
                "hidden": false,
                "name": "Command Prompt"
            },
            {
                "guid": "{b453ae62-4e3d-5e58-b989-0a998ec441b8}",
                "hidden": false,
                "name": "Azure Cloud Shell",
                "source": "Windows.Terminal.Azure"
            },
            {
    "commandline": "powershell.exe -NoExit -Command \"Connect-DevEast1\"",
    "guid": "{0ddbd457-7185-576b-9010-ef97541b8925}",
    "name": "AWS - DEV (us-east-1)",
    "suppressApplicationTitle": true,
    "tabColor": "#38BDF8",
    "tabTitle": "[DEV] us-east-1"
},
```

```
{
    "commandline": "powershell.exe -NoExit -Command \"Connect-TestEast1\"",
    "guid": "{8d25a6ec-a3a9-5815-85e1-c19f939c5c8d}",
    "name": "AWS - TEST (us-east-1)",
    "suppressApplicationTitle": true,
    "tabColor": "#FACC15",
    "tabTitle": "[TEST] us-east-1"
},
{
    "commandline": "powershell.exe -NoExit -Command \"Connect-PreprodEast1\"",
    "guid": "{0c38da38-5c01-5164-9c6f-fa9261462488}",
    "name": "AWS - PREPROD (us-east-1)",
    "suppressApplicationTitle": true,
    "tabColor": "#C084FC",
    "tabTitle": "[PREPROD] us-east-1"
},
{
    "commandline": "powershell.exe -NoExit -Command \"Connect-PreprodWest2\"",
    "guid": "{99a5a3d6-71cb-5494-bc7e-40b9f772d131}",
    "name": "AWS - PREPROD (us-west-2)",
    "suppressApplicationTitle": true,
    "tabColor": "#A855F7",
    "tabTitle": "[PREPROD] us-west-2"
},
{
    "commandline": "powershell.exe -NoExit -Command \"Connect-ProdEast1\"",
    "guid": "{5a357215-1a62-557a-9eaa-f1b37bf68294}",
    "name": "AWS - PROD (us-east-1)",
    "suppressApplicationTitle": true,
    "tabColor": "#EF4444",
    "tabTitle": "[PROD] us-east-1"
},
{
    "commandline": "powershell.exe -NoExit -Command \"Connect-ProdWest2\"",
    "guid": "{b1d0b991-75d4-5da2-8dbc-177a45f6c497}",
    "name": "AWS - PROD (us-west-2)",
    "suppressApplicationTitle": true,
    "tabColor": "#DC2626",
    "tabTitle": "[PROD] us-west-2"
}
        ]
    },
    "schemes": [],
    "themes": []
}
```

To open multiple panes (split panes) in Windows Terminal, you can use built-in keyboard shortcuts, UI clicks, automated startup layouts, or command-line commands.

1. Default Keyboard Shortcuts

Inside any active Windows Terminal window:

- **Split vertically (side-by-side):** `Alt + Shift + Plus (+)` (or `Alt + Shift + =`)
- **Split horizontally (top-and-bottom):** `Alt + Shift + Minus (-)`
- **Duplicate current profile into auto-split pane:** `Alt + Shift + D`
- **Close focused pane:** `Ctrl + Shift + W`
- **Switch between panes:** `Alt + Arrow Keys`
- **Resize active pane:** `Alt + Shift + Arrow Keys`

2. Alt-Clicking the UI

Hold down the **`Alt`** key on your keyboard and click:

- **The `+` (New Tab) button:** Automatically splits the current pane vertically with your default profile.
- **Any profile in the dropdown menu (chevron icon):** Splits the active pane and launches that specific environment (e.g., dev or prod).

3. Dedicated Split-Pane Shortcuts in `settings.json`

You already have an action defined for `User.splitPane.ProdEast1` with `alt+shift+p`. You can add more split shortcuts inside the `"actions"` and `"keybindings"` arrays:

In `"actions"`:
JSON

```
None
{
  "command": {
    "action": "splitPane",
    "profile": "AWS - DEV (us-east-1)",
    "split": "right"
  },
  "id": "User.splitPane.DevEast1"
},
```

```
{
  "command": {
    "action": "splitPane",
    "profile": "AWS - TEST (us-east-1)",
    "split": "down"
  },
  "id": "User.splitPane.TestEast1"
}
```

In `"keybindings"`:
JSON

```
None
{
  "id": "User.splitPane.DevEast1",
  "keys": "alt+shift+1"
},
{
  "id": "User.splitPane.TestEast1",
  "keys": "alt+shift+2"
}
```

4. Open Multiple Panes at Once via CLI / Shortcut (`wt.exe`)

You can launch a multi-pane layout directly from PowerShell, Run (`Win + R`), or a `.bat` script using `wt.exe`:

Side-by-Side (2 Panes: Dev on Left, Test on Right):
PowerShell

```
None
wt.exe -p "AWS - DEV (us-east-1)" `; split-pane -H -p "AWS - TEST
(us-east-1)"
```

(Note: In `wt.exe`, `-H` splits horizontally across the screen side-by-side; `-V` splits vertically top/bottom).

2x2 Grid (4 Panes: Dev, Test, Preprod, Prod):

PowerShell

```
None
wt.exe -p "AWS - DEV (us-east-1)" `; split-pane -H -p "AWS - TEST
(us-east-1)" `; move-focus left `; split-pane -V -p "AWS - PREPROD
(us-east-1)" `; move-focus right `; split-pane -V -p "AWS - PROD
(us-east-1)"
```

5. Broadcast Input to All Panes Simultaneously

To run the same command across all open panes at the same time:

1. Press `Ctrl + Shift + P` to open the Command Palette.
2. Type **"Toggle broadcast input to all panes"** and hit Enter.
3. A red border or warning indicator will appear around active panes; anything typed in one pane will execute across all of them. Use the same command to toggle it off.